

Sistemi di Software a Oggetti

[[0 - Index]]

Mail: sistemidisoftwhareaoggetti@live.unibo.it

Un linguaggio a oggetti è necessario per gestire problemi di ampiezza superiore e sistemi software complessi.

Nuovi Concetti

I linguaggi moderni nascono per risolvere i limiti dell'approccio procedurale (programmazione *in-the-small*) e introducono nuovi concetti per la progettazione *in-the-large*:

- **Novità rispetto al C:** L'obiettivo è sostituire costrutti linguistici obsoleti e poco chiari, e intercettare a compile-time quanti più errori possibile per garantire maggiore sicurezza.
- **Tutto è un oggetto:** Abolizione (o **mascheramento**) dei **tipi primitivi** in favore di un approccio in cui ogni entità è un oggetto ("**everything is an object**").
- **Immutabilità:** Distinzione netta tra variabili modificabili e valori non modificabili, con una preferenza per le **strutture dati immutabili** ("compute by synthesis") per garantire maggiore sicurezza.

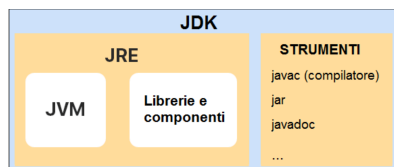
L'Infrastruttura di Java (JVM, JRE, JDK)

L'obiettivo principale dell'infrastruttura Java è massimizzare la portabilità del software, realizzando il sogno del "write once, run everywhere" (scrivi una volta, esegui ovunque).

Il Processo di Compilazione

- Il **compilatore** (`javac`) non genera codice macchina, ma traduce il file sorgente `.java` in un formato intermedio universale chiamato **bytecode** (file `.class`).
- **Linking Dinamico:** Durante la compilazione le librerie vengono caricate e collegate dinamicamente (a run-time) solo quando servono.

I Livelli dell'Infrastruttura Java



Per far funzionare il bytecode su un computer fisico, serve uno "strato-ponte" che interpreti il formato portabile e lo adatti allo specifico sistema. L'infrastruttura si divide in **tre livelli concentrici**:

JVM (Java Virtual Machine)

- Si occupa di astrarre l'hardware e il sistema operativo sottostante.
- È **l'unico strato dipendente dalla piattaforma**: esiste una JVM specifica per Windows, una per Mac, una per Linux, ecc., ma tutte eseguono lo stesso identico bytecode.

JRE (Java Runtime Environment)

- È l'ambiente necessario per **eseguire** le applicazioni Java.
- È composto da: **JVM + Librerie e Componenti standard** di Java.

JDK (Java Development Kit)

- È il pacchetto completo necessario per **sviluppare** applicazioni Java.
- Contiene l'intero **JRE** (per poter testare il codice) più una serie di **Strumenti di Sviluppo (Tools)**.
- Tra gli strumenti inclusi ci sono:
 - `javac` (il compilatore)
 - `jar` (per impacchettare le applicazioni)
 - `javadoc` (per generare la documentazione automatica)

Infrastruttura e Portabilità (La JVM):

Java è pensato per essere estremamente versatile e portabile.

- Perché il codice possa funzionare su diversi processori, la compilazione non produce un eseguibile di sistema (come in C), ma un formato intermedio (**bytecode**).
- Questo bytecode viene eseguito dalla **JVM (Java Virtual Machine)**, che si occupa di tradurlo per lo specifico processore.
- I collegamenti alle librerie (**linker**) vengono risolti **a runtime** (dinamicamente): questo rende **l'avvio leggermente più lento rispetto al C**, ma rende il programma estremamente flessibile e capace di gestire **aggiornamenti** automatici delle librerie.

Compilazione ed esecuzione da terminale

L'eseguibile in Java non è un file monolitico (`.exe`) ma un **archivio zippato** `.jar` (Java ARchive) (non va estratto).

- Per **librerie**, basta impacchettare le classi (file `.class`).
- Per **applicazioni eseguibili**, occorre specificare la classe contenente il main attraverso un file descrittore `MANIFEST.MF` (creabile passando un file di testo come `info.txt`).

Compilazione ed Esecuzione Locale

1. **Compilazione:** Trasforma il file sorgente in bytecode (`.class`).

```
1 javac MyProg.java
2
3 javac -cp Lib.jar MyProg.java //se contiene librerie
```

Risultato: Viene creato il file `MyProg.class` . Se ci sono altre classi nel progetto, puoi compilarle tutte insieme con `javac *.java` .

2. **Esecuzione semplice:** Avvia la Java Virtual Machine.

```
1 java MyProg.class
```

Creare una Libreria JAR (Non eseguibile)

Se hai sviluppato delle classi di utilità e vuoi distribuirle come libreria.

```
1 jar cf MiaLibreria.jar *.class
```

- `c` sta per *create* (crea un nuovo archivio).
- `f` sta per *file* (specifica il nome del file jar da creare).

Creare un'Applicazione JAR Eseguitibile

Per rendere un `.jar` eseguibile con un doppio clic o da terminale, la JVM deve sapere qual è la classe che contiene il metodo `main` . Questo si fa usando un file manifesto (`MANIFEST.MF`).

- **Usando un file di testo**

1. Crea un normale file di testo chiamato `info.txt` .
2. Scrivici dentro questa singola riga (assicurati di andare a capo alla fine):

```
Main-Class: MyProg
```

3. Lancia il comando per creare il jar iniettando il manifesto:

```
1 jar cmf info.txt AppEseguitibile.jar *.class //o lista delle classi
```

(`m` sta per *manifest*).

- **Comando diretto**

Puoi dire direttamente a Java quale classe è l'entry-point (la `e` sta per *entry point*):

```
1 jar cef AppEseguitibile.jar MyProg *.class //o lista delle classi
2 //verboso
3 jar --create --main-class NomeclasseMain --file nomeapp.jar classi
```

Eseguire il file JAR

```
1 java -jar AppEseguitibile.jar
2
3 java -cp _/Lib.jar;. MyMain //se contiene librerie
```

Se c'è una **libreria** nel file `MANIFEST.MF` bisogna **aggiungere** `Class-Path: ../Lib.jar`

(Se il programma prevede argomenti da terminale, aggiungili alla fine, es: `java -jar AppEseguitibile.jar argomentiVari`).

Struttura del Codice e Gestione Memoria

Le operazioni

Si inverte il punto di vista rispetto al C, passando dal focus sull'operazione al focus sull'entità.

```
1 // Approccio procedurale C
2 operation(comp, argomenti);
3 fprintf(fout, "Hello!");
```

diventa:

```
1 // Approccio a oggetti Java
```

```
2 comp.operation(argumenti);
```

- **comp** = a CHI mi rivolgo
- **operation** = COSA gli chiedo di fare

Elementi di base e Memoria

- **Entità Statiche:** Esistono prima dell'inizio del programma e permangono per tutta la sua durata (es. moduli, libreria matematica) -> **CLASSI**.
- **Entità Dinamiche:** Vengono create durante l'esecuzione solo al momento del bisogno -> **OGGETTI**.
- **Gestione della memoria:**
 - Sostituzione dei puntatori del C con i **riferimenti**.
 - Dereferenzamento automatico.
 - **Allocazione e deallocazione automatica** della memoria tramite il **Garbage Collector**.
 - **Type safety** = type system stringente + controlli a run-time per prevenire crash.

Convenzioni di Naming

- **Classi e File:** Il nome della classe deve iniziare con la maiuscola (*CamelCase*). Il file **deve** chiamarsi esattamente come la classe pubblica che contiene con estensione `.java`.
- **Metodi e Variabili:** Iniziano con la minuscola (*camelCase*). Le costanti sono tutte in MAIUSCOLO.

Il Main e l'IO

Il punto di partenza dell'applicazione è il `main`. In Java classico è contenuto in una classe pubblica dedicata.

Il tipo di **ritorno** è sempre `void` e ha **sempre** come argomento un array di stringhe (`String[] args`).

`static` significa che il metodo esiste dall'inizio, indipendentemente dalla creazione di oggetti.

```
1 public class MyProg {
2     public static void main(String[] args){
3         // In Java, args[0] è il primo argomento passato dall'utente, non il nome del programma
4         if (args.length > 0) {
5             System.out.println("Primo argomento: " + args[0]);
6         }
7     }
8 }
```

Il Main semplificato

Per alleggerire la sintassi, Java 25 permette di omettere la classe contenitore, `public`, `static` e persino l'argomento `String[] args`. Introduce anche il modulo `IO` per semplificare stampe e letture.

```
1 // Il file si chiama App.java (la classe è generata in automatico)
2 import static java.lang.IO.*;
3
4 void main() {
5     println("Hello!"); // Sostituisce System.out.println
6     String nome = readln("Come ti chiami? "); // Sostituisce l'uso complesso di System.in
7 }
```

Tipi di Base, Conversioni e Classi Wrapper

I tipi base definiscono la dimensione e i limiti dei dati:

- **Boolean:** separato dagli interi, ammette solo `false` e `true` (non 0 e 1).
- **Int** (le diverse lunghezze vengono standardizzate e la sintassi semplificata)
 - **byte** (1 byte) -128 ... +127
 - **short** (2 byte) -32768 ... +32767
 - **int** (4 byte) -2 10⁹ +2 10⁹
 - **long** (8 byte) -9 10¹⁸ +9 10¹⁸

Nota errori aritmetici intera: L'overflow intero non genera errori bloccanti ma "sborda" nei numeri negativi (wrap-around). La **divisione per zero** tra interi genera un'eccezione bloccante `ArithmeticException` (esplode a runtime).
- **Float**
 - **float** (4 byte) - 10⁴⁵ ... +10³⁸
 - **double** (8 byte) -10³²⁴ ... +10³⁰⁸
 - **Caratteri:** `char` (2 byte, Unicode/UTF-16). Permette di usare caratteri speciali tramite codifica esadecimale (es. `\u00B1` per `±`).

Nota errori aritmetici reali: La divisione per `0.0` non causa crash ma restituisce `Infinity` o `-Infinity`. Le forme indeterminate (es. `0.0/0.0`) restituiscono `NaN` (Not a Number).
- **Caratteri** 127 caratteri non bastano quindi si assegnano più byte a ogni carattere 1-4 (UTF-8)(1.114.112 caratteri)

Piano	Intervallo	Descrizione
0	00000-00FFFF	Basic Multilingual
1	010000-01FFFF	Supplementary Multilingual
2	020000-02FFFF	Supplementary Ideographic
3	030000-03FFFF	Tertiary Ideographic
...	040000-0DFFFF	non usati
14	0E0000-0EFFFF	Supplementary Special-purpose
15	0F0000-0FFFFF	Supplementary Private Use Area A
16	100000-10FFFF	Supplementary Private Use Area B

C'è ancora la corrispondenza diretta fra caratteri e numeri come in C.

```
1 char ch = 'A';
2 ch = 72;
3 char cStran = '\u2122'
```

- **Var** (tipo generico) permette l'inferenza del tipo deducendolo dal contesto --> snellisce la sintassi ma va usato solo quando è facile capire il tipo

Assegnamenti e Cast:

Sono ammessi implicitamente solo gli assegnamenti che non generano una perdita di informazioni.

```
1 double x = 3.54F; // Permessso (nessuna perdita)
2 // float f = 3.54; // DÀ ERRORE (double in float perde precisione)
3 float f = (float) 3.54; // Corretto: forzatura tramite CAST esplicito
```

Classi, Librerie e Wrapper

Le classi **partizionano** lo spazio dei **nomi** (possono esistere nomi uguali in classi diverse).

I tipi primitivi in Java non sono oggetti e non hanno metodi diretti. Per operazioni avanzate si usano le Classi di utilità (Wrapper e Librerie):

- **Math:** Libreria statica per calcoli matematici. Es. `Math.sin(Math.PI/3)` , `Math.sqrt(x)` . Funzione utile: `Math.hypot(x,y)` calcola $x^2 + y^2$ prevenendo overflow/underflow intermedi.
- **Character:** Es. `Character.toUpperCase(ch)` , `Character.isWhitespace(ch)` , `Character.digit(ch, base)` (per ottenere il valore numerico del carattere).
- **Integer:** Es. `Integer.signum(n)` , `Integer.rotateLeft()` .

Documentazione (Javadoc)

La documentazione è generata automaticamente a patto di usare una sintassi specifica.

```
1 /** * Javadoc Classico
2 * @author Mario Rossi
3 * @version 1.0
4 */
```

In Java 23+ si può usare il **Markdown** antepoendo `///` a ogni riga.

Generazione automatica manuale HTML: `javadoc -d docs NomeFile.java` .

Componenti Software in C

Tipi di Componenti in C

- **Librerie (senza stato):** Solo funzioni matematiche o di utilità (header + implementazione).
- **Moduli Singleton (con stato):** Variabile globale protetta con `static` . Utile ma unico per intero progetto.
- **ADT (Tipi di Dato Astratti):** Creati con `typedef` , permettono la creazione di istanze multiple (es. Liste, Frazioni).

Dichiarazioni Vuote

- **Problema:** Scrivere `int getValue();` in C indica "argomenti sconosciuti", disabilitando i controlli di tipo del compilatore.
- **Soluzione:** Usare sempre `void` per vietare i parametri: `int getValue(void);` .

Limiti Pratici del C

- **Singleton -> Non Scalabile:** Ottimo incapsulamento (stato nascosto), ma non è possibile istanziarlo più di una volta senza fare brutali "copia-incolla" del codice.
- **ADT -> Incapsulamento Violato:** Permette istanze multiple, ma per funzionare richiede che la `typedef` sia visibile a tutti nel file `.h` . Chiunque può leggere/modificare i dati interni bypassando le funzioni.
- **I Puntatori** sono intrinsecamente pericolosi perchè danno accesso alla memoria senza limiti.

Costruzione e Collaudo

- **Costruzione Fragile:** In C l'allocazione (`malloc`) e l'inizializzazione (`init`) sono fasi staccate. Dimenticarne una causa malfunzionamenti.
- **Pattern "Make":** Si aggira il problema unendo i due step in un'unica funzione `make()` (che prefigura i "costruttori" dei linguaggi a oggetti).
- **Collaudo (Testing):** Abbandono dei `printf` manuali a favore delle **asserzioni** (`<assert.h>`) , per verificare in automatico i risultati attesi. (es. `assert(getValue() == 1);`)

Classi e Oggetti

seguendo l'esempio di un contatore:

Singleton

In C

```
1 void reset(void);
2 void inc(void);
3 int getValue(void);
4
5 static int stato;
6 void reset(){stato=0;}
7 void inc(){stato++;}
8 int getValue(){return stato;}
```

in JAVA

```
1 public class Contatore {
2     private static int stato;
3     public static void reset(){stato=0;}
4     public static void inc(){stato++;}
5     public static int getValue(){return stato;}
6 }
7
8 //main
9 public class MyMain {
10     public static void main(String[] args) {
11         Contatore.reset();
12         Contatore.inc();
13         assert Contatore.getValue()==1;
14         System.out.println(Contatore.getValue()==1);
15     }
16 }
```

Contatore

- Il file **header** viene **eliminato** (le **parti pubbliche** di ogni classe le sostituiscono)
- il livello di protezione (**privato/pubblico**) è espresso **esplicitamente**
 - le **funzioni** sono **pubbliche** perchè tutti le usino
 - lo **stato** è **privato** per **proteggere** il valore
- è tutto **statico** perchè la memoria **non** è allocata **dinamicamente**
- Main
- non occorre creare esplicitamente il contatore perchè esso coincide con la **classe**, che è un'entità **statica e preesistente (esiste solo un contatore)**
- non occorre alcuna include: le **classi vengono caricate al momento del bisogno**, cercandole nel classpath
- Collaudo
- il collaudo è nativo tramite la funzione assert (il programma restituisce errore se la condizione non si verifica)

In C:

- ogni sorgente include dichiarazioni
- si compila ogni file sorgente
- si collegano (*link*) i file oggetto
- si crea un *eseguibile* che non contiene più riferimenti esterni
- *max efficienza & max rigidità: ogni modifica implica il rebuild dell'intero eseguibile* e la sua redistribuzione

In tale schema:

- il compilatore accetta l'uso delle entità esterne "con riserva"
- il linker verifica la presenza delle definizioni risolvendo i riferimenti incrociati fra i file

In Java & co.:

- **non esistono dichiarazioni**
- si compilano le singole classi
- **non si collegano staticamente le classi**
- **non esiste più l'eseguibile monolitico:** l'applicazione è un pacchetto di classi → **si esegue quella contenente il main**
- max flessibilità: in caso di modifiche, **basta ricompilare la classe modificata**

In questo nuovo schema:

- il compilatore verifica subito il corretto uso delle altre classi (perché *sa dove trovarle nel file system*)
- le classi vengono *caricate e collegate solo al momento dell'uso*

L'ADT

In C

```
1 typedef struct {int value;} contatore;
2 void reset(contatore*);
3 void inc(contatore*);
4 int getValue(contatore);
5
```

```

6 void reset(contatore* pc){ pc -> value =0; }
7 void inc(contatore* pc) { (pc ->value)++; }
8 int getValue(contatore c){ return c.value; }
9

```

in **JAVA**

```

1 public class Counter {
2     private int value;
3     public void reset() { value = 0; }
4     public void inc() { value++; }
5     public int getValue(){ return value; }
6 }
7
8 //main
9 public class MyMain {
10     public static void main(String[] args) {
11         int v1, v2; Counter c1, c2;
12         c1 = new Counter(); c2 = new Counter();
13         c1.reset(); c2.reset();
14         c1.inc(); c1.inc(); c2.inc();
15         v1 = c1.getValue(); v2 = c2.getValue();
16         System.out.println(v1); System.out.println(v2);
17     }
18 }

```

Important

Contents- **La classe come "Progetto"**: La classe definisce un modello che specifica la struttura interna e il comportamento condiviso.

- **L'oggetto come "Istanza"**: Ogni volta che si usa il timbro, si crea un nuovo oggetto distinto (un'istanza) con la propria identità e il proprio stato indipendente.
- **Passaggio dei parametri implicito**: Poiché dati e funzioni sono uniti, le funzioni interne alla classe accedono direttamente ai dati dell'oggetto senza bisogno di passare puntatori come argomenti.

Gli oggetti

Gli oggetti sono **istanze di una classe**

Se la classe è il progetto di una macchina l'oggetto è la macchina stessa

la creazione di un nuovo oggetto (**allocazione dinamica**) è affidata all'operatore **new**

Il costruttore

seguendo l'esempio di una frazione:

```

1 public class Frazione {
2     private int num, den;
3     // Costruttore primario a due argomenti
4     public Frazione(int n, int d) {
5         num = n;
6         den = d;
7     }
8     // Costruttore ausiliario a un argomento (per numeri interi)
9     public Frazione(int n) {
10         num = n;
11         den = 1;
12     }
13     public int getNum() { return num; }
14     public int getDen() { return den; }
15     // Metodi omessi per brevità: equals() e minTerm()
16 }
17
18
19 // Esempio d'uso
20 public class MyMain {
21     public static void main(String[] args) {
22         Frazione f1 = new Frazione(3,4); // costruttore primario
23         Frazione f2 = new Frazione(2); // costruttore ausiliario (den=1)
24         System.out.println(f1.getNum() + "/" + f1.getDen()); // L'operatore + concatena e converte i numeri in
stringhe
25     }
26 }

```

Inizializzare significa assegnare un valore iniziale a un oggetto che esiste già in memoria.

Creare significa esclusivamente allocare la memoria necessaria per un nuovo oggetto.

Java permette di **creare un oggetto senza inicializzarlo** --> alta probabilità di introdurre bug.

Costruire rappresenta l'unione logica delle due azioni, ovvero **creare l'oggetto e contemporaneamente inicializzarlo**.

- Il metodo **costruttore** ha sempre **nome uguale alla classe**
- viene **invocato automaticamente** ogni volta che si istanzia un **nuovo oggetto**
- non può **mai essere richiamato** esplicitamente **dall'utente**
- si possono creare costruttori diversi (stesso nome) con parametri diversi
- il costruttore di default (no argomenti) inicializza le **variabili numeriche a zero** e i **referimenti a null**

Riferimenti

- **Puntatori (Stile C):** Contengono l'**indirizzo di memoria di una variabile** e permettono di **manipolarlo liberamente** tramite l'**aritmetica** dei puntatori. Sono potenti ma **pericolosi**, poiché richiedono il **dereferencing esplicito** (tramite l'operatore `*`) ed espongono al rischio di **invadere aree di memoria** non pertinenti.
- **Riferimenti (Java):** Rappresentano un'astrazione di più alto livello. **Contengono l'indirizzo di un oggetto** ma **non consentono di vederlo né di manipolarlo**.
 - **Non esiste l'aritmetica** dei puntatori, garantendo così l'inviolabilità della barriera di astrazione.
 - Il **dereferencing avviene in automatico** tramite la notazione puntata (es. `c.inc()`), azzerando i rischi del dereferencing manuale.
- **Tipi di Dati (Java)** Il linguaggio impone il **passaggio per valore per i tipi primitivi** e il **passaggio per riferimento per gli oggetti**.

Operazioni sui Riferimenti e Aliasing

- **Dichiarazione:** Definirli senza inicializzarli (es. `Counter c;`).
- **Assegnazione a null:** Assegnare la costante `null` per indicare che il riferimento non punta a nulla. PERICOLOSO
- **Creazione:** Usarli per allocare nuovi oggetti tramite la keyword `new` (es. `c = new Counter();`).
- **Confronto:** Verificare se due riferimenti puntano al medesimo indirizzo.
- **Aliasing:** Assegnare un riferimento a un altro (es. `Counter c2 = c1;`).

Se si assegna `c1` a `c2`, entrambi i riferimenti punteranno allo stesso identico oggetto in memoria. Di conseguenza, un'operazione che muta lo stato dell'oggetto tramite `c2` (es. `c2.inc();`) si rifletterà istantaneamente anche su `c1`.

```
1 Counter c1 = new Counter();
2 c1.reset();
3 c1.inc(); // c1 ora vale 1
4 Counter c2 = c1; // Creazione dell'alias
5 c2.inc(); // Incrementa c2
6 System.out.println(c1.getValue()); // Stamperà 2, perché c1 e c2 sono lo stesso oggetto
```

Confronto: Identità (`==`) vs. Uguaglianza (`equals`)

- **Identità (`==`):** In Java, l'operatore `==` **confronta esclusivamente l'identità di due oggetti**. L'espressione `c1 == c2` è vera se e solo se `c1` e `c2` sono alias, **ovvero puntano allo stesso identico spazio di memoria**. Se due oggetti hanno lo **stesso contenuto** ma sono **istanze distinte** create con due `new` separati, il confronto con `==` darà `false`.
- **Uguaglianza Semantica (`equals`):** Quando serve confrontare il "valore" o il significato logico di due oggetti, il progettista deve specificare un **criterio personalizzato** implementando il metodo `equals`.
es. `counter`

```
1 public boolean equals(Counter x) {return value == x.value;}
```

due `Counter` sono uguali se il valore dell'oggetto corrente («questo») è uguale a quello dell'oggetto ricevuto come argomento («l'altro»)

Per meglio evidenziare la simmetria fra i due oggetti del confronto si usa

- `this` serve a denotare esplicitamente **l'oggetto corrente**
- `that` **non è una parola chiave** del linguaggio. È una convenzione, ovvero un nome di variabile scelto liberamente dal programmatore per indicare **l'oggetto ricevuto come argomento**.

```
1 public boolean equals(Counter that) {this.value == that.value;}
```

Essendo all'interno della classe posso usare `that.value`

la keyword `this` si utilizza anche nei costruttori

```
1 public Counter() {this.value = 1; }
```

Eclipse può generare i costruttori automaticamente

La keyword `this` può essere utilizzata per **richiamare un costruttore dall'interno di un altro costruttore della stessa classe**.

```

1  public class Point {
2      double x, y, z;
3      // Costruttore a 3 argomenti (il caso più generale)
4      public Point(double x, double y, double z) {
5          this.x = x; this.y = y; this.z = z;
6      }
7      // Costruttore a 2 argomenti: richiama quello a 3 argomenti
8      public Point(double x, double y) {
9          this(x, y, 0);
10     }
11     // Costruttore a 1 argomento: richiama quello a 2 argomenti
12     public Point(double x) {
13         this(x, 0);
14     }
15 }

```

- economia di codice
- collaudabilità
- single entry

Pre-inizializzazioni

Quando ci sono **inizializzazioni** che sono identiche per tutti i costruttori è possibile specificarle **direttamente nella dichiarazione del campo**.

```

1  public class Point {
2      double x, y, z = 18; // z è pre-inizializzato
3      // ...
4  }

```

Incapsulamento e Accesso ai Campi Privati

Java **permette** di violare l'incapsulamento nel definire i metodi di una classe

```

1  public boolean equals(Frazione that){
2      return this.num * that.den == this.den * that.num;
3  }

```

Una good practice è utilizzare comunque i getter

```

1  public class Frazione {
2      private int num, den;
3
4      public boolean equals(Frazione that) {
5          // Incapsulamento rispettato sia per "this" che per "that"
6          return this.getNum() * that.getDen() == this.getDen() * that.getNum();
7      }
8  }

```

Overloading di Funzioni

- in Java anche le funzioni ordinarie possono essere **omonime** all'interno della stessa classe, a condizione che siano **distinguibili dalla lista degli argomenti** (firme diverse).
- Il **tipo di ritorno**, da solo, **non è sufficiente** per distinguere due metodi omonimi.
- **Obiettivo**: Evitare la proliferazione di **nomi diversi** (es. `inc1()`, `inc(int k)`) per operazioni che sono varianti della stessa azione.

```

1  public class Counter {
2      // ...
3      // Metodo senza argomenti
4      public void inc() { this.value++; }
5      // Metodo sovraccaricato con un argomento intero
6      public void inc(int k) { this.value += k; }
7  }

```

Stringhe

le stringhe sono **oggetti** della classe `String`. Sono **immutabili**. Ogni stringa rappresenta una specifica sequenza di caratteri, se si vuole una stringa diversa, si crea un nuovo oggetto.

la sintassi per la creazione di una stringa è semplificata

```

1  String s = "ciao";

```

Le stringhe **non sono array di caratteri** (non si può usare l'operatore `[]`)

```

1  String s = "Nel mezzo del cammin";

```

```

2  char ch = s.charAt(4);    // OK, lettura
3  // s.charAt(4) = 'Q';    // ERRORE: non si può modificare

```

Metodo	Descrizione	Esempio
<code>charAt(int index)</code>	Restituisce il carattere alla posizione specificata (da 0 a <code>length-1</code>).	<code>"ciao".charAt(1) → 'i'</code>
<code>length()</code>	Restituisce il numero di caratteri della stringa.	<code>"ciao".length() → 4</code>
<code>equals(Object obj)</code>	Confronta il contenuto di due stringhe (case sensitive).	<code>"ciao".equals("CIAO") → false</code>
<code>equalsIgnoreCase(String s)</code>	Confronta il contenuto ignorando maiuscole/minuscole.	<code>"ciao".equalsIgnoreCase("CIAO") → true</code>
<code>compareTo(String s)</code>	Confronta lessicograficamente due stringhe (restituisce negativo, zero, positivo).	<code>"a".compareTo("b") → negativo</code>
<code>indexOf(String s)</code>	Cerca la prima occorrenza di una sottostringa (o carattere).	<code>"banana".indexOf("na") → 2</code>
<code>lastIndexOf(String s)</code>	Cerca l'ultima occorrenza di una sottostringa.	<code>"banana".lastIndexOf("na") → 4</code>
<code>startsWith(String prefix)</code>	Verifica se la stringa inizia con un dato prefisso.	<code>"Java".startsWith("Ja") → true</code>
<code>endsWith(String suffix)</code>	Verifica se la stringa termina con un dato suffisso.	<code>"file.txt".endsWith(".txt") → true</code>
<code>substring(int begin)</code>	Estrae la sottostringa da begin fino alla fine.	<code>"programma".substring(4) → "gramma"</code>
<code>substring(int begin, int end)</code>	Estrae la sottostringa da begin (incluso) a end (escluso).	<code>"programma".substring(4,7) → "gra"</code>
<code>replace(char old, char new)</code>	Sostituisce tutte le occorrenze di un carattere.	<code>"casa".replace('a','o') → "coso"</code>
<code>toLowerCase()</code>	Converte tutti i caratteri in minuscolo.	<code>"Java".toLowerCase() → "java"</code>
<code>toUpperCase()</code>	Converte tutti i caratteri in maiuscolo.	<code>"Java".toUpperCase() → "JAVA"</code>
<code>trim()</code>	Rimuove spazi bianchi iniziali e finali.	<code>" ciao ".trim() → "ciao"</code>
<code>split(String regex)</code>	Divide la stringa in array usando un delimitatore (regex).	<code>"a,b,c".split(",") → ["a","b","c"]</code>
<code>concat(String s)</code>	Concatena la stringa con un'altra (equivalente a <code>+</code>).	<code>"ciao".concat(" mondo") → "ciao mondo"</code>
<code>join(CharSequence delim, CharSequence... elems)</code>	Metodo statico che unisce più stringhe con un delimitatore.	<code>String.join("-", "a","b","c") → "a-b-c"</code>
<code>valueOf(tipo primitivo)</code>	Metodo statico che converte un valore primitivo in stringa.	<code>String.valueOf(123) → "123"</code>
<code>format(String format, Object... args)</code>	Metodo statico che restituisce una stringa formattata.	<code>String.format("%d più %d fa %d", 2,3,5)</code>

Concatenazione

L'operatore `+` crea un **nuovo oggetto** `String` che è la concatenazione degli operandi.

```

1  String s1 = "ciao";
2  String s2 = " mondo";
3  String s3 = s1 + s2; // nuovo oggetto "ciao mondo"

```

Se si riassegna `s1 = s1 + s2;`, il riferimento `s1` **punta al nuovo oggetto**, ma l'oggetto originale `"ciao"` **rimane in memoria** (eventualmente raccolto dal garbage collector se non più referenziato).

Carattere di nuova riga

Java usa `\n` ma i sistemi operativi hanno convenzioni diverse quindi per scrivere codice portabile si usa `System.lineSeparator()` (molto verboso).

Text Blocks

Consentono di scrivere multilinea con **sintassi alleggerita**

Sintassi:

```

1  String s = """
2      E' la prima volta che mi capita
3      Prima mi chiudevo in una scatola
4      Sempre un po' distante...
5      """;

```

Uguaglianza di stringhe

In Java, `==` confronta i **referimenti** (se due variabili puntano allo stesso oggetto).

Per confrontare il contenuto si utilizza `equals` o `equalsIgnoreCase`

```

1 String a = "ciao";
2 String b = "ciao";
3 String c = new String("ciao");
4 System.out.println(a == b);      // true (stesso oggetto literal internato)
5 System.out.println(a == c);      // false (oggetti diversi)
6 System.out.println(a.equals(c)); // true (stesso contenuto)

```

Metodo toString

Ogni classe in Java eredita il metodo `toString()` da `Object`

La versione di default restituisce una stringa con il nome della classe e un codice hash (es. `Counter@15db9742`).

```

1 Counter c = new Counter(5);
2 System.out.println(c.toString()); // output: Counter@15db9742
3 System.out.println(c);           // stessa cosa (println chiama toString automaticamente)

```

Le informazioni stampate di default sono poco utili quindi ha senso sovrascriverle (`@Override`)

```

1 public class Counter {
2     private int value;
3
4     // costruttori e altri metodi...
5
6     @Override
7     public String toString() {
8         return "Counter di valore " + value;
9     }
10 }

```

Conversione dei metodi primitivi

I tipi primitivi (`int`, `double`, `boolean`, ecc.) **non sono oggetti**, quindi non hanno metodi come `toString()`.

Java fornisce una serie di metodi **statici** nella classe `String` chiamati `valueOf`

```

1 int a = 35;
2 double d = 3.14;
3 String s1 = String.valueOf(a); // "35"
4 String s2 = String.valueOf(d); // "3.14"
5 String s3 = String.valueOf(true); // "true"

```

L'utilizzo di questo metodo è sottointeso nell'utilizzo dell'operatore `+`

Formattazione stringhe --C

Metodo statico

```

1 String s = String.format("%-20s canta la canzone %-35s", "Francesca Michielin", "Nessun grado di separazione");

```

Metodo di istanza

```

1 String s = "%-20s canta la canzone %-35s".formatted("Francesca Michielin", "Nessun grado di separazione");

```

StringBuilder

La concatenazione con `+` in un ciclo è inefficiente perché crea **nuovi oggetti** a ogni iterazione, copiando i caratteri e abbandonando i precedenti al garbage collector.

```

1 StringBuilder sb = new StringBuilder();
2 for (int i = 0; i < 1000; i++) {
3     sb.append("La nebbia agli irti colli");
4 }
5 String risultato = sb.toString(); // converte in String immutabile

```

- `StringBuilder` ha metodi come `append`, `insert`, `delete`, ecc.
- È molto più veloce

StringJoiner e String.Join

`StringJoiner` è un contenitore progettato per concatenare più stringhe con un separatore, gestendo automaticamente l'assenza del separatore dopo l'ultimo elemento.

```

1 StringJoiner sj = new StringJoiner(", ");
2 for (String arg : args) {
3     sj.add(arg);
4 }
5 System.out.println(sj); // "alfa, beta, gamma, delta" (senza virgola finale)

```

Es. codice fiscale
(per criteri codice vedi slide)
Si utilizza il **pattern di programmazione FACADE**

- i **metodi pubblici costituiscono la "facciata"** del componente
- lavorano coordinando il lavoro di altri metodi (non visibili)

Per distinguere vocali e consonanti verifico la loro appartenenza a una stringa di sole vocali o consonanti.

JUnit

è uno strumento di testing più evoluto rispetto ad assert.

- Notifica dei problemi in formato amichevole e analizzabile.
 - Esecuzione di tutti i test anche in caso di fallimenti.
 - Report chiaro e compatto sui risultati.
- JUnit sostituisce l'istruzione `assert` con una serie di **metodi statici** della classe `Assertions` (package `org.junit.jupiter.api.Assertions`).

Import Necessary

```
1 import org.junit.jupiter.api.*;           // per le annotazioni
2 import static org.junit.jupiter.api.Assertions.*; // per i metodi assert
```

Metodo	Descrizione
<code>assertEquals(expected, actual)</code>	Verifica che due valori/oggetti siano uguali (usa <code>equals</code> per oggetti)
<code>assertEquals(expected, actual, delta)</code>	Per valori floating-point: verifica uguaglianza entro un delta (tolleranza)
<code>assertArrayEquals(expected, actual)</code>	Verifica che due array siano uguali (elemento per elemento)
<code>assertTrue(condition)</code>	Verifica che la condizione sia vera
<code>assertFalse(condition)</code>	Verifica che la condizione sia falsa
<code>assertNull(value)</code>	Verifica che il valore sia nullo
<code>assertNotNull(value)</code>	Verifica che il valore non sia nullo
<code>assertSame(expected, actual)</code>	Verifica che i riferimenti puntino allo stesso oggetto (<code>==</code>)
<code>assertNotSame(expected, actual)</code>	Verifica che i riferimenti puntino a oggetti diversi
<code>fail()</code>	Forza il fallimento del test (utile per percorsi eccezionali)

esempio

```
1 package counter;
2
3 public class Counter {
4     private int val;
5
6     public Counter() { val = 1; }
7     public Counter(int v) { val = v; }
8     public void reset() { val = 0; }
9     public void inc() { val++; }
10    public void inc(int k) { val += k; }
11    public int getValue() { return val; }
12    public boolean equals(Counter x) { return val == x.val; }
13    public String toString() { return "Counter di valore " + val; }
14 }
```

```
1 package counter.test;
2
3 import static org.junit.jupiter.api.Assertions.*;
4 import org.junit.jupiter.api.Test;
5 import counter.Counter;
6
7 class CounterTest {
8
9     // Test costruttori
10    @Test
11    void testConstructor0() {
12        Counter c = new Counter();
13        assertEquals(1, c.getValue());
14    }
15 }
```

```

16     @Test
17     void testConstructor1() {
18         Counter c = new Counter(10);
19         assertEquals(10, c.getValue());
20     }
21
22     @Test
23     void testConstructor1b() {
24         Counter c = new Counter(27);
25         assertEquals(27, c.getValue());
26     }
27
28     // Test reset
29     @Test
30     void testReset() {
31         Counter c = new Counter(5);
32         c.reset();
33         assertEquals(0, c.getValue());
34     }
35
36     // Test inc senza parametri
37     @Test
38     void testInc() {
39         Counter c = new Counter(5);
40         c.inc();
41         c.inc();
42         assertEquals(7, c.getValue());
43     }
44
45     // Test inc con parametro
46     @Test
47     void testIncWithK() {
48         Counter c = new Counter(5);
49         c.inc(3);
50         assertEquals(8, c.getValue());
51     }
52
53     @Test
54     void testIncWithKMultiple() {
55         Counter c = new Counter(5);
56         c.inc(2);
57         c.inc(3);
58         assertEquals(10, c.getValue());
59     }
60
61     // Test equals
62     @Test
63     void testEquals() {
64         Counter c1 = new Counter(7);
65         Counter c2 = new Counter(7);
66         Counter c3 = new Counter(10);
67
68         assertTrue(c1.equals(c2));    // simmetria
69         assertFalse(c1.equals(c3));
70         assertTrue(c1.equals(c1));    // riflessività
71     }
72 }

```

Gestione in Eclipse

- Tasto destro sulla classe da testare → **New** → **JUnit Test Case**.
 - Wizard: scegliere i metodi da collaudare (opzionale).
 - Selezionare la source folder di destinazione (es. `tests`).
 - Eclipse crea automaticamente lo scheletro della classe di test con i metodi vuoti.
 - Se JUnit non è già nel build path, Eclipse lo aggiunge automaticamente (chiede conferma).
- Esecuzione
- Tasto destro sulla classe di test → **Run As** → **JUnit Test**.
 - La vista JUnit mostra:
 - **Barra verde:** tutti i test passano.
 - **Barra rossa:** almeno un test fallisce.
 - Dettaglio degli errori: blu per assertion fallite, rosso per eccezioni (stack trace).

Strutturazione dei test

Quando più test condividono operazioni di setup o cleanup, è utile fattorizzarle in metodi dedicati, annotati opportunamente.

Annotazione	Descrizione
@BeforeAll	Metodo statico eseguito una sola volta prima di tutti i test della classe.
@AfterAll	Metodo statico eseguito una sola volta dopo tutti i test della classe.
@BeforeEach	Metodo non statico eseguito prima di ogni metodo @Test .
@AfterEach	Metodo non statico eseguito dopo ogni metodo @Test .

Gestione delle Eccezioni

In Java esiste un sistema integrato per gestire errori e situazioni anomale basato sul concetto di **eccezione**. L'obiettivo è separare nettamente il flusso normale (l'*happy path*) dal codice di gestione errori, evitando codice rumoroso pieno di `if` e permettendo una propagazione controllata dei problemi.

Blocco Try-Catch

Il costrutto `try-catch` delimita un blocco "sorvegliato":

- si **tenta** (`try`) di eseguire le operazioni critiche;
- se qualcosa va storto, viene **lanciata un'eccezione**, l'esecuzione si interrompe immediatamente e il controllo passa al blocco `catch` corrispondente.

```
1  try {
2      // codice che può generare eccezioni
3      System.out.println("Accedo all'elemento 5: " + numeri[5]);
4  } catch (TipoEccezione e) {
5      // gestione dell'eccezione
6      System.out.println("Errore: indice dell'array non valido!");
7      System.out.println("Messaggio: " + e.getMessage());
8  }
9  System.out.println("Il programma continua normalmente...");
```

Se non si verifica alcun errore, il `catch` viene saltato e l'esecuzione prosegue normalmente dopo il blocco.

Catch multipli e Multi-catch

Per gestire eccezioni di tipo diverso si possono usare più blocchi `catch`, **sempre dalla più specifica alla più generale** (la prima corrispondenza termina la ricerca).

```
1  try {
2      // operazioni che possono lanciare eccezioni diverse
3  } catch (FileNotFoundException e) {
4      // gestione file mancante (specifico)
5  } catch (IOException e) {
6      // gestione altri errori di I/O (generale)
7  }
```

quando si vuole eseguire lo stesso trattamento per più tipi di eccezione, si può usare il **multi-catch**:

```
1  catch (IOException | SQLException e) {
2      // gestione unificata per entrambi i tipi
3  }
```

Finally

Il blocco `finally` contiene codice che viene **sempre** eseguito, sia che l'operazione riesca sia che venga lanciata un'eccezione. Viene usato tipicamente per rilasciare risorse (file, connessioni, lock). Con l'introduzione del **try-with-resources**, molte operazioni di chiusura vengono gestite automaticamente, riducendo la necessità del `finally`.

```
1  FileReader file = null;
2  try {
3      file = new FileReader("testo.txt");
4      // operazioni sul file
5  } catch (IOException e) {
6      System.out.println("Errore nella lettura");
7  } finally {
8      if (file != null) {
9          try {
10             file.close(); // chiusura sicura
11         } catch (IOException e) {
12             System.out.println("Errore nella chiusura");
13         }
14     }
15 }
```

```
15 }
```

Throw: lanciare deliberatamente un'eccezione

L'istruzione `throw` serve a **lanciare esplicitamente** un'eccezione. Si crea un oggetto eccezione con `new` e lo si "lancia" interrompendo il flusso corrente.

```
1 throw new TipoEccezione("Messaggio di errore");
```

Esempio pratico: controllo di precondizioni.

```
1 public static void verificaEta(int eta) {
2     if (eta < 0) {
3         throw new IllegalArgumentException("L'età non può essere negativa: " + eta);
4     }
5     System.out.println("Età valida: " + eta);
6 }
```

Questo meccanismo è fondamentale per impedire la creazione di oggetti inconsistenti (ad es. un triangolo impossibile o una frazione con denominatore zero) o per segnalare argomenti non validi.

Eccezioni Checked e Unchecked

In Java le eccezioni si dividono in due grandi categorie:

Tipo	Gerarchia	Obbligo di gestione	Significato tipico
Checked (a controllo obbligatorio)	Exception (ma non RuntimeException)	Sì: try-catch o throws	Situazioni esterne al controllo del programma (file mancante, rete assente)
Unchecked (a controllo facoltativo)	RuntimeException e sottoclassi Error e sottoclassi	No (facoltativo)	Bug o violazioni di precondizioni (NullPointerException, IllegalArgumentException, ecc.) Gli Error sono problemi gravi della JVM (es. OutOfMemoryError) e non andrebbero catturati.

Gerarchia semplificata:

```
1 Throwable
2 |— Error (unchecked, problemi JVM)
3 |— Exception
4 |   |— RuntimeException (unchecked, bug/precondizioni)
5 |   |— (altre) Exception (checked, situazioni esterne)
```

La scelta tra checked e unchecked è parte della progettazione:

- Un'eccezione **checked** costringe il chiamante a pensare alla possibile anomalia. Funziona bene in sistemi a piccola/media scala, ma può diventare onerosa in architetture a molti strati.
- Un'eccezione **unchecked** non impone vincoli di compilazione; si assume che le precondizioni siano rispettate e che eventuali violazioni vadano scoperte durante il testing.

Throws: dichiarare le eccezioni lanciabili

Quando un metodo può lanciare un'eccezione **checked** senza gestirla internamente, **deve** dichiararlo nella propria firma con la clausola `throws`. In questo modo avverte i chiamanti che quel metodo è "pericoloso" e che devono farsene carico (con un try-catch o rilanciandola).

```
1 public void leggiFile(String nomeFile) throws IOException {
2     // ...
3 }
```

Per le eccezioni **unchecked** la dichiarazione `throws` è **facoltativa**; per chiarezza, la si può aggiungere solo nei casi in cui l'eccezione è particolarmente inattesa o significativa per chi usa il metodo.

Rilancio di un'eccezione: un metodo può catturare un'eccezione ma decidere di non saperla gestire, limitandosi a **rilanciarla** con `throws`.

```
1 void read(String fname) throws FileNotFoundException{
2     FileReader f = new FileReader(fname);
3 }
4
5 public void printAll(String filename){
6     try{
7         read(filename); // è pericolosa!
8     } catch(FileNotFoundException e){ ... }
```

```

9  }
10
11 //posso rilanciarla ancora
12 public void printAll(String filename) throws FileNotFoundException{
13     read(filename); // è pericolosa!
14 }

```

Eccezioni Personalizzate

Creare nuovi tipi di eccezione aiuta a descrivere con precisione il problema e permette di distinguere in fase di `catch` situazioni diverse (ad es. `ImpossibleTriangleException`, `BadFormatException`).

- Per un'eccezione **checked**, estendere direttamente `Exception`.
- Per un'eccezione **unchecked**, estendere `RuntimeException` (o una sua sottoclasse come `IllegalArgumentException`).

Tipicamente si forniscono costruttori che accettano un messaggio e, se si vuole mantenere traccia della causa originale, un `Throwable` (l'eccezione fisica).

```

1  public class ImpossibleTriangleException extends IllegalArgumentException {
2      public ImpossibleTriangleException() { super(); }
3      public ImpossibleTriangleException(String message) { super(message); }
4      public ImpossibleTriangleException(String message, Throwable cause) { super(message, cause); }
5  }

```

Uso:

```

1  if (a >= b + c || b >= a + c || c >= a + b) {
2      throw new ImpossibleTriangleException(
3          "Lati impossibili: " + a + ", " + b + ", " + c);
4  }

```

Incapsulamento di Eccezioni (fisiche → logiche)

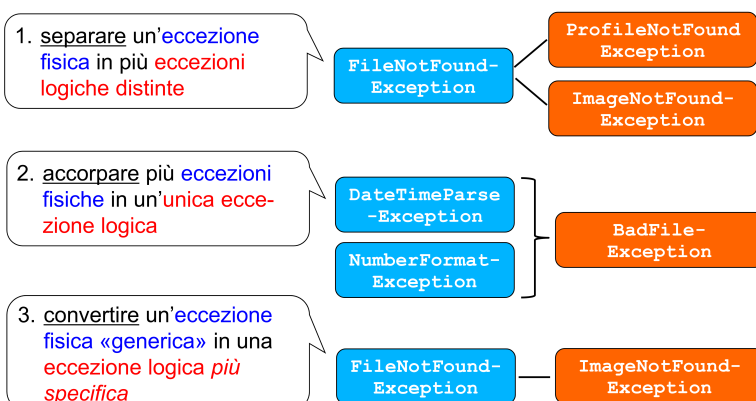
Spesso si intercetta un'eccezione “fisica” (es. `FileNotFoundException`) per convertirla in un'eccezione “logica” che abbia più significato nel contesto dell'applicazione (es. `ImageNotFoundException`).

Ciò si realizza catturando l'eccezione originale e costruendo la nuova eccezione passandole la causa con il costruttore apposito. In questo modo lo stack della causa rimane accessibile.

```

1  public void caricaImmagine(String percorso) throws ImageNotFoundException {
2      try {
3          FileReader f = new FileReader(percorso);
4          // ...
5      } catch (FileNotFoundException e) {
6          throw new ImageNotFoundException("Immagine non trovata: " + percorso, e);
7      }
8  }
9
10 public void readUserData(String filename) throws BadFormatException {
11     try {
12         FileReader f = new FileReader(filename);
13         String s = null; ... // lettura linea di testo
14         int n = Integer.parseInt(s);
15     } catch (FileNotFoundException | NumberFormatException e) { //sintassi comoda
16         throw new BadFormatException(e);
17     }
18 }

```



Il parsing non lancia eccezioni anche se a volte sarebbe necessario

```

1 ParsePosition position = new ParsePosition(0);
2 Number n = f.parse("10-10", position);
3 if (position.getIndex() != str.length()) { throw new OpportunaEccezione(...); }

```

Se il parsing non è arrivato in fondo lancia un'eccezione.

Verifica delle Precondizioni con java.util.Objects

La classe di utilità `java.util.Objects` offre metodi statici per controllare i parametri in modo sintetico e standardizzato:

- `requireNonNull(T obj)` lancia `NullPointerException` se `obj` è `null`.
 - `requireNonNull(T obj, String message)` con messaggio personalizzato.
 - `checkIndex(...)` e `checkFromToIndex(...)` per validare indici e intervalli, lanciano `IndexOutOfBoundsException`.
- Questi metodi vengono impiegati spesso all'inizio di un metodo o costruttore per validare le precondizioni.

```

1 public void setName(String nome) {
2     this.nome = Objects.requireNonNull(nome, "Il nome non può essere null");
3 }

```

Esempio Completo: Lettura di un File con Eccezioni Personalizzate

```

1 /**
2  * Legge il contenuto di un file di testo.
3  *
4  * @param percorso percorso del file
5  * @return contenuto del file come stringa
6  * @throws FileNotFoundException se il file non esiste
7  * @throws LetturaFileException per altri errori di I/O
8  */
9 public String leggiFile(String percorso) throws FileNotFoundException, LetturaFileException {
10     StringBuilder contenuto = new StringBuilder();
11
12     // try-with-resources: il reader viene chiuso automaticamente
13     try (BufferedReader reader = new BufferedReader(new FileReader(percorso))) {
14         String linea;
15         while ((linea = reader.readLine()) != null) {
16             contenuto.append(linea).append("\n");
17         }
18     } catch (FileNotFoundException e) {
19         // incapsulo l'eccezione fisica in una logica specifica
20         throw new FileNotFoundException("File non trovato: " + percorso, e);
21     } catch (IOException e) {
22         throw new LetturaFileException("Errore durante la lettura del file", e);
23     }
24     return contenuto.toString();
25 }

```

Definizione delle eccezioni personalizzate:

```

1 public class FileNotFoundException extends Exception {
2     public FileNotFoundException(String msg, Throwable cause) { super(msg, cause); }
3 }
4
5 public class LetturaFileException extends Exception {
6     public LetturaFileException(String msg, Throwable cause) { super(msg, cause); }
7 }

```

Array

In Java gli array sono **oggetti** veri e propri, istanze di una classe speciale.

Dichiarazione e creazione

```

1 int[] v;           // dichiarazione di un riferimento
2 v = new int[3];    // creazione dinamica dell'oggetto array

```

- `new int[3]` crea un array di **3 elementi**, ciascuno **inizializzato al valore di default** per il tipo (0 per interi, `false` per boolean, `\u0000` per char, `null` per riferimenti).

****Differenza tra array di tipi primitivi e array di oggetti**

- **Primitivi:** ogni cella contiene direttamente il valore (es. `int`).

- **Oggetti:** ogni cella contiene un **referimento** a un oggetto (inizialmente `null`). Dopo la creazione dell'array, occorre creare singolarmente gli oggetti da inserire:

```
1 Counter[] w = new Counter[6]; // tutti i riferimenti sono null
2 w[0] = new Counter(11);      // ora w[0] punta a un oggetto
```

È possibile inserire riferimenti a oggetti già esistenti:

```
1 Counter c1 = new Counter(3);
2 w[2] = c1;
```

Proprietà `length`

Ogni array in Java ha una proprietà pubblica `length` (finale) che indica il numero di elementi.

```
1 int[] v = new int[3];
2 System.out.println(v.length); // stampa 3
```

Inizializzazione degli array

- **Inizializzazione automatica:** al momento della creazione con `new`, tutti gli elementi sono posti al valore di default del tipo.
- **Inizializzazione contestuale (solo al momento della dichiarazione):**

```
1 int[] v1 = {2, 3, 4}; // array di 3 elementi
2 Counter[] w1 = { new Counter(2), new Counter(3) };
```

Questa sintassi è valida solo nella dichiarazione, non per riassegnazioni successive.

- Inizializzazione al volo come argomento di una funzione

```
1 void f(int[] v) { ... }
2 f(new int[]{2, 3, 4}); // crea e passa un array anonimo
```

Il problema del metodo `toString()`

La classe "array" non ha un metodo `toString` personalizzabile; eredita quello di `Object`, che produce una stringa del tipo `[I@6659c656` (dove `[I` indica array di `int`).

Soluzioni per **stampare il contenuto**:

- Scrivere un ciclo esplicito.
- Usare i metodi statici della classe `java.util.Arrays`:
 - `Arrays.toString(array)` **per array monodimensionali**.
 - `Arrays.deepToString(array)` **per array multidimensionali**.
- Convertire l'array in una lista (ad es. `Arrays.asList()`), ma attenzione: funziona solo per array di oggetti, non per primitivi. es.

```
1 public class EsempioMain {
2     public static void main(String[] args) {
3         if (args.length == 0)
4             System.out.println("Nessun argomento");
5         else
6             for (int i = 0; i < args.length; i++)
7                 System.out.println("argomento " + i + ": " + args[i]);
8     }
9 }
```

Array come valore di ritorno di una funzione

in Java una funzione può restituire un array:

```
1 public static int[] creaTabella(int n) {
2     int[] tab = new int[n];
3     for (int i = 0; i < n; i++)
4         tab[i] = i * i;
5     return tab;
6 }
```

Metodo o static?

- un **metodo** verrebbe **invocato su uno specifico oggetto** (ad es. `c1` in `c1.inc()`, `f2` in `f2.getNum()`, etc.). che dovrebbe quindi essere **preventivamente creato**

- una **funzione statica** («di libreria») viene invece **invocata semplicemente col suo nome** assoluto (Math.sin()): **non occorre creare nulla**, perché ci si rivolge a una classe

Nel caso in questione non c'è uno specifico oggetto «creatore di tabelle» → meglio una funzione statica

Ciclo "for each" (enhanced for)

Sintassi che permette di iterare su tutti gli elementi senza usare un indice esplicito.

```
1  for (int x : tab) {
2      System.out.println(x);
3  }
4
5  //main dell'es prec
6  public static void main(String[] args){
7      int[] tab = creaTabella(4);
8      for (int x : tab) System.out.println(x);
9  }
```

- A ogni iterazione, la variabile (x) contiene una **copia** del **valore dell'elemento** (per i primitivi) o una **copia del riferimento** (per oggetti).
- **Non si può usare per modificare gli elementi dell'array** (nel caso di primitivi perché si modifica la copia; nel caso di oggetti si può modificare l'oggetto tramite il riferimento, ma non si può cambiare il riferimento stesso).
- Utile solo per lettura o per operazioni che non richiedono l'indice.

Esempio di confronto di array (uguaglianza)

- per tipi primari

```
1  public static boolean idem(int[] a, int[] b) {
2      if (a.length != b.length) return false;
3      for (int i = 0; i < a.length; i++)
4          if (a[i] != b[i]) return false;
5      return true;
6  }
```

- Per array di oggetti, si usa il metodo `equals` della classe degli oggetti (che deve essere opportunamente sovrascritto)

```
1  if (!a[i].equals(b[i])) return false;
```

Funzioni di utilità della classe `java.util.Arrays`

La classe `Arrays` fornisce numerosi metodi statici per operare sugli array:

Metodo	Descrizione
<code>sort(array)</code>	Ordina l'array (per tipi primitivi e oggetti)
<code>binarySearch(array, key)</code>	Ricerca binaria (array deve essere ordinato)
<code>copyOf(array, newLength)</code>	Restituisce una copia dell'array con la lunghezza specificata
<code>equals(array1, array2)</code>	Confronto superficiale (shallow)
<code>deepEquals(array1, array2)</code>	Confronto profondo (deep) per array annidati
<code>fill(array, value)</code>	Riempie l'array con un valore
<code>toString(array)</code>	Rappresentazione in stringa (monodimensionale)
<code>deepToString(array)</code>	Rappresentazione in stringa per array multidimensionali

Es. Stampare il contenuto

Si utilizza `toString(array)` da NON confondere con il metodo della classe `[]`

```
jshell> int[] v = {1,2,3,4,5};
v => int[5] { 1, 2, 3, 4, 5 }
jshell> String s1 = v.toString();
s1 => "[I@621be5d1"
jshell> String s2 = Arrays.toString(v);
s2 => "[1, 2, 3, 4, 5]"
```

Metodo `toString` della classe `[]`
(non personalizzabile né modificabile)

Funzione statica `toString`
della classe `Arrays`

vedi altri es. nel pdf

Costanti (final)

```
1  final int DIM = 8;
```

Array multidimensionali (matrici)

In Java non esistono array multidimensionali "veri", ma array di array.

- Dichiarazione

```
1 double[][] m; // array di array di double
```

- Creazione in due fasi (per **matrici irregolari**):

```
1 m = new double[3][]; // array esterno di 3 righe (riferimenti ad array)
2 m[0] = new double[5]; // prima riga di 5 elementi
3 m[1] = new double[5]; // seconda riga di 5 elementi
4 m[2] = new double[5]; // terza riga di 5 elementi
```

- Creazione concisa** per matrici regolari:

```
1 double[][] m = new double[3][5]; // matrice 3x5
```

- Accesso** `m[i][j]` dove `i` è l'indice di riga, `j` di colonna.
- Lunghezze
 - `m.length` è il numero di righe.
 - `m[i].length` è il numero di colonne della riga `i`-esima.
- Stampa** devo utilizzare la `Arrays.deepToString(m)`
- Equals** `Arrays.equals(array1, array2)` confronta i riferimenti agli array interni, non i loro contenuti. Per confrontare in profondità si usa `Arrays.deepEquals(m1, m2)` che **confronta ricorsivamente il contenuto**.
es. somma di matrici

```
1 public static double[][] sommaMatrici(double[][] a, double[][] b) {
2     // Supponiamo che a e b abbiano le stesse dimensioni
3     double[][] c = new double[a.length][a[0].length];
4     for (int i = 0; i < a.length; i++)
5         for (int j = 0; j < a[0].length; j++)
6             c[i][j] = a[i][j] + b[i][j];
7     return c;
8 }
```

FAI PRODOTTO DI MATRICI

Package

Applicazioni complesse sono composte da **molte classi e librerie**. Senza una struttura, si incorre in due problemi principali:

- Conflitti di Nome (Name Clash):** Due classi con lo stesso nome (es. `Book`) potrebbero essere sviluppate da team diversi o per scopi diversi, rendendo impossibile utilizzarle insieme nello stesso progetto.
- Ingestibilità:** Un insieme "piatto" di centinaia di classi è caotico quanto una singola cartella piena di file senza sottocartelle. I **package** (in Java) introducono uno **spazio di nomi strutturato**. Permettono di raggruppare logicamente classi correlate in un "pacchetto software".
- è un contenitore che raggruppa classi logicamente correlate.
- Definisce un nuovo livello di visibilità intermedio tra `public` e `private` : la **visibilità nel package** (è di default, non va specificata)
- Crea uno spazio di nomi: il nome completo di una classe è composto dal nome del package + il nome semplice della classe (es. `edenti.Book`).

Livello di Visibilità	Qualificatore Java	Descrizione
Pubblico	public	Visibile a qualsiasi classe di qualsiasi package.
Package	(nessuna parola chiave, è il default)	Visibile solo alle classi che si trovano all'interno dello stesso package . È il livello di default in Java.
Privato	private	Visibile solo all'interno della stessa classe.

```
1 // Nel file edenti/Book.java
2 package edenti;
3
4 class Book { // <-- Visibilità di package (default)
5     String title; // <-- Visibilità di package (default)
6
7     void printTitle() { // <-- Visibilità di package (default)
8         System.out.println(title);
9     }
}
```

```
10 }
```

```
1 // Nello stesso package edenti/Library.java
2 package edenti;
3
4 public class Library {
5     public void addBook(String title) {
6         Book b = new Book(); // OK: Library è nello stesso package di Book
7         b.title = title;      // OK: title ha visibilità di package
8         b.printTitle();       // OK: printTitle ha visibilità di package
9     }
10 }
```

```
1 // In un altro package, ad esempio un cliente
2 import edenti.Book; // ERRORE: Book non è public, quindi non è importabile/visibile qui!
3 public class Client {
4     public void test() {
5         Book b = new Book(); // NON COMPILA: Book non è visibile
6     }
7 }
```

Convenzioni di Nomenclatura

- **Nomi:** I nomi dei package sono convenzionalmente scritti in **minuscolo**.
- **Nome Assoluto:** È il nome univoco di una classe, ottenuto concatenando il nome del package e il nome della classe (es. `it.unibo.utilities.Point`) **reverse internet naming**.
- **Corrispondenza File System:** Java impone una rigida corrispondenza tra il nome del package e la struttura delle directory nel file system.
 - Il package **edenti** deve risiedere in una cartella chiamata `edenti`.
 - Il package **multilivello** `it.unibo.utilities` deve risiedere in una sequenza di cartelle innestate `it/unibo/utilities/`.
 - La classe `it.unibo.utilities.Point` deve trovarsi fisicamente nel file `it/unibo/utilities/Point.java`.
- **Default Package:** Se non si specifica un package, la classe finisce nel "default package".
 - **È da evitare assolutamente in progetti reali.**
 - Le classi nel default package non hanno un nome assoluto e non possono essere importate da classi che si trovano in un package con nome.

Utilizzo dei Package: `import`

Per usare una classe da un altro package, si deve usare il suo **nome assoluto**.

```
1 public class MyClient {
2     public static void main(String[] args) {
3         // Uso del nome assoluto
4         edenti.Book myBook = new edenti.Book("Il Nome della Rosa");
5     }
6 }
```

Per evitare di scrivere ogni volta il nome assoluto (che può essere molto lungo), si usa la direttiva `import` che permette di riferirsi a una classe di un altro package usando il suo **nome relativo (semple)**. Non include il codice ma permette al compilatore di risolvere il nome.

```
1 import edenti.Book; // Importa la singola classe Book
2
3 public class MyClient {
4     public static void main(String[] args) {
5         // Uso del nome relativo grazie all'import
6         Book myBook = new Book("Il Nome della Rosa");
7     }
8 }
```

Si può importare tutte le classi di un package con `import edenti.*;`.

Se ci sono omonimie la soluzione è importare la classe più usata e usare il nome assoluto per l'altra.

Compilazione e esecuzione

I comandi `javac` e `java` devono essere eseguiti dalla **directory che sta al di sopra della radice dei package**.

Nella stessa directory

```
1 javac edenti/Book.java edenti/Library.java
2
3 java edenti.Library
```

In un'altra directory

```
1 # -cp .;.. (Su Windows) indica di cercare le classi nella cartella corrente (.) e in quella superiore (..)
```



```

2  javac -cp .;.. MyClient.java
3
4  java -cp .;.. MyClient

```

I percorsi sono separati da `;` (Windows) o `:` (Linux/macOS).

Importazione statica

Introdotta per permettere l'uso di membri `static` (campi e metodi) di una classe senza dover prefissare il nome della classe.

```

1  // Senza import static
2  public class TestMath {
3      public double areaCerchio(double raggio) {
4          return Math.PI * raggio * raggio; // Devo sempre scrivere "Math."
5      }
6  }
7
8  // Con import static
9  import static java.lang.Math.PI;
10 import static java.lang.Math.pow; // O anche import static java.lang.Math.*;
11
12 public class TestMath {
13     public double areaCerchio(double raggio) {
14         return PI * pow(raggio, 2); // Non serve più "Math."
15     }
16 }

```

Da usare con parsimonia

Il Package `java.lang`

È il package più importante di Java, importato **automaticamente** in ogni programma.

Contiene le classi fondamentali del linguaggio:

- `Object` (la radice di tutte le classi)
- `String`, `StringBuilder`
- Classi wrapper: `Integer`, `Double`, `Boolean`, etc.
- `System` (per `System.out.println`)
- `Math`
- `Class` (per la reflection)
- Eccezioni e errori di base (`Throwable`, `Exception`, `RuntimeException`)

I moduli

Con applicazioni molto grandi (come l'intera piattaforma Java stessa), il solo meccanismo dei package si è rivelato insufficiente.

Un modulo è un raggruppamento di package che aggiunge un ulteriore livello di incapsulamento. Si specifica quali package **esporta** (rende visibili all'esterno) e da quali altri moduli **dipende**.

Le specifiche del modulo sono in un file speciale: `module-info.java`, posto nella directory radice dei package del modulo.

```

1  module it.unibo.mylib {
2      // Esporta il package 'it.unibo.mylib.utils' a tutti
3      exports it.unibo.mylib.utils;
4
5      // Esporta il package 'it.unibo.mylib.api' solo al modulo 'it.unibo.myapp'
6      exports it.unibo.mylib.api to it.unibo.myapp;
7
8      // Dichiarare che questo modulo dipende dal modulo 'java.sql'
9      requires java.sql;
10
11     // Dichiarare che questo modulo usa un servizio (per ServiceLoader)
12     uses it.unibo.mylib.spi.MyService;
13
14     // Dichiarare che questo modulo fornisce un'implementazione di un servizio
15     provides it.unibo.mylib.spi.MyService with it.unibo.mylib.internal.MyServiceImpl;
16 }

```

- I package non esportati sono accessibili solo all'interno del modulo. Non è più necessario usare `public` per farli vedere a package "amici".
- **Immagine di Runtime Ridotta:** Strumenti come `jlink` possono creare un'immagine di runtime di Java contenente solo i moduli strettamente necessari all'applicazione, riducendo le dimensioni.
- **Retrocompatibilità** Le classi tradizionali (senza modulo) finiscono in un "modulo senza nome" (unnamed module) e possono accedere a tutti i moduli della piattaforma.

Enumerativi

Un `enum` in Java è una classe a tutti gli effetti, con alcune caratteristiche speciali:

- Ha un **costruttore privato**, quindi l'utente non può creare istanze arbitrariamente.
- In quanto classe, può avere **stato (campi)** e **comportamento (metodi)**.
- I valori elencati (es. `NORTH`, `SOUTH`) sono le **uniche istanze pubbliche e statiche** della classe, create dal compilatore.

```
1 public enum Direction {
2     NORTH, SOUTH, EAST, WEST;
3 }
4
5 Direction dir = Direction.NORTH;
```

Utilizzo negli `switch`

```
1 Direction dir = Direction.NORTH;
2
3 // switch tradizionale (statement)
4 switch (dir) {
5     case NORTH: System.out.println("North"); break;
6     case SOUTH: System.out.println("South"); break;
7     case EAST: System.out.println("East"); break;
8     case WEST: System.out.println("West"); break;
9 }
```

oppure

```
1 var name = switch (dir) {
2     case NORTH -> "North";
3     case SOUTH -> "South";
4     case EAST -> "East";
5     case WEST -> "West";
6 }; // <-- Punto e virgola obbligatorio perché è un'assegnazione
```

- È un'espressione, quindi restituisce un valore che può essere assegnato a una variabile.
- I rami sono mutuamente esclusivi, quindi **non serve il** `break`
- Deve essere esaustiva: coprire tutti i casi possibili o includere un ramo `default`

L'Enum come Classe: Metodi Automatici

- `ordinal()` : Restituisce l'indice (0-based) della costante nell'ordine in cui è stata dichiarata. Da usare con cautela, perché l'ordine è parte della definizione dell'enum e non dovrebbe essere usato per logiche di business.
- `values()` : Metodo statico che restituisce un array contenente tutte le costanti dell'enum nell'ordine di dichiarazione. Utile per iterare.
- `valueOf(String name)` : Metodo statico che restituisce la costante dell'enum il cui nome corrisponde esattamente alla stringa passata.

```
1 for (Direction d : Direction.values()) {
2     System.out.println(d.ordinal()); // Stampa 0, 1, 2, 3
3 }
4
5 Direction d = Direction.valueOf("EAST"); // d sarà Direction.EAST
6 System.out.println(d); // Stampa "EAST"
```

Estendere l'Enum: Aggiungere Comportamento

Essendo una classe, possiamo arricchire un enum con metodi personalizzati.

```
1 public enum Direction {
2     NORTH, SOUTH, EAST, WEST;
3
4     public int getDegrees() {
5         return switch (this) {
6             case NORTH -> 0;
7             case SOUTH -> 180;
8             case EAST -> 90;
9             case WEST -> 270;
10        };
11    }
12
13    @Override
14    public String toString() {
15        return switch (this) {
16            case NORTH -> "Nord";
17            case SOUTH -> "Sud";
18            case EAST -> "Est";
19            case WEST -> "Ovest";
```

```

20     } + " a " + getDegrees() + "°";
21 }
22 }

```

Soluzione all switch

si associano i dati direttamente a ogni costante, spostando la conoscenza dalle funzioni (metodi) ai dati stessi.

si definisce un costruttore privato che accetta i dati e li salva in campi dell'istanza.

```

1  public enum Direction {
2      // Le costanti ora invocano il costruttore con i loro dati specifici
3      NORTH("Nord", 0),
4      SOUTH("Sud", 180),
5      EAST("Est", 90),
6      WEST("Ovest", 270);
7
8      // Campi per lo stato
9      private String italianName;
10     private int degrees;
11
12     // Costruttore privato
13     private Direction(String italianName, int degrees) {
14         this.italianName = italianName;
15         this.degrees = degrees;
16     }
17
18     // I metodi ora restituiscono semplicemente lo stato dell'istanza
19     public int getDegrees() {
20         return degrees;
21     }
22
23     @Override
24     public String toString() {
25         return italianName + " a " + degrees + "°";
26     }
27
28     // Il metodo getOpposite, purtroppo, non può essere implementato con dati,
29     // perché l'opposto è una forward reference.
30     public Direction getOpposite() {
31         return switch (this) {
32             case NORTH -> SOUTH;
33             case SOUTH -> NORTH;
34             case EAST  -> WEST;
35             case WEST  -> EAST;
36         };
37     }
38 }

```

Limitazioni e Workaround per Riferimenti Circolari

Non è possibile, durante la dichiarazione di una costante, riferirsi a un'altra costante che non è stata ancora definita (*forward reference*). Questo impedisce, ad esempio, di passare l'oggetto `SOUTH` al costruttore di `NORTH` per il metodo `getOpposite`.

```

1  // NON COMPILA! 'SOUTH' è ancora sconosciuto quando si definisce NORTH.
2  NORTH("Nord", 0, SOUTH),

```

Workaround: Si può passare il **nome** della costante opposta come stringa e usare il metodo `valueOf()` per ottenere l'istanza corretta al momento dell'esecuzione. (Metodo comunque fragile)

Es. Banconote e Portafogli

```

1  public enum Taglio {
2      CINQUECENTO(500), DUECENTO(200), /* ... */ UNO(1);
3
4      private final int valore;
5
6      private Taglio(int valore) {
7          this.valore = valore;
8      }
9
10     public int getValore() {
11         return valore;
12     }
13 }

```

il **Portafoglio** stesso è un'entità del mondo reale e merita una sua classe. Questo incapsula i dati (l'array di tagli) e le operazioni (metodi `valore()` e `toString()`).

```
1  import java.util.Arrays;
2  import java.util.StringJoiner;
3
4  public class Portafoglio {
5      private Taglio[] contenuto;
6      private int logicalSize; // Tiene traccia degli elementi effettivamente inseriti
7
8      // Costruttore per un portafoglio vuoto con una capacità iniziale
9      public Portafoglio(int capacita) {
10         contenuto = new Taglio[capacita];
11         logicalSize = 0;
12     }
13
14     // Costruttore da un array esistente
15     public Portafoglio(Taglio[] tagliIniziali) {
16         this.contenuto = Arrays.copyOf(tagliIniziali, tagliIniziali.length);
17         this.logicalSize = tagliIniziali.length;
18     }
19
20     public void add(Taglio t) {
21         if (logicalSize < contenuto.length) {
22             contenuto[logicalSize++] = t;
23         } else {
24             // Gestire l'overflow (es. ridimensionare l'array)
25             System.out.println("Portafoglio pieno!");
26         }
27     }
28
29     public int valore() {
30         int sum = 0;
31         for (int i = 0; i < logicalSize; i++) {
32             sum += contenuto[i].getValore(); // Chiede il valore al taglio!
33         }
34         return sum;
35     }
36
37     @Override
38     public String toString() {
39         StringJoiner sj = new StringJoiner(", ");
40         for (int i = 0; i < logicalSize; i++) {
41             sj.add(contenuto[i].toString());
42         }
43         return sj.toString();
44     }
45 }
```

il main è pulito

```
1  Portafoglio pf = new Portafoglio(10);
2  pf.add(Taglio.CINQUANTA);
3  pf.add(Taglio.VENTI);
4  // ...
5
6  System.out.println("Contenuto: " + pf); // Stampa: CINQUANTA, VENTI, ...
7  System.out.println("Valore: " + pf.valore()); // Stampa: 70
```

- **Enum Evoluto (con stato):** Incapsula la conoscenza, eliminando gli switch per le proprietà intrinseche. Il codice diventa più robusto e auto-documentante.
- **Nascita del Portafoglio (classe contenitore):** Incapsula la collezione e le operazioni correlate, dando una "casa" a metodi che altrimenti sarebbero rimasti in librerie statiche senza una chiara appartenenza. Il modello finale rispecchia la realtà, è robusto, estendibile e manutenibile.

Il pattern Factory

Il pattern **Factory** è un design pattern creazionale che incapsula la logica di costruzione degli oggetti, nascondendone i dettagli all'utente. Viene utilizzato quando:

- La **costruzione di un oggetto è complessa** e richiede molti parametri.
 - Si vogliono **applicare regole o politiche specifiche durante la creazione**.
 - Si desidera **restituire oggetti di tipi diversi ma compatibili in base al contesto**.
- Caratteristiche principali**
- La fabbrica espone uno o più **metodi factory statici** (tipicamente chiamati `of()`, `valueOf()`, o in passato `getXYZ()`).

- I costruttori della classe dell'oggetto vengono resi non pubblici, così che solo la fabbrica possa istanziare l'oggetto.
- La fabbrica può essere incorporata nella stessa classe dell'oggetto (parte statica) o essere una classe separata.

es.

```

1  public class Prodotto {
2      private String nome;
3      private double prezzo;
4
5      // Costruttore privato
6      private Prodotto(String nome, double prezzo) {
7          this.nome = nome;
8          this.prezzo = prezzo;
9      }
10
11     // Metodo factory statico
12     public static Prodotto of(String nome, double prezzo) {
13         // eventuali controlli o logiche aggiuntive
14         if (prezzo < 0) {
15             throw new IllegalArgumentException("Il prezzo non può essere negativo");
16         }
17         return new Prodotto(nome, prezzo);
18     }
19 }
20
21 // Utilizzo
22 Prodotto p = Prodotto.of("Laptop", 999.99);

```

Cultura locale `Locale`

La classe `java.util.Locale` rappresenta una specifica cultura locale (lingua + paese) e definisce le regole per la formattazione e il parsing di dati come **numeri**, **valute**, **date**, **orari**.

Componenti di un `Locale`

- **Lingua**: codice ISO a due lettere minuscole (es. `it`, `en`, `fr`).
 - **Paese**: codice ISO a due lettere maiuscole (es. `IT`, `US`, `CH`).
- Esempi: `it_IT` (italiano Italia), `it_CH` (italiano Svizzera), `en_US` (inglese USA), `en_GB` (inglese Regno Unito).

Ottenere un oggetto `Locale`

- **Costanti predefinite** (solo per i casi più comuni):

```
1  Locale.ITALY, Locale.US, Locale.CANADA_FRENCH, Locale.UK, ...
```

- **Costruttori** (fino a Java 18):

```
1  Locale l = new Locale("it", "CH"); // italiano Svizzera
```

- **Metodi factory** (da Java 19):

```

1  Locale l1 = Locale.of("it", "CH");
2  Locale l2 = Locale.forLanguageTag("it-CH");

```

`Locale` predefinito e disponibili

- **`Locale` predefinito** del sistema:

```
1  Locale defaultLocale = Locale.getDefault();
```

- **Tutti i `Locale` disponibili** nella JVM:

```
1  Locale[] available = Locale.getAvailableLocales();
```

es.

```

1  Locale italia = Locale.ITALY;           // it_IT
2  Locale svizzeraItaliana = Locale.of("it", "CH"); // it_CH
3  Locale usa = Locale.US;                 // en_US
4
5  System.out.println(italia.getDisplayName()); // Italian (Italy)
6  System.out.println(usa.getDisplayName());    // English (United States)

```

Formattatori numerici: `NumberFormat`

La classe `java.text.NumberFormat` fornisce metodi per formattare e analizzare numeri, percentuali e valute in base a una specifica cultura locale. Essa stessa funge da factory per i propri oggetti.

I metodi factory principali sono:

- `NumberFormat.getNumberInstance(Locale locale)` – per numeri generici.
- `NumberFormat.getPercentInstance(Locale locale)` – per percentuali.
- `NumberFormat.getCurrencyInstance(Locale locale)` – per valute.

Se non si specifica un Locale, viene usato quello predefinito.

es.

```
1  double x = 43.12345678;
2  double y = 0.7;
3  double z = 13456.78;
4
5  NumberFormat fN = NumberFormat.getNumberInstance(Locale.ITALY);
6  NumberFormat fP = NumberFormat.getPercentInstance(Locale.ITALY);
7  NumberFormat fV = NumberFormat.getCurrencyInstance(Locale.ITALY);
8
9  System.out.println(fN.format(x)); // 43,123 (a seconda delle cifre decimali predefinite)
10 System.out.println(fP.format(y)); // 70%
11 System.out.println(fV.format(z)); // 13.456,78 € (nota: il simbolo € è in fondo)
```

È possibile controllare il numero minimo e massimo di cifre frazionarie:

```
1  NumberFormat fN = NumberFormat.getNumberInstance(Locale.CANADA);
2  fN.setMaximumFractionDigits(2);
3  System.out.println(fN.format(43.12345678)); // 43.12
```

Differenze tra culture

Lo stesso numero viene formattato in modo diverso a seconda del Locale:

```
1  double valore = 12345.678;
2
3  NumberFormat it = NumberFormat.getNumberInstance(Locale.ITALY);
4  NumberFormat us = NumberFormat.getNumberInstance(Locale.US);
5  NumberFormat frCA = NumberFormat.getNumberInstance(Locale.CANADA_FRENCH);
6
7  System.out.println(it.format(valore)); // 12.345,678
8  System.out.println(us.format(valore)); // 12,345.678
9  System.out.println(frCA.format(valore)); // 12 345,678 (nota: spazio come separatore migliaia)
```

Attenzione: il separatore delle migliaia per `Locale.CANADA_FRENCH` è uno spazio **hard** (non divisibile), non un semplice spazio ASCII.

Parsing di stringhe numeriche

I formattatori supportano anche l'operazione inversa: convertire una stringa (formattata secondo le regole di una certa cultura) in un oggetto `Number` tramite il metodo `parse(String source)`.

Metodo `parse`

```
1  NumberFormat fV = NumberFormat.getCurrencyInstance(Locale.US);
2  try {
3      Number n = fV.parse("$123.56");
4      double d = n.doubleValue();
5      System.out.println(d); // 123.56
6  } catch (ParseException e) {
7      e.printStackTrace();
8  }
```

Il metodo restituisce un `Number` (che può essere convertito in `double`, `float`, `int`, ecc.) e può lanciare un'eccezione `ParseException` se la stringa non è conforme.

Esempi con jshell (senza gestione eccezioni)

```
1  jshell> NumberFormat fV = NumberFormat.getCurrencyInstance(Locale.US)
2  jshell> fV.parse("$123.456789987654321")
3  $3 ==> 123.45678998765432
4
5  jshell> double d = n.doubleValue()
6  d ==> 123.45678998765432
```

il parsing è rigoroso

Con culture diverse il comportamento cambia e può essere inaspettato se la stringa non rispetta esattamente il formato previsto.

Esempio problematico con Locale.ITALY e percentuali:

```
1 NumberFormat fP = NumberFormat.getPercentInstance(Locale.ITALY);
2 // La stringa "70%" viene interpretata erroneamente (ignora il % e restituisce 70, non 0.7)
3 Number n = fP.parse("70%"); // ATTENZIONE: risultato 70, non 0.7!
```

Esempio con Locale.CANADA_FRENCH:

```
1 NumberFormat fP = NumberFormat.getPercentInstance(Locale.CANADA_FRENCH);
2 fP.setMinimumFractionDigits(2);
3 String formatted = fP.format(0.7235); // "72,35 %" (con spazio hard prima di %)
4 // Per fare il parsing corretto, la stringa deve contenere lo spazio hard e la virgola
5 Number n = fP.parse("72,35 %"); // spazio non divisibile
```

Se si omette lo spazio o si usa uno spazio normale, il parsing fallisce.

il parsing è delicato; in questa fase è meglio limitarsi alla formattazione (output) e rimandare lo studio approfondito del parsing a quando si affronterà la gestione delle eccezioni.

Formattatori personalizzati con DecimalFormat

è possibile creare un formattatore personalizzato usando la classe `java.text.DecimalFormat`.

Il pattern utilizza simboli speciali:

- ¤ (U+00A4) – simbolo di valuta generico (verrà sostituito con il simbolo locale).
- # – cifra opzionale (non viene mostrata se zero iniziale).
- 0 – cifra obbligatoria (viene mostrata anche se zero).
- , – separatore di gruppo (migliaia).
- . – separatore decimale.

Nota: nel pattern si usano i simboli anglosassoni (. per il decimale, , per le migliaia), ma il formattatore li sostituirà automaticamente con i separatori della cultura locale al momento della formattazione.

Es

```
1 DecimalFormat f = new DecimalFormat("¤ #,##0.##");
2 f.setCurrency(Currency.getInstance("EUR"));
3
4 System.out.println(f.format(1234.567)); // € 1.234,57
5 System.out.println(f.format(-1234.567)); // € -1.234,57
6 System.out.println(f.format(0.567)); // € 0,57
7 System.out.println(f.format(12345678.91234)); // € 12.345.678,91
```

È possibile specificare due pattern separati da un punto e virgola: il primo per i valori positivi, il secondo per i negativi.

```
1 DecimalFormat f = new DecimalFormat("¤ #,##0.##; ¤ -#,##0.##");
2 f.setCurrency(Currency.getInstance("EUR"));
3
4 System.out.println(f.format(1234.567)); // € 1.234,57
5 System.out.println(f.format(-1234.567)); // € -1.234,57
```

A partire da **Java 9**, il JDK ha adottato il database Unicode **CLDR** (Common Locale Data Repository) come database predefinito per le informazioni sulle culture locali. Questo cambiamento ha introdotto una modifica significativa per la valuta in **Locale.ITALY**:

- Fino a Java 8:** il simbolo dell'euro veniva posizionato **prima** dell'importo (es. € 1.234,56).
- Da Java 9 in poi:** il simbolo dell'euro viene posizionato **dopo** l'importo (es. 1.234,56 €), seguendo le convenzioni CLDR.

Java.time

La gestione del tempo nelle applicazioni software è complessa per vari motivi:

- Concetti relativi vs. assoluti:** "ci vediamo alle 10" (relativo al luogo) vs. "l'aereo parte alle 11:30 GMT+1" (assoluto).
- Convenzioni culturali diverse:** formati di data, nomi dei mesi, calendari differenti (gregoriano, giuliano, ebraico, islamico...).
- Unità di misura variabili:** un mese può durare 28, 29, 30 o 31 giorni; un anno può essere bisestile.

Pattern Factory in java.time

Tutte le classi principali di `java.time` seguono due assunti fondamentali:

- Oggetti immutabili:** una volta creati, non possono più essere modificati. Qualsiasi operazione che sembra modificarli restituisce in realtà una nuova istanza.

2. **Costruzione indiretta tramite factory:** i costruttori non sono pubblici. Si utilizzano metodi factory statici (principalmente `of()`, ma anche `now()`, `from()`, `parse()`, ecc.) per creare oggetti.
Questo approccio garantisce:

- Controllo sulla creazione (evita duplicati, applica validazioni).
- Chiarezza e leggibilità del codice.

Esempi di factory methods:

```
1 // Invece di "new LocalDate(2020, 12, 25)" (NON FUNZIONA)
2 LocalDate xmas = LocalDate.of(2020, 12, 25);
3
4 Month m = Month.of(10);           // OCTOBER
5 DayOfWeek d = DayOfWeek.of(1);   // MONDAY
6
7 LocalTime noon = LocalTime.of(12, 0);
8 LocalTime now = LocalTime.now();
9
10 LocalDateTime nowDateTime = LocalDateTime.now();
```

Concetti relativi (locali)

I concetti relativi rappresentano data e/o orario **senza riferimento a un fuso orario**. Sono utili per esprimere eventi che hanno significato in un contesto locale.

Classe	Descrizione	Esempio di formato
LocalDate	Data (anno, mese, giorno)	2020-12-25
LocalTime	Orario (ora, minuto, secondo, nanosecondo)	12:00:00
LocalDateTime	Data e orario combinati	2020-12-25T12:00:00

Creazione:

```
1 LocalDate dataNascita = LocalDate.of(1990, 5, 15);
2 LocalTime oraLezione = LocalTime.of(9, 30);
3 LocalDateTime dataOra = LocalDateTime.of(2026, 3, 17, 14, 0);
```

Metodi accessor:

```
1 LocalDate oggi = LocalDate.now();
2 int giorno = oggi.getDayOfMonth();
3 Month mese = oggi.getMonth();           // restituisce un enum Month
4 int anno = oggi.getYear();
5 int giornoAnno = oggi.getDayOfYear();   // 1..366
6 DayOfWeek giornoSettimana = oggi.getDayOfWeek(); // enum DayOfWeek
7 boolean bisestile = oggi.isLeapYear();
```

Enumerativi: Month e DayOfWeek

`Month` e `DayOfWeek` sono enumerazioni che facilitano la gestione di mesi e giorni.

```
1 Month m = Month.APRIL;
2 System.out.println(m.getValue());       // 4
3 System.out.println(m.getDisplayName(...)); // "aprile" (se si usa un formattatore)
4
5 DayOfWeek d = DayOfWeek.MONDAY;
6 System.out.println(d.getValue());       // 1 (lunedì = 1, domenica = 7)
```

Period: durata relativa

`Period` rappresenta un lasso di tempo in **anni, mesi, giorni**. È "volutamente impreciso" perché anni e mesi non hanno durata fissa.

Creazione:

```
1 Period dueMesiTreGiorni = Period.ofMonths(2).plusDays(3); // P2M3D
2 Period unAnno = Period.ofYears(1);                        // P1Y
```

Differenza tra due date:


```

1  LocalDate inizio = LocalDate.of(2025, 9, 15);
2  LocalDate fine = LocalDate.of(2026, 6, 10);
3  Period periodo = Period.between(inizio, fine);
4  System.out.println(periodo.getYears() + " anni, " + periodo.getMonths() + " mesi, " + periodo.getDays() + " giorni");

```

Sommare/sottrarre un Period a una data:

```

1  LocalDate nuovaData = (LocalDate) periodo.addTo(inizio); // richiede cast (vedi spiegazione sotto)
2
3  LocalDate nuovaData = inizio.plus(periodo); // versione ottimale

```

Perché il cast? I metodi `addTo` e `subtractFrom` lavorano su oggetti `Temporal` (interfaccia generica). Il tipo effettivo dipende dall'input: se passo una `LocalDate`, ottengo una `LocalDate`; se passo una `LocalDateTime`, ottengo una `LocalDateTime`. Il cast è necessario per riottenere il tipo specifico.

Concetti assoluti

I concetti assoluti rappresentano un **istante preciso sulla linea del tempo**, indipendente dal luogo in cui ci si trova. Richiedono l'indicazione del fuso orario o dell'offset da UTC.

Instant: punto sulla linea del tempo

`Instant` è il concetto più "fisico": è il numero di nanosecondi trascorsi dalla mezzanotte del 1° gennaio 1970 UTC (epoca Unix). Non ha nulla a che fare con calendari, mesi o giorni umani.

```

1  Instant adesso = Instant.now(); // 2026-03-17T14:30:00.123456789Z (formato ISO)

```

Duration: durata in nanosecondi

`Duration` rappresenta un lasso di tempo **preciso** in nanosecondi. Tipicamente si usa per differenze tra `Instant`.

Creazione:

```

1  Duration unGiorno = Duration.ofDays(1);
2  Duration dueOre = Duration.ofHours(2);
3  Duration complessa = Duration.ofDays(1)
4                      .plusHours(3)
5                      .minusMinutes(4)
6                      .minusSeconds(10); // fluent interface

```

Differenza tra due Instant:

```

1  Instant inizio = Instant.now();
2  // ... esecuzione di un'operazione
3  Instant fine = Instant.now();
4  Duration durata = Duration.between(inizio, fine);
5  System.out.println("Durata: " + durata.toMillis() + " ms");

```

Metodi per estrarre il totale in varie unità:

```

1  long totaleOre = durata.toHours();
2  long totaleMinuti = durata.toMinutes();
3  long totaleSecondi = durata.toSeconds();
4  long totaleMillis = durata.toMillis();

```

OffsetDateTime e ZonedDateTime

Queste classi aggiungono a una data/ora locale l'informazione necessaria per renderla assoluta.

Classe	Descrizione	Esempio
<code>OffsetDateTime</code>	Data/ora + offset fisso rispetto a UTC (es. +01:00)	2026-03-17T15:30:00+01:00
<code>ZonedDateTime</code>	Data/ora + fuso orario (es. Europe/Rome), che include regole per ora legale	2026-03-17T15:30:00+01:00 Europe/Rome

Creazione:

```

1 LocalDateTime inizioLezioni = LocalDateTime.of(2026, 2, 17, 9, 0);
2
3 // OffsetDateTime con offset +1 (ora solare Italia)
4 OffsetDateTime offsetInizio = OffsetDateTime.of(inizioLezioni, ZoneOffset.ofHours(1));
5
6 // ZonedDateTime con fuso CET (Central European Time)
7 ZonedDateTime zonedInizio = ZonedDateTime.of(inizioLezioni, ZoneId.of("Europe/Rome"));

```

Ottenere fusi orari:

- Usare **nomi estesi** come "Europe/Rome", "America/New_York". Le sigle a tre lettere (CET , EST) sono deprecate perché ambigue.
- `ZoneId.getAvailableZoneIds()` per elencare tutti i fusi supportati.

Conversione tra classi:

```

1 ZonedDateTime zdt = ZonedDateTime.now();
2 OffsetDateTime odt = zdt.toOffsetDateTime();
3 Instant instant = zdt.toInstant(); // da ZonedDateTime a Instant
4 LocalDate data = zdt.toLocalDate();
5 LocalTime ora = zdt.toLocalTime();
6
7 // Da LocalDateTime a Instant (serve l'offset)
8 LocalDateTime ldt = LocalDateTime.now();
9 Instant ist = ldt.toInstant(ZoneOffset.ofHours(1));

```

Operazioni comuni su date e orari

Tutte le classi di `java.time` offrono metodi per manipolare le date in modo **immutabile**: restituiscono sempre una nuova istanza.

Aggiungere/sottrarre tempo: `plusXxx()` / `minusXxx()`

```

1 LocalDate oggi = LocalDate.now();
2 LocalDate domani = oggi.plusDays(1);
3 LocalDate meseProssimo = oggi.plusMonths(1);
4 LocalDate annoScorso = oggi.minusYears(1);
5
6 LocalTime ora = LocalTime.now();
7 LocalTime fraUnOra = ora.plusHours(1);
8 LocalTime dieciMinutiFa = ora.minusMinutes(10);

```

Modificare componenti specifici: `withXxx()`

```

1 LocalDate data = LocalDate.of(2026, 3, 17);
2 LocalDate stessoGiornoMeseDiverso = data.withMonth(12).withDayOfMonth(25); // 2026-12-25
3
4 LocalTime ora = LocalTime.of(10, 30);
5 LocalTime stessaOraSecondiZero = ora.withSecond(0);

```

Confronti: `isBefore()`, `isAfter()`, `isEqual()`

```

1 LocalDate d1 = LocalDate.of(2026, 1, 1);
2 LocalDate d2 = LocalDate.of(2026, 12, 31);
3
4 if (d1.isBefore(d2)) {
5     System.out.println("d1 viene prima di d2");
6 }
7
8 ZonedDateTime z1 = ZonedDateTime.now();
9 ZonedDateTime z2 = z1.plusHours(2);
10 if (z2.isAfter(z1)) {
11     System.out.println("z2 è successivo a z1");
12 }

```

es.

Quanto manca al prossimo compleanno?

```

1 public static Period toNextBirthDay(LocalDate dateOfBirth) {
2     LocalDate today = LocalDate.now();
3     int currentYear = today.getYear();
4     LocalDate nextBirthDay = dateOfBirth.withYear(currentYear);
5     if (nextBirthDay.isBefore(today)) {
6         nextBirthDay = dateOfBirth.withYear(currentYear + 1);
7     }
8     return Period.between(today, nextBirthDay);

```

```

9  }
10
11 // Utilizzo
12 LocalDate mioCompleanno = LocalDate.of(1990, 5, 15);
13 Period attesa = toNextBirthDay(mioCompleanno);
14 System.out.println("Giorni: " + attesa.getDays() + ", Mesi: " + attesa.getMonths());

```

Effetto dell'ora legale

```

1  ZonedDateTime inizio = ZonedDateTime.of(2026, 3, 25, 10, 0, 0, 0, ZoneId.of("Europe/Rome"));
2  ZonedDateTime fine = inizio.plusDays(10);
3
4  Duration differenza = Duration.between(inizio, fine);
5  System.out.println(differenza.toHours()); // 239 ore (non 240!) perché a fine marzo scatta l'ora legale

```

Durata tra due date espresse come LocalDateTime (ma attenzione!)

```

1  LocalDate d1 = LocalDate.of(2024, 3, 12);
2  LocalDateTime dt1 = LocalDateTime.of(d1, LocalTime.now());
3
4  // Calcolo durata tra date con offset fisso
5  OffsetDateTime od1 = OffsetDateTime.of(dt1, ZoneOffset.ofHours(1));
6
7  long giorni1 = Duration.between(od1.plusMonths(3), od1.plusMonths(5)).toDays(); // 61
8  long giorni2 = Duration.between(od1.plusMonths(4), od1.plusMonths(6)).toDays(); // 62

```

La differenza è dovuta ai diversi numeri di giorni nei mesi (giugno-agosto vs. luglio-settembre).

Cheatsheet

- `of / ofXxx` → factory methods per creare oggetti.
- `now` → ottiene l'oggetto corrispondente all'istante corrente (o data/ora corrente).
- `getXxx` → restituisce una componente (es. `getYear`, `getHour`).
- `withXxx` → modifica una componente (es. `withYear`, `withMinute`).
- `plusXxx / minusXxx` → aggiunge/sottrae una quantità di tempo.
- `isBefore / isAfter / isEqual` → confronti.
- `toXxx` → converte in un altro tipo (es. `toLocalDate`, `toInstant`).

Importante: tutti i metodi che sembrano modificare l'oggetto restituiscono **una nuova istanza**; l'originale rimane invariato.

Formattatori per date e orari

La classe principale per la formattazione di date e orari in Java è `DateTimeFormatter`, presente nel package `java.time.format`. A differenza dei formattatori numerici (`NumberFormat`), qui:

- La costruzione avviene tramite **metodi factory** (non `new`).
- È disponibile una **duplice sintassi**:

```

1  String s = formatter.format(value); // stile classico
2  String s = value.format(formatter); // stile fluente (preferibile)

```

Creazione di formattatori localizzati

Per formattare secondo le convenzioni di una cultura locale, si usano i metodi factory:

Metodo	Destinazione
<code>ofLocalizedDate(FormatStyle)</code>	solo data (<code>LocalDate</code>)
<code>ofLocalizedTime(FormatStyle)</code>	solo orario (<code>LocalTime</code>)
<code>ofLocalizedDateTime(FormatStyle)</code>	data+orario (<code>LocalDateTime</code>) – stesso stile per entrambi
<code>ofLocalizedDateTime(dateStyle, timeStyle)</code>	stili distinti per data e orario

FormatStyle è un enum con quattro valori: `SHORT`, `MEDIUM`, `LONG`, `FULL`.

- Per le **date** sono tutti ammessi.
- Per gli **orari** sono ammessi solo `SHORT` e `MEDIUM` (l'uso di `LONG / FULL` su `LocalTime` genera eccezione).

Nota: il formattatore appena creato usa il **Locale di default** della JVM. Per cambiare cultura si usa il metodo `withLocale(Locale)`:

```

1  DateTimeFormatter f = DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT)
2  .withLocale(Locale.ITALY);

```

Es.

```

1 // dichiarazioni
2 System.out.println(d.format(shortF)); // 04/03/26
3 System.out.println(d.format(mediumF)); // 4 mar 2026
4 System.out.println(d.format(longF)); // 4 marzo 2026
5 System.out.println(d.format(fullF)); // mercoledì 4 marzo 2026

```

Cambiando il Locale:

```

1 Locale fr = Locale.FRANCE;
2 System.out.println(d.format(shortF.withLocale(fr))); // 04/03/2026
3 System.out.println(d.format(mediumF.withLocale(fr))); // 4 mars 2026
4 System.out.println(d.format(longF.withLocale(fr))); // 4 mars 2026
5 System.out.println(d.format(fullF.withLocale(fr))); // mercredi 4 mars 2026

```

Formattazione di `LocalTime`

Solo `SHORT` e `MEDIUM` sono validi:

```

1 //dichiarazioni
2 System.out.println(t.format(shortTime)); // 18:37
3 System.out.println(t.format(mediumTime)); // 18:37:41

```

Formattazione di `LocalDateTime`

Con stile unico per data e ora:

```

1 LocalDateTime dt = LocalDateTime.of(2026, 3, 4, 18, 37, 41);
2 DateTimeFormatter f = DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT);
3 System.out.println(dt.format(f)); // 04/03/26, 18:37

```

Con stili distinti:

```

1 DateTimeFormatter f = DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT, FormatStyle.MEDIUM);
2 System.out.println(dt.format(f)); // 04/03/26, 18:37:41

```

Sono possibili tutte le 8 combinazioni (4 stili data × 2 stili ora).

Formattazione di date e orari assoluti (`ZonedDateTime` , `OffsetDateTime`)

Le classi `ZonedDateTime` e `OffsetDateTime` si formattano con gli stessi metodi, ma con una differenza importante: per `ZonedDateTime` gli stili `LONG` e `FULL` dell'ora mostrano anche il **fuso orario** (e l'ora legale se attiva). Per `OffsetDateTime` invece lo stile `LONG` / `FULL` non aggiunge informazioni aggiuntive perché l'offset è già noto.

```

1 ZonedDateTime zdt = ZonedDateTime.now(); // es. 2026-03-04T18:37:41+01:00[Europe/Rome]
2 DateTimeFormatter longTime = DateTimeFormatter.ofLocalizedTime(FormatStyle.LONG);
3 System.out.println(zdt.format(longTime)); // 18:37:41 CET (oppure CEST se ora legale)

```

Attenzione: la rappresentazione del fuso dipende da come è stato creato lo `ZonedDateTime`. Usare `ZoneId.of("Europe/Rome")` è più preciso di "CET" o "GMT+1". La forma `GMT±n` produce un output più povero (es. `18:37:41 GMT+01:00`).

Formattatori personalizzati con pattern (`ofPattern`)

Se gli stili localizzati non soddisfano, si può definire un pattern con `DateTimeFormatter.ofPattern(String pattern, Locale)`.

Esempio: data in formato `dd/MM/yyyy` :

```

1 DateTimeFormatter f = DateTimeFormatter.ofPattern("dd/MM/yyyy");
2 System.out.println(LocalDate.now().format(f)); // 04/03/2026

```

Simbolo	Significato	Esempi
y	anno	yy → 26, yyyy → 2026
M	mese (numero/testo)	M → 3, MM → 03, MMM → mar, MMMM → marzo
d	giorno del mese	d → 4, dd → 04
E	giorno della settimana (testo)	E → mer, EEEE → mercoledì
h	ora in formato 12 ore (1-12)	h → 6
H	ora in formato 24 ore (0-23)	H → 18
m	minuti	m → 37
s	secondi	s → 41
a	AM/PM	a → PM
z	nome del fuso orario	z → CET, zzzz → Central European Time
Z	offset di zona (stile ISO)	Z → +0100

Simbolo	Significato	Esempi
'	testo letterale	'alle' → "alle"

Parsing di date e orari

Il parsing può essere effettuato in due modi:

- Tramite il formattatore (restituisce un `TemporalAccessor`)

```
1 DateTimeFormatter fmt = DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT)
2                               .withLocale(Locale.ITALY);
3 TemporalAccessor ta = fmt.parse("12/02/26");
4 LocalDate d = LocalDate.from(ta);           // 2026-02-12
```

Il `TemporalAccessor` è un contenitore generico; lo si converte con i metodi `from()` delle classi specifiche (`LocalDate.from()` , `LocalTime.from()` , `LocalDateTime.from()`).

- Tramite i metodi `parse` delle classi `LocalDate` , `LocalTime` , ecc.

```
1 DateTimeFormatter fmt = DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT, FormatStyle.SHORT)
2                               .withLocale(Locale.ITALY);
3 LocalDateTime dt = LocalDateTime.parse("12/02/26, 12:30", fmt);
4 LocalDate d = LocalDate.parse("12/02/26, 12:30", fmt);    // solo data
5 LocalTime t = LocalTime.parse("12/02/26, 12:30", fmt);    // solo ora
```

Vantaggio: il tipo di ritorno è direttamente quello desiderato.

Attenzione: il parsing può sollevare eccezioni se la stringa non è conforme al formato e alla cultura. In jshell le eccezioni vengono gestite automaticamente, ma in un programma reale vanno gestite con `try-catch` .

Es. parsing del nome del mese

Obiettivo: ottenere il numero del mese a partire dal nome del mese in una data lingua.

```
1 public static int getMonth(String monthName, Locale locale) {
2     DateTimeFormatter f = DateTimeFormatter.ofPattern("MMMM", locale);
3     TemporalAccessor ta = f.parse(monthName);
4     return ta.get(ChronoField.MONTH_OF_YEAR);
5 }
6 // Utilizzo
7 System.out.println(getMonth("Maggio", Locale.ITALY));    // 5
8 System.out.println(getMonth("May", Locale.ENGLISH));     // 5
9 System.out.println(getMonth("mai", Locale.of("nl", "NL"))); // 5 (olandese)
```

Nota: il pattern `"MMMM"` richiede il nome completo del mese, in maiuscolo/minuscolo come previsto dalla lingua (es. in italiano "Maggio", in inglese "May", in olandese "mei"). Se necessario, si può normalizzare la stringa prima del parsing.

Formattatori ISO predefiniti

`DateTimeFormatter` fornisce costanti per formati ISO comuni, adatti allo scambio machine-to-machine:

Costante	Esempio
<code>ISO_LOCAL_DATE</code>	2026-03-04
<code>ISO_LOCAL_TIME</code>	18:37:41
<code>ISO_LOCAL_DATE_TIME</code>	2026-03-04T18:37:41
<code>ISO_OFFSET_DATE_TIME</code>	2026-03-04T18:37:41+01:00
<code>ISO_ZONED_DATE_TIME</code>	2026-03-04T18:37:41+01:00[Europe/Rome]
<code>BASIC_ISO_DATE</code>	20260304
<code>RFC_1123_DATE_TIME</code>	Thu, 4 Mar 2026 18:37:41 GMT

```
1 ZonedDateTime zdt = ZonedDateTime.now();
2 System.out.println(zdt.format(DateTimeFormatter.ISO_ZONED_DATE_TIME));
```

Cheatsheet

- **Costruzione:** `DateTimeFormatter.ofLocalizedDate/Style/DateTime` (con `FormatStyle`) oppure `ofPattern` .
- **Localizzazione:** usare `withLocale(Locale)` .
- **Formattazione:** `formatter.format(value)` o `value.format(formatter)` .
- **Parsing:** `LocalDate.parse(string, formatter)` o `formatter.parse(string)` con conversione tramite `TemporalAccessor` .
- **Pattern personalizzati:** simboli come `yyyy` , `MM` , `dd` , `HH` , `mm` , `ss` , `MMMM` , `EEEE` , `z` , `Z` .
- **Stili per orari:** solo `SHORT` e `MEDIUM` per `LocalTime` ; per `ZonedDateTime` sono ammessi anche `LONG` e `FULL` (mostrano il fuso).

- **ISO predefiniti:** utili per scambio dati.

Sistemi di software

Un sistema a oggetti è **costituito da classi e oggetti che interagiscono tra loro**. La progettazione di un sistema che risolve un problema richiede diverse fasi:

1. **Analisi del problema** – partendo dai requisiti si costruisce un modello del problema (architettura logica).
2. **Pianificazione del collaudo** e del lavoro.
3. **Progettazione della soluzione** – si definisce l'architettura del sistema (modello della soluzione).
4. **Implementazione** – scrittura del codice.
5. **Collaudo** – verifica dell'implementazione.

Modelli e viste

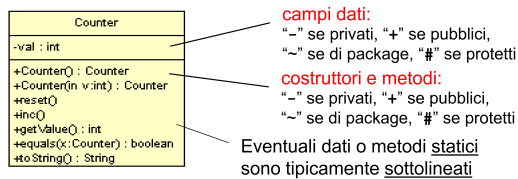
I modelli sono rappresentazioni (spesso grafiche) del sistema. Si distinguono:

- **Architettura logica** – descrive il problema, indipendentemente dalla soluzione. Si articola in viste:
 - **Struttura:** macro-blocchi derivanti dai requisiti (diagramma delle classi generale).
 - **Interazione:** relazioni dinamiche tra blocchi (diagrammi di sequenza).
 - **Comportamento** osservabile dei singoli blocchi (diagrammi degli stati).
- **Architettura del sistema** – descrive la soluzione specifica. Viste analoghe ma di dettaglio.
 - Struttura: classi di progetto (anche 10-100 volte più classi).
 - Interazione: dettaglio di come avviene.
 - Comportamento di dettaglio.

UML (Unified Modeling Language)

è il linguaggio grafico standard per esprimere questi modelli. Supporta tutte le fasi e permette forward/reverse engineering.

Classi



Dipendenza

Una classe **dipende** da un'altra se la usa (ad esempio come parametro, tipo di ritorno, o creazione locale). In UML si rappresenta con una **freccia tratteggiata** etichettata `<<use>>`.

- CASO 1: un'operazione della classe A richiede come argomento una istanza della classe B

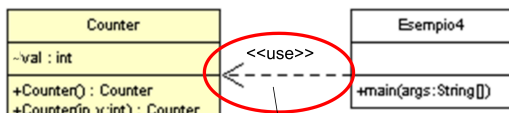
```
1 void fun1(B b) { ... usa b ... }
```

- CASO 2: un'operazione della classe A restituisce un oggetto di tipo B

```
1 B fun2(...) { B b; ... return b; }
```

- CASO 3: un'operazione della classe A utilizza una specifica istanza della classe B, senza che però esista un'associazione tra A e B

```
1 void fun3(...) { B b = new B(...); ... usa b ... }
```



Associazioni

Relazione strutturale tra classi. Può essere unidirezionale o bidirezionale, con molteplicità (es. 1, 0.., 1..). fondamentale distinguere i diversi significati del verbo "avere" nelle descrizioni. Esempi:

- "Una flotta ha un ammiraglio" → **relazione generica** (non è parte della flotta).
- "Una flotta ha delle navi" → **aggregazione** (le navi esistono indipendentemente).
- "Un esagono ha sei vertici" → **composizione** (i vertici esistono solo con l'esagono).
- "Un libro ha delle pagine" → **composizione** (le pagine non hanno senso fuori dal libro).

Associazione generica



Ogni classe contiene un riferimento all'altra. Se la molteplicità è maggiore di 1, si usa una collezione (array, List, ecc.).

Esempio: Ammiraglio e Flotta (un ammiraglio può essere associato a più flotte).

```

1  public class Ammiraglio {
2      private List<Flotta> flotte; // o array
3      // ...
4  }
5
6  public class Flotta {
7      private Ammiraglio ammiraglio;
8      // ...
9  }
  
```

Aggregazione



La classe contenitore ha un riferimento a oggetti che esistono indipendentemente. Non è necessario fare copie difensive nei costruttori perché gli oggetti possono essere condivisi.

Esempio: Flotta e Nave (una nave può appartenere a più flotte).

```

1  public class Flotta {
2      private Nave[] navi; // o List<Nave>
3
4      public Flotta(Nave[] navi) {
5          // NON si fa copia difensiva perché le navi sono indipendenti
6          this.navi = navi;
7      }
8  }
  
```

Composizione



La composizione implica una dipendenza, ma non viceversa.

Le parti hanno ciclo di vita legato al contenitore. Per garantire l'esclusività, nei costruttori si usa la **copia difensiva** (se gli oggetti sono mutabili) e si impedisce la condivisione.

Esempio: Triangolo e Vertici (un triangolo è composto esattamente da tre vertici, che non possono essere condivisi).

```

1  public class Triangolo {
2      private Vertice[] vertici;
3
4      public Triangolo(Vertice[] vertici) {
5          // copia difensiva per evitare modifiche esterne
6          this.vertici = Arrays.copyOf(vertici, vertici.length);
7      }
8
9      public Vertice[] getVertici() {
10         // restituisce una copia per mantenere l'incapsulamento
11         return Arrays.copyOf(vertici, vertici.length);
12     }
13 }
  
```

se gli oggetti parte sono a loro volta immutabili, la copia difensiva potrebbe non essere necessaria, ma è buona norma per evitare sorprese.

Es. orologio slide pag 35

Es. traghetti slide pag 63

Es. Punti e Poligoni slide pag 75

Finestra pag 91

Esercizi Elezioni e 7 segmenti

Ereditarietà

Spesso, nello sviluppo software, ci si trova a dover creare un nuovo componente che è molto simile a uno già esistente, ma con qualche funzionalità aggiuntiva o modificata.

Approcci iniziali (prima di ereditarietà):

1. **Copia e Incolla:** Copiare il codice della classe esistente e modificarlo. È una pessima pratica perché duplica codice, rendendo difficile la manutenzione e l'evoluzione.
2. **Composizione con Adapter:** Creare una nuova classe che contiene (compone) un'istanza della classe esistente.
 - **Vantaggio:** Riutilizza la classe esistente senza modificarne il codice sorgente.
 - **Come funziona:**
 1. La nuova classe incapsula un oggetto della classe esistente.
 2. I metodi già presenti vengono delegati all'oggetto incapsulato.
 3. I nuovi metodi vengono implementati sfruttando l'oggetto incapsulato.
 - **Pattern Adapter:** Questo approccio è una forma del pattern Adapter, che adatta un'interfaccia esistente a una nuova.

Esempio: Counter (solo avanti) a CounterDec (avanti/indietro) con Adapter

```
1 // Classe esistente: Counter (solo incremento)
2 public class Counter {
3     private int val;
4     public Counter() { val = 1; }
5     public Counter(int v) { val = v; }
6     public void reset() { val = 0; }
7     public void inc() { val++; }
8     public int getValue() { return val; }
9     public String toString() { return "Counter di valore " + getValue(); }
10 }
11
12 // Nuova classe CounterDec che usa Adapter
13 public class CounterDec {
14     private final Counter c; // Incapsula un Counter
15
16     public CounterDec() { this.c = new Counter(); }
17     public CounterDec(int value) { this.c = new Counter(value); }
18
19     // Delega dei metodi esistenti
20     public void reset() { c.reset(); }
21     public void inc() { c.inc(); }
22     public int getValue() { return c.getValue(); }
23     public String toString() { return c.toString(); }
24
25     // Nuovo metodo: decremento
26     public void dec() {
27         // Strategia 1: reset e reincremento (inefficiente)
28         int v = c.getValue();
29         c.reset();
30         for (int i = 1; i <= v - 1; i++) {
31             c.inc();
32         }
33
34         // Strategia 2: creazione di un nuovo Counter (più efficiente)
35         // Nota: richiederebbe che c fosse modificabile (non final)
36         // c = new Counter(c.getValue() - 1);
37     }
38 }
```

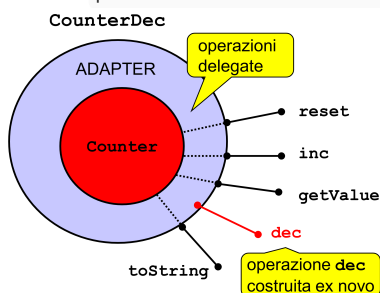
Limiti dell'Adapter:

- I campi privati della classe incapsulata non sono accessibili.
- Bisogna scrivere codice di "delega" anche per i metodi che rimangono identici (es. `reset`, `inc`, `getValue`).
- Non è sempre possibile implementare la nuova funzionalità sfruttando solo i metodi pubblici dell'oggetto incapsulato.

Inheritance

permette di definire una nuova classe (**sottoclasse**, **classe derivata**) a partire da una classe esistente (superclasse, classe base), specificando solo le *differenze*.

Sintassi: `public class NuovaClasse extends ClasseEsistente { ... }`



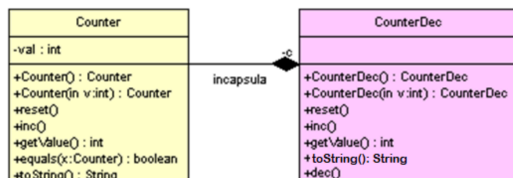
Cosa può fare la classe derivata:

- Aggiungere nuovi campi dati.
- Aggiungere nuovi metodi.
- Ridefinire (override) i metodi ereditati, modificarne il comportamento.

Cosa non può fare:

- Eliminare o nascondere campi o metodi ereditati.

Esempio: Counter2 (con decremento) che estende Counter



```
1 // Classe base Counter (da modificare per l'ereditarietà)
2 public class Counter {
3     // Il campo val non può essere private, altrimenti Counter2 non lo vedrà
4     protected int val; // <--- Livello di protezione protetto
5
6     public Counter() { val = 1; }
7     public Counter(int v) { val = v; }
8     public void reset() { val = 0; }
9     public void inc() { val++; }
10    public int getValue() { return val; }
11 }
12
13 // Classe derivata Counter2
14 public class Counter2 extends Counter {
15     // Aggiunge un nuovo metodo
16     public void dec() {
17         val--; // Ora val è accessibile perché è protected
18     }
19     // Non ha bisogno di ridefinire inc, reset, getValue
20 }
```

Il Modificatore protected

Il problema emerso nell'esempio precedente è l'accesso al campo `val` di `Counter` da parte di `Counter2`. Se `val` è `private`, la classe derivata non può accedervi. Se è `public`, si viola l'incapsulamento.

Soluzione: protected (# in UML)

- Un membro `protected` è accessibile:
 - All'interno della stessa classe.
 - A tutte le classi nello stesso package.
 - A tutte le sottoclassi (anche in package diversi).

Attenzione: L'uso di `protected` va fatto con giudizio. Rende i dati accessibili a *tutte* le sottoclassi presenti e future. Un'alternativa più sicura è mantenere i campi `private` e fornire metodi accessor `protected` per le sottoclassi.

```
1 public class Counter {
2     private int val; // campo privato
3
4     protected int getVal() { return val; } // metodo protetto per leggere
5     protected void setVal(int v) { val = v; } // metodo protetto per scrivere
6
7     // ... costruttori e metodi ...
8 }
9
10 public class Counter2 extends Counter {
11     public void dec() {
12         // setVal(getVal() - 1); // uso dei metodi protetti
13         int current = getVal();
14         setVal(current - 1);
15     }
16 }
```

Ereditarietà e Costruttori

I costruttori **non vengono ereditati**.

Ogni classe è responsabile di inizializzare i propri campi. La classe derivata può avere campi aggiuntivi, e i costruttori della classe base non sanno come inizializzarli.

Meccanismo di costruzione:

1. Quando si crea un oggetto di una classe derivata, viene chiamato il suo costruttore.
2. La prima operazione di ogni costruttore della classe derivata è chiamare un costruttore della classe base.
3. Se non si specifica esplicitamente, il compilatore tenta di chiamare il costruttore di default (senza argomenti) della classe base.
4. Se la classe base non ha un costruttore di default, si verifica un errore di compilazione. Bisogna quindi chiamare esplicitamente un costruttore della classe base usando `super(...)`.

La parola chiave `super` (in Java):

- `super(...)` : Chiama il costruttore della classe base (deve essere la prima istruzione nel costruttore).
- `super.metodo()` : Invoca un metodo della classe base (utile se è stato ridefinito).
- `super.campo` : Accede a un campo della classe base.

Esempio: Counter2 con costruttori espliciti

```
1 public class Counter2 extends Counter {
2     public Counter2() {
3         super(); // Chiama Counter() (opzionale, sarebbe implicito)
4     }
5
6     public Counter2(int v) {
7         super(v); // Chiama Counter(int v) -> OBBLIGATORIO perché non esiste un costruttore di default
8     }
9
10    public void dec() {
11        val--;
12    }
13 }
```

Esempio: ColoredCounter (aggiunge un campo colore)

```
1 import java.awt.Color;
2
3 public class ColoredCounter extends Counter {
4     private Color color; // nuovo campo
5
6     public ColoredCounter(int v) {
7         super(v); // Inizializza la parte ereditata (val)
8         this.color = Color.BLACK; // Inizializza il nuovo campo
9     }
10
11    public ColoredCounter(int v, Color color) {
12        super(v);
13        this.color = color;
14    }
15
16    public Color getColor() { return color; }
17
18    // Ridefinizione di toString
19    @Override
20    public String toString() {
21        // super.toString() chiama il toString di Counter
22        return super.toString() + " e colore " + color;
23    }
24
25    // Ridefinizione di equals
26    @Override
27    public boolean equals(Object obj) {
28        if (this == obj) return true;
29        if (!(obj instanceof ColoredCounter)) return false;
30        ColoredCounter that = (ColoredCounter) obj;
31        // super.equals(that) confronta la parte ereditata
32        return super.equals(that) && this.color.equals(that.color);
33    }
34 }
```

Principio: "Ognuno costruisce ciò che gli compete". Il costruttore di `Counter` inizializza `val`, il costruttore di `ColoredCounter` inizializza `color` e delega a `super` il resto.

Ridefinizione dei Metodi (Override) e Comportamento

Una classe derivata può **ridefinire un metodo ereditato per modificarne il comportamento**.

Regole per un override corretto (estensioni conservative):

- Il metodo ridefinito dovrebbe mantenere la semantica e il contratto del metodo originale.
- Può estendere il comportamento (es. fare qualcosa in più), ma non sovvertirlo.
es. Non è corretto ridefinire `inc()` in modo che divida il valore per 10 o che faccia un'altra operazione non correlata all'"incremento".

Esempio di estensione conservativa: `Finestra3`

- La classe base `Finestra` ha un metodo `print(String txt)`.
- La classe `Finestra3` estende `Finestra2` e ridefinisce `print` per stampare in maiuscolo, ma *chiama comunque il metodo della superclasse* per svolgere il compito principale.

```
1 public class Finestra3 extends Finestra2 {
2     public Finestra3() { super(); }
3     public Finestra3(String titolo) { super(titolo); }
4
5     @Override
6     public void print(String txt) {
7         super.print(txt.toUpperCase()); // Estensione conservativa
8     }
9 }
```

Classi e Metodi `final`

Per impedire che un metodo venga ridefinito o che una classe venga estesa, si usa la parola chiave `final`.

- `public final void mioMetodo() { ... }`: Questo metodo non può essere sovrascritto in alcuna sottoclasse.
- `public final class MiaClasse { ... }`: Questa classe non può essere estesa.

Modelliamo il concetto di "Studente" come specializzazione di "Persona".

```
1 import java.time.LocalDate;
2 import java.time.format.DateTimeFormatter;
3 import java.time.format.FormatStyle;
4
5 public class Persona {
6     private final String nome;
7     private final LocalDate dataNascita;
8
9     public Persona(String nome, LocalDate dataNascita) {
10         this.nome = nome;
11         this.dataNascita = dataNascita;
12     }
13
14     public String getNome() { return nome; }
15     public LocalDate getDataNascita() { return dataNascita; }
16
17     @Override
18     public String toString() {
19         DateTimeFormatter formatter = DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT);
20         return "Mi chiamo " + nome + " e sono nato/a il " + formatter.format(dataNascita);
21     }
22 }
23
24 public class Studente extends Persona {
25     private final int matricola;
26
27     public Studente(String nome, LocalDate dataNascita, int matricola) {
28         super(nome, dataNascita); // Inizializza la parte di Persona
29         this.matricola = matricola;
30     }
31
32     public int getMatricola() { return matricola; }
33
34     @Override
35     public String toString() {
36         // Estensione conservativa: usa super.toString() e aggiunge la matricola
37         return super.toString() + ", matricola " + matricola;
38     }
39 }
```

Punti chiave:

- `Studente` eredita da `Persona` i campi `nome` e `dataNascita` e il metodo `getNome()`.
- Il costruttore di `Studente` chiama `super(nome, dataNascita)` per inizializzare la parte ereditata.
- `toString()` è ridefinito per aggiungere la matricola, ma si basa su `super.toString()` per non duplicare la logica di stampa della persona.

Ecco degli appunti dettagliati in italiano basati esclusivamente sui concetti relativi a Java e ai principi generali di programmazione orientata agli oggetti presenti nel PDF, ignorando deliberatamente i riferimenti a Scala, Kotlin e C#.

Il Principio di Sostituibilità (e la Relazione IS-A)

- **Concetto Base:** L'ereditarietà definisce una relazione **IS-A** (È-UN).
- **Definizione:** Una classe derivata (es. `Counter2`) estende la classe base (`Counter`) **aggiungendo** funzionalità, **mai togliendone**.
- **Conseguenza:** Dove il codice si aspetta un oggetto della classe base, posso sempre fornire un oggetto della classe derivata.
 - *Analogia:* Non ci si può lamentare se, allo stesso prezzo, si riceve un oggetto "più ricco" di funzionalità.
- **Direzionalità:** La sostituibilità vale solo in una direzione.
 - Posso dare un `Counter2` a chi vuole un `Counter`.
 - **Non posso** dare un semplice `Counter` a chi si aspetta un `Counter2` (perché il `Counter` non ha i metodi aggiuntivi del `Counter2`).

Subclassing = Subtyping

Il tipo di una sottoclasse è un **sottotipo** del tipo della sua superclasse. Questa compatibilità di tipo si manifesta in tre modi principali:

1. **Assegnamenti:** Un riferimento di un tipo base può puntare a un oggetto di un tipo derivato.

```
1 Counter c = new Counter2(); // OK
```

2. **Passaggio di Parametri:** Come visto sopra, un metodo che accetta un tipo base può ricevere un'istanza di un tipo derivato.

3. **Valori di Ritorno:** Un metodo che dichiara di restituire un tipo base può restituire un'istanza di un tipo derivato.

```
1 public static Counter creaCounter(int v) {  
2     // Restituisce un Counter2, che è comunque un Counter  
3     return new Counter2(v);  
4 }
```

È invece **sbagliato fare il contrario**: assegnare un oggetto base a un riferimento derivato non è permesso senza un cast esplicito (che potrebbe fallire a runtime).

```
1 Counter c = new Counter();  
2 Counter2 c2 = c; // ERRORE DI COMPILAZIONE
```

Non tutte le relazioni IS-A che sembrano valide nel mondo reale lo sono nel mondo della programmazione a oggetti, specialmente se gli oggetti sono **mutabili**.

Es quadrato e rettangolo.

Implementazione (Problematica):

```
1 public class Rettangolo {  
2     private double base, altezza;  
3  
4     public Rettangolo(double base, double altezza) {  
5         setBase(base);  
6         setAltezza(altezza);  
7     }  
8  
9     public void setBase(double base) { this.base = base; }  
10    public void setAltezza(double altezza) { this.altezza = altezza; }  
11    // ... altri metodi  
12 }  
13  
14 public class Quadrato extends Rettangolo {  
15     public Quadrato(double lato) {  
16         super(lato, lato);  
17     }  
18  
19    // Tentativo di "riparare" la mutabilità:  
20    // Se cambio la base, cambio anche l'altezza per mantenere l'invariante del quadrato.  
21    @Override  
22    public void setBase(double base) {  
23        super.setBase(base);  
24        super.setAltezza(base); // Effetto collaterale imprevisto!  
25    }  
26  
27    @Override  
28    public void setAltezza(double altezza) {  
29        super.setBase(altezza); // Effetto collaterale imprevisto!  
30        super.setAltezza(altezza);  
31    }  
32 }
```

Un client che usa un `Quadrato` come se fosse un `Rettangolo` avrà un comportamento inaspettato:

```

1 public void modifica Rettangolo(Rettangolo r) {
2     r.setBase(5);
3     r.setAltezza(10);
4     // Il programmatore si aspetta che l'area ora sia 50.
5 }
6
7 Quadrato q = new Quadrato(4);
8 modificaRettangolo(q);
9 // Cosa è successo?
10 // 1. r.setBase(5) -> in Quadrato ha impostato base=5 E altezza=5
11 // 2. r.setAltezza(10) -> in Quadrato ha impostato base=10 E altezza=10
12 // Il quadrato ora ha lato 10 e area 100, non 50 come si aspetterebbe il client.
13 // Il client ha violato senza saperlo l'invariante del quadrato.

```

- Se gli oggetti sono mutabili, Quadrato **NON** è un sottotipo valido di Rettangolo perché aggiunge un vincolo comportamentale ("lati sempre uguali") che la superclasse non ha e che viene violato dai suoi stessi metodi pubblici.
- **Soluzione alternativa:** Rendere Quadrato e Rettangolo classi separate (fratelli) che derivano entrambe da una superclasse astratta comune, ad esempio FormaGeometrica. In questo modo si evita la relazione di sottotipizzazione forzata e incorretta.

Es Persona Studente

Al contrario di Quadrato/Rettangolo, la relazione Persona e Studente è un perfetto esempio di ereditarietà ben applicata.

- Uno Studente è sempre una Persona.
- Lo Studente non aggiunge vincoli che invalidano i comportamenti ereditati da Persona. Una Persona può cambiare nome, uno Studente anche, senza che si rompa alcun contratto.

Polimorfismo e Late Binding

La situazione più interessante si verifica quando si usa un riferimento di tipo base per puntare a un oggetto di tipo derivato:

```

1 Persona p = new Studente("Mario", 12345);

```

Ora p è formalmente un riferimento a Persona, ma **concretamente** punta a un oggetto Studente.

Quando invochiamo p.toString() viene chiamato il metodo della classe **effettiva** dell'oggetto a runtime (Studente.toString()). Questo meccanismo si chiama **Polimorfismo**.

Il Meccanismo: Late Binding (o Dynamic Binding)

La decisione su quale metodo chiamare non viene presa dal compilatore (binding statico/early), ma viene **rimandata a runtime** (binding dinamico/late).

- In Java, il late binding è l'**impostazione predefinita** per tutti i metodi di istanza non private e non static. Si può usare l'annotazione @Override per rendere esplicita l'intenzione di ridefinire un metodo della superclasse.

Esempio di Esecuzione:

```

1 Persona p = new Persona("John");
2 Studente s = new Studente("Tom", 98765);
3
4 System.out.println(p.toString()); // Output: Mi chiamo John
5 System.out.println(s.toString()); // Output: Mi chiamo Tom, matricola 98765
6
7 p = s; // p ora punta all'oggetto Studente di nome Tom
8 System.out.println(p.toString()); // Output: Mi chiamo Tom, matricola 98765
9 // Anche se p è di tipo Persona, viene eseguito il metodo di Studente!

```

Late Binding: Funzionamento Interno (VMT)

Come fa la JVM a sapere a runtime quale metodo chiamare? Usa la **Virtual Method Table (VMT)**.

Ogni classe ha una sua VMT, che è una tabella che associa ogni metodo (della classe e ereditato) all'indirizzo di memoria del codice da eseguire.

Quando una classe (Studente) estende un'altra classe (Persona), la VMT di Studente è costruita a partire da quella di Persona.

- I metodi ereditati ma **non** ridefiniti mantengono lo stesso indice e puntano al codice della superclasse.

- I metodi **ridefiniti** (con @Override) mantengono lo **stesso indice** nella tabella, ma l'indirizzo associato viene cambiato per puntare al nuovo codice della sottoclasse.

- I nuovi metodi della sottoclasse vengono aggiunti in fondo alla tabella.

A runtime, quando si esegue p.toString(), la JVM:

- Guarda l'oggetto puntato da p (che è di tipo Studente).

- Accede alla VMT della classe Studente.

- Prende l'indirizzo del codice associato al metodo toString() (che è sempre lo stesso indice della tabella).

- Esegue quel codice.

Il polimorfismo ha un limite fondamentale: funziona solo per i metodi che **esistono nella classe base**.

- **Metodi Ridefiniti:** Disponibili e polimorfici (es. toString()).

- **Metodi Aggiunti:** **NON** sono accessibili tramite un riferimento della classe base, perché il compilatore non può garantire che l'oggetto a runtime li possieda.

```
1  class Counter2 extends Counter {
2      public void dec() { /* decrementa */ }
3  }
4
5  Counter c = new Counter2();
6
7  c.dec(); // ERRORE DI COMPILAZIONE!
8  // 'c' è di tipo Counter e la classe Counter non ha un metodo 'dec()'.
9
10 // Per chiamarlo, è necessario un cast esplicito:
11 ((Counter2) c).dec(); // OK, ma rischioso se c non fosse veramente un Counter2
```

La Classe Object

In Java, tutte le classi formano una gerarchia la cui radice è la classe `java.lang.Object`.

- Se si scrive `class MiaClasse { ... }`, il compilatore la interpreta come `class MiaClasse extends Object { ... }`.
- Questo implica che **ogni** oggetto in Java eredita un insieme di metodi fondamentali da `Object`.

Metodi "Predefiniti" Principali di `Object` (e quindi disponibili per tutti gli oggetti):

Metodo	Descrizione	Comportamento Predefinito in <code>Object</code>
<code>String toString()</code>	Restituisce una rappresentazione in stringa dell'oggetto.	Restituisce una stringa come <code>NomeClasse@hashCode</code> (es. <code>Counter@712c1a3c</code>).
<code>boolean equals(Object obj)</code>	Confronta l'oggetto corrente con un altro per verificarne l'uguaglianza "logica".	Confronta i riferimenti in memoria, come l'operatore <code>==</code> .
<code>int hashCode()</code>	Restituisce un valore numerico (hash) che rappresenta l'oggetto. Deve essere coerente con <code>equals</code> (oggetti uguali devono avere lo stesso hash).	Tipicamente converte l'indirizzo di memoria in un intero.
<code>protected Object clone()</code>	Crea e restituisce una copia dell'oggetto.	Crea una copia "superficiale" (shallow copy).

Esempio di Utilizzo di `toString()` e `println()`:

Il metodo `System.out.println()` è progettato per sfruttare il polimorfismo di `toString()`. La sua implementazione (semplificata) è:

```
1  public void println(Object obj) {
2      // Chiama toString() sull'oggetto, che sarà la versione appropriata
3      // in base al tipo effettivo dell'oggetto a runtime.
4      String s = obj.toString();
5      // ... stampa la stringa s
6  }
```

Ecco perché fare `System.out.println(unapersona)` o `System.out.println(unostudente)` funziona sempre e produce un output sensato, se il metodo `toString()` è stato ridefinito correttamente.

Es Numeri Reali e Numeri Complessi

Analisi Errata (Relazione Invertita):

- Pensiero: "Un numero complesso ha una parte reale, come i reali, ma anche una parte immaginaria. Quindi `Complex extends Real`."
- **Problema:** Un numero reale è un sottoinsieme (caso particolare) dei numeri complessi ($R \subset C$). Matematicamente, l'insieme dei reali è più "piccolo". La relazione di ereditarietà "estende" indica un sottoinsieme, non un ampliamento.

Analisi Corretta:

- **Relazione:** `R  $\subset$  C -> Real` è un sottotipo di `Complex`.
- **Progetto:** La classe base è `Complex`. La classe `Real` estende `Complex`.

```
1  // Classe base: Numero Complesso
2  public class Complex {
3      protected float re, im;
4
5      public Complex(float r, float i) {
6          this.re = r;
7          this.im = i;
8      }
9
10     public Complex sum(Complex that) {
```

```

11     return new Complex(this.re + that.re, this.im + that.im);
12 }
13
14 public Complex sub(Complex that) {
15     return new Complex(this.re - that.re, this.im - that.im);
16 }
17
18 // Moltiplicazione: (a+ib)(c+id) = (ac-bd) + i(bc+ad)
19 public Complex mul(Complex that) {
20     return new Complex(
21         this.re * that.re - this.im * that.im,
22         this.im * that.re + this.re * that.im
23     );
24 }
25
26 // ... altri metodi come div, coniugato, modulo quadrato ...
27
28 @Override
29 public String toString() {
30     return re + " + i" + im;
31 }
32 }
33
34 // Classe derivata: Numero Reale (caso particolare di Complesso con parte immaginaria = 0)
35 public class Real extends Complex {
36
37     public Real(float r) {
38         super(r, 0.0f); // La parte immaginaria è sempre 0
39     }
40
41     // Metodi specializzati per operazioni fra reali (restituiscono un Real)
42     // per efficienza e per mantenere il tipo più specifico
43     public Real sum(Real that) {
44         // Si potrebbe usare this.re + that.re, ma re è protected in Complex
45         return new Real(this.re + that.re);
46     }
47
48     // Nota: i metodi ereditati come sum(Complex) funzionano ancora,
49     // ma restituiscono un Complex. Questo è corretto (polimorfismo).
50
51     @Override
52     public String toString() {
53         // Nascondo la parte immaginaria per una rappresentazione più pulita
54         return Float.toString(re);
55     }
56 }

```

1. **Correttezza Matematica:** Rispetta la relazione insiemistica $\mathbb{R} \subset \mathbb{C}$.
2. **Sostituibilità Corretta:** Posso passare un `Real` ovunque sia richiesto un `Complex` (es. sommare un reale a un complesso), il che è matematicamente corretto. Il viceversa non è permesso (non posso passare un `Complex` qualsiasi dove è richiesto un `Real`), che è ancora una volta corretto.
3. **Polimorfismo Naturale:** Un `Real` è trattato come un `Complex` con `im=0`, e i metodi di `Complex` funzionano perfettamente. Possono poi essere specializzati in `Real` per ottimizzazione.

Equals e hash code

equals

Nelle nostre prime implementazioni di classi come `Counter` e `Frazione`, avevamo definito un metodo `equals` con un argomento del tipo specifico della classe:

```

1 // Classe Counter
2 public boolean equals(Counter that) {
3     return this.val == that.val;
4 }
5
6 // Classe Frazione
7 public boolean equals(Frazione that) {
8     return this.num * that.den == this.den * that.num;
9 }

```

La classe `Object` (superclasse di tutte le classi in Java) dichiara:

```

1 public boolean equals(Object obj) {
2     return this == obj;

```

```
3 }
```

- `equals(Counter)` in `Counter`
- `equals(Frazione)` in `Frazione`
- `equals(Object)` ereditata da `Object`

Queste sono **tre funzioni diverse** (overloading, non overriding).

Di conseguenza, ogni classe contiene **due metodi** `equals` :

- quella specifica (con argomento del proprio tipo)
- quella ereditata (con argomento `Object`)

Caso 1: riferimenti di tipo `Counter`

```
1 Counter c1 = new Counter(13);
2 Counter c2 = new Counter(13);
3 System.out.println(c1.equals(c2)); // true (usa equals(Counter))
```

Caso 2: cambia il tipo nominale del riferimento

```
1 Object obj = c1;
2 System.out.println(obj.equals(c2)); // false (usa equals(Object) di Object)
3 System.out.println(c2.equals(obj)); // false (overloading: sceglie equals(Object))
```

- `obj.equals(c2)` – il target è `Object`, quindi si cerca in `Object` (l'unica `equals` disponibile è quella con `Object`). Il polimorfismo non trova una versione più specifica in `Counter` perché la signature `equals(Counter)` non è una ridefinizione di `equals(Object)`.
- `c2.equals(obj)` – il target è `Counter`, che ha due `equals`. La risoluzione dell'overloading guarda il tipo nominale dell'argomento (`Object`) e sceglie `equals(Object)` (quella ereditata), non la nostra.

avere due `equals` è pericoloso – quella “sbagliata” può emergere in contesti polimorfi.

overriding della `equals` di `Object`

Per un comportamento polimorfo corretto, dobbiamo **sostituire** (override) il metodo `equals` ereditato, **non aggiungerne uno nuovo**. La signature deve essere identica:

```
1 @Override
2 public boolean equals(Object obj) { ... }
```

`Object` generico, quindi non possiamo accedere direttamente a campi come `val` (di `Counter`) o `num`, `den` (di `Frazione`).

Serve un **cast** (downcast) esplicito.

Prima versione (fragile)

```
1 @Override
2 public boolean equals(Object obj) {
3     return this.val == ((Counter) obj).val; // può lanciare ClassCastException
4 }
```

Aggiungere un controllo di tipo con `instanceof`

Per evitare eccezioni, prima di eseguire il cast controlliamo se l'oggetto è del tipo atteso:

```
1 @Override
2 public boolean equals(Object obj) {
3     if (obj instanceof Counter) {
4         Counter that = (Counter) obj;
5         return this.val == that.val;
6     }
7     return false;
8 }
```

Ora il metodo è sicuro: se `obj` non è un `Counter`, restituisce `false`.

`instanceof` esteso (Java 16+)

A partire da Java 16, possiamo usare una forma più concisa che dichiara e casta automaticamente la variabile:

```
1 @Override
2 public boolean equals(Object obj) {
3     if (obj instanceof Counter that) {
4         return this.val == that.val;
5     }
6     return false;
7 }
```


All'interno del ramo `true`, `that` è già di tipo `Counter`.

Alternativa moderna: `switch` con pattern matching

```
1  @Override
2  public boolean equals(Object obj) {
3      return switch (obj) {
4          case Counter c -> this.val == c.val;
5          default -> false;
6      };
7  }
```

`hashCode`

- `Object` dichiara anche `public int hashCode()`.
 - **Regola aurea: se due oggetti sono uguali secondo `equals`, devono avere lo stesso `hashCode`**
 - `hashCode` è usato internamente da strutture dati come `HashMap`, `HashSet` per ricerche efficienti.
- Se non ridefiniamo `hashCode` in `Counter`:

```
1  Counter c1 = new Counter(13);
2  Counter c2 = new Counter(13);
3  System.out.println(c1.hashCode()); // 1523554304 (valore casuale)
4  System.out.println(c2.hashCode()); // 1175962212 (diverso!)
```

Nonostante `c1.equals(c2)` sia `true`, gli hash code sono diversi → violazione della regola.

Important

Eclipse può generare `equals` e `hashCode` automaticamente che funzionano SEMPRE

da rigenerare se modifichi i campi

Opzione consigliata in Eclipse: usa `Objects.hash` per un codice più compatto.

es.

```
1  public class Counter {
2      private int val;
3
4      public Counter(int val) {
5          this.val = val;
6      }
7
8      @Override
9      public boolean equals(Object obj) {
10         return (obj instanceof Counter that) && this.val == that.val;
11     }
12
13     @Override
14     public int hashCode() {
15         return Objects.hash(val);
16     }
17 }
```

`instanceof`, `switch` esteso e `overloading`

Talvolta è necessario **distinguere le istanze di una classe da quelle delle sue sottoclassi** per trattarle in modo diverso.

Se la funzione è un metodo (non statico), il polimorfismo risolve il problema automaticamente.

Se invece la funzione è **statica** (o comunque non legata al polimorfismo dinamico), occorre un approccio diverso.

Due strade possibili:

1. **Controllo dinamico del tipo** – usando `instanceof` o `switch` con pattern matching.
2. **Overloading** – definendo più versioni della stessa funzione con parametri di tipo diverso.

es. funzione `decrementa`

Scrivere una **funzione statica** `decrementa(Counter c)` che:

- Se `c` è effettivamente un `Counter2`, deve chiamare `c.dec()`.
- Altrimenti (è un `Counter` semplice), deve simulare il decremento usando `reset()`, `getValue()` e un ciclo di `inc()`.

Attenzione: l'argomento è passato **per valore** (riferimento), quindi non possiamo sostituire l'oggetto, dobbiamo modificare quello esistente.

uso di `instanceof`

```

1 public static void decrementa(Counter c) {
2     if (c instanceof Counter2) {
3         Counter2 c2 = (Counter2) c;
4         //if (c instanceof Counter2 c2) { // dichiara e casta automaticamente
5             c2.dec();
6         } else {
7             int valore = c.getValue();
8             c.reset();
9             while (--valore > 0) {
10                 c.inc();
11             }
12         }
13     }

```

uso di switch esteso con pattern matching

Il costrutto switch (dalla forma estesa) permette di distinguere per tipo.

```

1 public static void decrementa(Counter c) {
2     switch (c) {
3         case Counter2 c2 -> c2.dec(); // se è un Counter2
4         case Counter _ -> { // se è un Counter semplice
5             int valore = c.getValue();
6             c.reset();
7             while (--valore > 0) {
8                 c.inc();
9             }
10         }
11     }
12 }

```

- Il secondo caso usa Counter _ (variabile anonima) perché non ci serve un riferimento castato.
- La sintassi con -> evita il break esplicito.

overloading

Invece di distinguere dentro la funzione, possiamo definire **due funzioni diverse** con lo stesso nome e parametri di tipo differente.

```

1 // Versione per Counter2 (decremento diretto)
2 public static void decrementa(Counter2 c2) {
3     c2.dec();
4 }
5
6 // Versione per Counter generico (decremento simulato)
7 public static void decrementa(Counter c) {
8     int valore = c.getValue();
9     c.reset();
10    while (--valore > 0) {
11        c.inc();
12    }
13 }

```

Il compilatore decide **staticamente** (a compile time) quale versione chiamare in base al **tipo dichiarato** dell'argomento, non al tipo effettivo a runtime.

```

1 Counter c1 = new Counter(10);
2 Counter2 c2 = new Counter2(20);
3
4 decrementa(c1); // chiama decrementa(Counter)
5 decrementa(c2); // chiama decrementa(Counter2) perché c2 è dichiarato Counter2

```

Se avessimo:

```

1 Counter c = new Counter2(20);
2 decrementa(c); // chiama decrementa(Counter) !!! (perché il tipo dichiarato è Counter)

```

In questo caso la versione sbagliata verrebbe eseguita (quella con il ciclo, invece del dec()).

L'overloading non è polimorfo – è una scelta statica.

Perché l'overloading non funziona per equals

- L'**overloading** è una scelta **statica** (compile time) basata sul tipo dichiarato.
- Il metodo equals deve essere **polimorfo** (dynamic dispatch), quindi serve **override** (stessa identica firma) e un controllo dinamico del tipo con instanceof o pattern matching.

Wrapper per tipi primitivi in Java

In Java, i **tipi primitivi** (`int`, `double`, `char`, `boolean`, ecc.) **non sono classi**.

Questo crea una disuniformità rispetto ai tipi riferimento:

Caratteristica	Tipi primitivi	Tipi riferimento (classi)
Hanno metodi	No	Sì (es. <code>toString()</code>)
Uguaglianza	<code>==</code> (valore)	<code>equals()</code> (sovrascrivibile)
Passaggio parametri	per valore (copia)	per riferimento (copia del riferimento)
Array separati	<code>int[]</code> , <code>double[]</code>	<code>Object[]</code> , <code>Counter[]</code>
Supportano ereditarietà	No	Sì
Usabili in <code>Collection</code>	No (es. <code>List<int></code> non valido)	Sì (<code>List<Integer></code>)

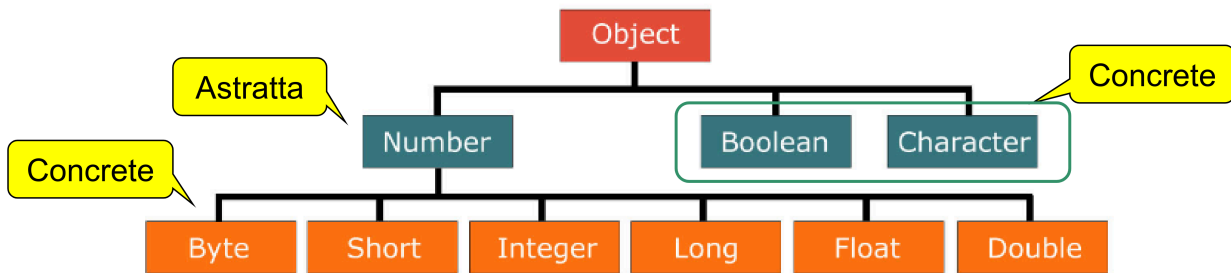
Conseguenze

Un valore primitivo **non può essere usato** dove è richiesto un `Object`:

```
1 Object[] array = new Object[10];
2 array[0] = "ciao";           // OK - String è Object
3 array[1] = new Counter(2);    // OK - Counter è Object
4 array[2] = 7;                 // errore di compilazione (prima del boxing automatico)
```

Success

Java fornisce per ogni tipo primitivo una **classe wrapper** che **incapsula un valore primitivo all'interno di un oggetto**.



Le classi wrapper (e altre come `Optional`, classi `java.time`) sono **value-based**:

- Sono **final** e **immutabili**.
- L'uguaglianza va testata con `equals()`, **non con** `==` (perché istanze diverse possono essere considerate intercambiabili).
- Non espongono costruttori pubblici; si usano metodi `factory` (`valueOf`).
- `equals()`, `hashCode()`, `toString()` dipendono solo dallo stato, non dall'identità.

Boxing e unboxing

```
1 Integer i = Integer.valueOf(22);
2 Double d = Double.valueOf(3.14);
```

`valueOf()` può ottimizzare con **caching** (es. per interi tra -128 e 127 riutilizza le stesse istanze).

```
1  int x = i.intValue();           // 22
2  double z = d.doubleValue();     // 3.14
```

Esempi di conversione tra tipi

```
1 Integer i = Integer.valueOf(22);
2 System.out.println(i.intValue()); // 22
3 System.out.println(i.doubleValue()); // 22.0
4
5 Double d = Double.valueOf(3.14);
6 System.out.println(d.intValue()); // 3 (troncato)
7
8 Float f = Float.valueOf(1.73F);
9 System.out.println(f.longValue()); // 1
```

Boxing e unboxing automatici

Java converte automaticamente tra primitivi e wrapper quando necessario:

```

1 // Boxing automatico
2 Integer k = 42; // equivale a Integer.valueOf(42)
3
4 // Unboxing automatico
5 int val = k; // equivale a k.intValue()
6
7 // Operazioni miste
8 Integer a = 10;
9 Integer b = 20;
10 Integer c = a + b; // unboxing, somma, boxing

```

Uso con array di Object

```

1 Object[] array = new Object[10];
2 array[0] = "ciao";
3 array[1] = new Counter(2);
4 array[2] = 7; // boxing automatico: int → Integer
5 array[3] = 7.2; // boxing automatico: double → Double
6 // Funziona!

```

Uso con collection

```

1 List<Integer> list = new ArrayList<>();
2 list.add(5); // boxing automatico: int → Integer
3 int primo = list.get(0); // unboxing automatico: Integer → int

```

Nota: non si può usare `List<int>` – i tipi primitivi non sono ammessi come parametri di tipo.

Parsing

Da String a numero primitivo – `parseXXX()`

```

1 String s = "23";
2 int i = Integer.parseInt(s); // int
3 long l = Long.parseLong(s); // long
4 double d = Double.parseDouble(s); // double

```

Da String a wrapper – `valueOf()`

```

1 Integer k = Integer.valueOf(s); // Integer
2 Long L = Long.valueOf(s); // Long

```

Se la stringa non è un numero valido, si ha `NumberFormatException` a runtime:

```

1 Integer.parseInt("23a"); // NumberFormatException

```

Da numero a String

```

1 int i = 12;
2 String i = String.valueOf(i); // "12"
3
4 Integer k = 24;
5 String k = k.toString(); // "24"

```

Conversioni in basi diverse (radix)

```

1 // String → int in base binaria (radix 2)
2 int bin = Integer.parseInt("1010", 2); // 10
3
4 // int → String in base esadecimale
5 String hex = Integer.toString(255, 16); // "ff"
6
7 // Metodi helper predefiniti
8 String binStr = Integer.toBinaryString(10); // "1010"
9 String octStr = Integer.toOctalString(10); // "12"
10 String hexStr = Integer.toHexString(255); // "ff"

```

`instanceof` e `switch` con pattern matching lavorano su **oggetti**, non su primitivi.

Per testare un valore primitivo avvolto in un wrapper:

```

1 Object v = 17; // boxing automatico
2
3 if (v instanceof Integer && (Integer)v < 18) {
4     System.out.println("Voto insufficiente: " + v);
5 }

```

```

5  }
6
7  switch (v) {
8      case Integer i when i < 18 -> System.out.println("Bocciato: " + i);
9      case Integer i -> System.out.println("Promosso: " + i);
10 }

```

Da Java 25 (in preview) è possibile usare pattern direttamente sui tipi primitivi:

```

1  // Con instanceof
2  if (v instanceof int && v < 18) { // v deve essere int
3      System.out.println("Voto insufficiente: " + v);
4  }
5
6  // Con switch esteso
7  switch (v) {
8      case int i when i < 18 -> System.out.println("Bocciato: " + i);
9      case int i -> System.out.println("Promosso: " + i);
10     default -> System.out.println("Input non valido");
11 }

```

Per abilitare le preview feature:

```

1  javac --enable-preview --source 25 NomeFile.java
2  java --enable-preview NomeFile

```

Record e classi di dati

Nel modellare un sistema, capita spesso di avere classi che sono semplici **aggregati di dati immutabili**:

- campi dati (tipicamente `final`)
- costruttore (che inizializza i campi)
- accessor (getter)
- metodi `equals`, `hashCode`, `toString` standard

La struttura di queste classi è meccanica e ripetitiva.

Gli IDE (Eclipse, IntelliJ) possono generarli automaticamente, ma nei linguaggi moderni (**Java da versione 15+**) **esistono i record****.

Un **record** è una forma leggera di classe, pensata per essere un semplice contenitore di dati immutabili.

```

1  public record Persona(String nome, int anni) {}

```

Con questa sola riga, il compilatore genera automaticamente:

- campi `private final` per nome e anni;
- un costruttore canonico (`public Persona(String nome, int anni)`);
- metodi accessor **con lo stesso nome del campo** (`nome()` e `anni()`);
- `equals` (confronto campo per campo);
- `hashCode` (basato sui campi);
- `toString` (restituisce una stringa con tutti i campi).

```

1  public static void main(String[] args) {
2      var p1 = new Persona("John", 25);
3      var p2 = new Persona("Mary", 22);
4      var p3 = new Persona("John", 25);
5
6      System.out.println(p1);           // Persona[nome=John, anni=25]
7      System.out.println(p1.equals(p3)); // true
8      System.out.println(p1 == p3);    // false (identità oggetti)
9
10     System.out.println(p1.nome());    // John
11     System.out.println(p1.anni());    // 25
12 }

```

Nota bene: l'operatore `==` confronta l'identità degli oggetti, non il contenuto. Per il contenuto si usa `equals` (che nei record è già implementato correttamente).

Proprietà	Descrizione
Immutabilità	Tutti i campi dichiarati nell'intestazione sono <code>final</code>
Non Estensibilità	Il record è <code>final</code> : non può essere esteso da altre classi, né estendere altre classi (deriva implicitamente da <code>java.lang.Record</code>)

Proprietà	Descrizione
Metodi aggiuntivi	È possibile aggiungere metodi di istanza, metodi statici, campi statici
Interfacce	Un record può implementare una o più interfacce
Costruttori ausiliari	Si possono definire costruttori aggiuntivi, ma devono richiamare un altro costruttore con <code>this(...)</code>

I record possono essere usati nel **pattern matching** all'interno di `instanceof` e `switch`.
Ciò permette la **destrutturazione** (estrazione dei componenti) in modo conciso.

Destrutturazione con `instanceof`

```
1 static void print(Object obj) {
2     if (obj instanceof Persona(String n, int a)) {
3         System.out.println("Persona: nome=" + n + ", anni=" + a);
4     } else {
5         System.out.println(obj);
6     }
7 }
```

Con type inference (var):

```
1 if (obj instanceof Persona(var n, var a)) { ... }
```

Unnamed patterns

Quando un componente del record non serve, si può usare il carattere `_` per ignorarlo.

```
1 if (obj instanceof Persona(_, int a)) {
2     System.out.println("Età: " + a); // il nome viene ignorato
3 }
```

Pattern innestati

Se un record contiene un altro record, si possono destrutturare entrambi in un unico pattern.

```
1 record Point(int x, int y) {}
2 record Rectangle(Point a, Point b) {}
3
4 static void print(Rectangle r) {
5     if (r instanceof Rectangle(Point(var x1, var y1), Point(var x2, var y2))) {
6         System.out.println("Vertici: (" + x1 + ", " + y1 + "), (" + x2 + ", " + y2 + ")");
7     }
8 }
```

Guardie semantiche (clausola `when`)

nei pattern di `switch` si può aggiungere una condizione booleana con `when`.

```
1 public String show() {
2     return switch(this.anni) {
3         case int i when i < 18 -> nome + " è minorenne";
4         case int i when i > 70 -> nome + " è pensionato";
5         default -> nome + " è un lavoratore";
6     };
7 }
```

Aspetto	Classe tradizionale	Record
Dichiarazione	Molto verbosa (campi, costruttore, getter, equals, hashCode, toString)	Una sola riga
Mutabilità	I campi possono essere <code>final</code> o meno	Tutti i campi dell'istestazione sono <code>final</code> (immutabili)
Ereditarietà	Può estendere altre classi	Non può essere esteso (è <code>final</code>)
Metodi accessor	Di solito <code>getNome()</code>	<code>nome()</code> (stesso nome del campo)
Pattern matching	Non supportato (o richiede <code>instanceof</code> con cast)	Supporto nativo alla destrutturazione

Quando usare un record

- Quando la classe è un semplice **contenitore di dati** (DTO, value object).
- Quando l'**immutabilità** è desiderata.

- Quando non serve ereditarietà.
 - Quando si vuole sfruttare il **pattern matching** per destrutturare gli oggetti.
- In tutti gli altri casi (gerarchie, oggetti mutevoli, comportamenti complessi) si usano le normali classi.

Classi astratte

Nel modellare il mondo reale, molte entità sono **categorie concettuali** e non esistono come oggetti concreti.

Esempi:

- Non esiste il “generico animale” – esistono cani, gatti, lepri, ...
- Non esiste la “generica roccia” – esistono granito, quarzo, basalto, ...
- Non esiste la “generica forma geometrica” – esistono triangoli, quadrati, cerchi, ...

Le **classi astratte** servono proprio a rappresentare queste categorie concettuali, di cui **non possono esistere istanze**. Le istanze effettive appartengono a sottoclassi concrete.

In Java si usa la parola chiave `abstract`:

- per dichiarare una classe astratta
- per dichiarare uno o più **metodi astratti** (senza implementazione)

```

1  public abstract class Animale {
2      // campo privato (esiste in tutte le sottoclassi)
3      private String mioNome;
4
5      // campo protected (accessibile alle sottoclassi)
6      protected String verso;
7
8      // costruttore
9      public Animale(String nome) {
10         this.mioNome = nome;
11     }
12
13     // metodo astratto (senza corpo, solo dichiarazione)
14     public abstract String siMuove();
15
16     public abstract String vive(); // verrà definito da una sottoclasse
17
18     public abstract String nome();
19
20     // metodo concreto (implementato qui)
21     public void mostra() {
22         System.out.println(mioNome + ", " + nome() + ", " + verso +
23                             ", si muove " + siMuove() + " e vive " + vive());
24     }
25 }

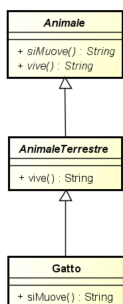
```

Regole fondamentali:

- Una classe che contiene almeno un metodo astratto **deve** essere dichiarata `abstract`.
- Una classe astratta **non può essere istanziata** (non si può fare `new Animale(...)`).
- Una classe astratta può contenere anche metodi concreti (con implementazione) e campi.

Sottoclassi astratte e concrete

Una sottoclasse può implementare solo **una parte** dei metodi astratti ereditati. Se ne rimane almeno uno non implementato, anche la sottoclasse deve essere dichiarata `abstract`.



sottoclasse astratta

```

1  public abstract class AnimaleTerrestre extends Animale {
2      public AnimaleTerrestre(String s) {

```

```

3         super(s);
4     }
5
6     // implementa vive() e nome(), ma non siMuove()
7     @Override
8     public String vive() {
9         return "sulla terraferma";
10    }
11
12    @Override
13    public String nome() {
14        return "un animale terrestre";
15    }
16
17    // siMuove() rimane astratto – quindi la classe è astratta
18 }

```

Sottoclasse concreta

Una classe è **concreta** quando **tutti** i metodi astratti ereditati sono stati implementati (direttamente o ereditati da una superclasse).

```

1 public class Gatto extends AnimaleTerrestre {
2     public Gatto(String nome) {
3         super(nome);
4         this.verso = "miagola";
5     }
6
7     @Override
8     public String siMuove() {
9         return "saltando";
10    }
11
12    @Override
13    public String nome() {
14        return "un gatto";
15    }
16 }

```

Esempio animali

```

1 public abstract class Animale {
2     private String mioNome;
3     protected String verso;
4
5     public Animale(String nome) {
6         this.mioNome = nome;
7     }
8
9     public abstract String siMuove();
10    public abstract String vive();
11    public abstract String nome();
12
13    public void mostra() {
14        System.out.println(mioNome + ", " + nome() + ", " + verso +
15                            ", si muove " + siMuove() + " e vive " + vive());
16    }
17 }

```

AnimaleTerrestre

```

1 public abstract class AnimaleTerrestre extends Animale {
2     public AnimaleTerrestre(String s) {
3         super(s);
4     }
5
6     @Override
7     public String vive() {
8         return "sulla terraferma";
9     }
10
11    @Override
12    public String nome() {
13        return "un animale terrestre";
14    }
15 }

```


AnimaleAcquatico

```
1 public abstract class AnimaleAcquatico extends Animale {
2     public AnimaleAcquatico(String s) {
3         super(s);
4     }
5
6     @Override
7     public String vive() {
8         return "nell'acqua";
9     }
10
11    @Override
12    public String nome() {
13        return "un animale acquatico";
14    }
15 }
```

AnimaleMarino (specializzazione di AnimaleAcquatico)

```
1 public abstract class AnimaleMarino extends AnimaleAcquatico {
2     public AnimaleMarino(String s) {
3         super(s);
4     }
5
6     @Override
7     public String vive() {
8         return "in mare";
9     }
10
11    @Override
12    public String nome() {
13        return "un animale marino";
14    }
15 }
```

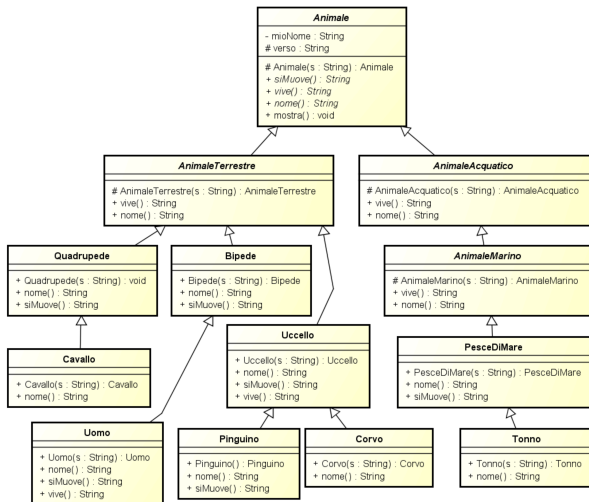
Classi concrete

```
1 public class PesceDiMare extends AnimaleMarino {
2     public PesceDiMare(String nome) {
3         super(nome);
4         this.verso = "non fa versi";
5     }
6
7     @Override
8     public String nome() {
9         return "un pesce (di mare)";
10    }
11
12    @Override
13    public String siMuove() {
14        return "nuotando";
15    }
16 }
17
18 public class Uccello extends AnimaleTerrestre {
19     public Uccello(String nome) {
20         super(nome);
21         this.verso = "cinguetta";
22     }
23
24    @Override
25    public String vive() {
26        return "in un nido su un albero";
27    }
28
29    @Override
30    public String nome() {
31        return "un uccello";
32    }
33
34    @Override
35    public String siMuove() {
36        return "volando";
37    }
38 }
```

```

38 }
39
40 public class Tonno extends PesceDiMare {
41     public Tonno(String nome) {
42         super(nome);
43     }
44
45     @Override
46     public String nome() {
47         return "un tonno";
48     }
49 }

```



Esempio forme geometriche

```

1 public abstract class Forma {
2     public abstract double area();
3     public abstract double perimetro();
4     public abstract String nome();
5
6     @Override
7     public String toString() {
8         return nome() + " di area " + area() + " e perimetro " + perimetro();
9     }
10 }

```

Triangolo

```

1 public class Triangolo extends Forma {
2     private double lato1, lato2, lato3;
3
4     public Triangolo(double l1, double l2, double l3) {
5         this.lato1 = l1;
6         this.lato2 = l2;
7         this.lato3 = l3;
8     }
9
10    @Override
11    public double perimetro() {
12        return lato1 + lato2 + lato3;
13    }
14
15    @Override
16    public double area() {
17        // Formula di Erone
18        double p = perimetro() / 2;
19        return Math.sqrt(p * (p - lato1) * (p - lato2) * (p - lato3));
20    }
21
22    @Override
23    public String nome() {
24        return "Triangolo qualsiasi";
25    }
26 }

```

TriangoloIsoscele (estende Triangolo)

```

1 public class TriangoloIsoscele extends Triangolo {
2     public TriangoloIsoscele(double base, double lato) {
3         super(base, lato, lato);
4     }
5
6     @Override
7     public String nome() {
8         return "Triangolo isoscele";
9     }
10 }

```

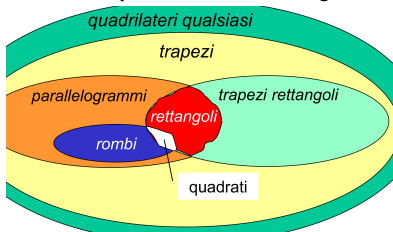
TriangoloEquilatero

```

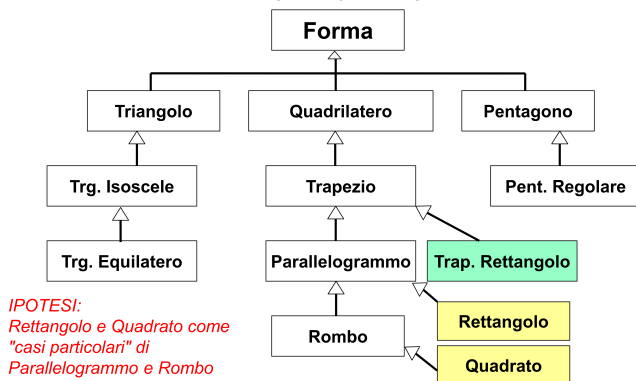
1 public class TriangoloEquilatero extends Triangolo {
2     public TriangoloEquilatero(double lato) {
3         super(lato, lato, lato);
4     }
5
6     @Override
7     public String nome() {
8         return "Triangolo equilatero";
9     }
10 }

```

Il caso del quadrato e del rettangolo



Nella realtà un **quadrato** è un caso particolare di **rettangolo** (ha tutti gli angoli retti e i lati uguali). Tuttavia, con l'ereditarietà singola si può scegliere solo un criterio di classificazione alla volta.



Se definiamo:

```

1 public class Rettangolo extends Forma { ... }
2 public class Quadrato extends Rettangolo { ... }

```

funziona, ma se volessimo anche classificare il quadrato come "rombo" o "trapezio rettangolo" non possiamo, perché in Java una classe può estendere una sola altra classe.

```

1 Rettangolo r = new Quadrato(5); // vorremmo che fosse valido, ma se Quadrato non estende Rettangolo dà errore

```

In un modello mal progettato (ad esempio se `Quadrato` e `Rettangolo` sono fratelli sotto `Forma`), l'assegnazione non è permessa, violando l'intuizione geometrica.

Intersezioni di insiemi

Missing

L'ereditarietà singola modella bene le relazioni di **inclusione insiemistica** (un sottoinsieme è un caso particolare). Ma la realtà spesso presenta **intersezioni**:

- Un triangolo rettangolo è sia "triangolo" sia "rettangolo" (angolo retto).

- Un quadrato è sia “rettangolo” sia “rombo”.

Queste situazioni richiederebbero **ereditarietà multipla**, che Java non supporta tra classi per evitare problemi complessi (diamond problem, duplicazione di dati, conflitti di metodi).

Ereditarietà Multipla e interfacce

Per risolvere il problema delle intersezioni senza i problemi dell'ereditarietà multipla tra classi, Java introduce il concetto di **interfaccia**.

Un'interfaccia permette di dichiarare comportamenti (metodi astratti) che una classe può implementare **molteplice** (implementa più interfacce). Le interfacce non contengono stato (campi di istanza) – solo costanti statiche e metodi astratti (o default/statici da Java 8).

Interfaccia

Definizione di Classe: Una classe definisce un Tipo di Dato Astratto (ADT) specificando contemporaneamente due aspetti:

1. **Front-End (Pubblico):** Cosa fa l'oggetto (firme dei metodi pubblici).
2. **Back-End (Privato):** Come lo fa (implementazione dei metodi e dati interni).

Questa contemporaneità è un limite progettuale per tre motivi:

- **Fasi Preliminari:** Non sempre si conoscono i dettagli implementativi all'inizio del progetto.
- **Flessibilità:** Non si vogliono vincolare le scelte implementative a priori (es. per cambiare idea dopo o scegliere a runtime).
- **Ereditarietà Multipla Assente:** Java non supporta l'ereditarietà multipla tra classi, limitando la modellazione di concetti come `StudenteLavoratore`.

Classi Astratte: Una Soluzione Parziale

- Permettono di dichiarare metodi senza implementarli (`abstract`).
- **Limite:** Vincolano l'implementazione dei metodi astratti a una **sottoclasse**, legando il tutto a una gerarchia singola e rigida. Non risolvono il problema dell'ereditarietà multipla né del disaccoppiamento totale.

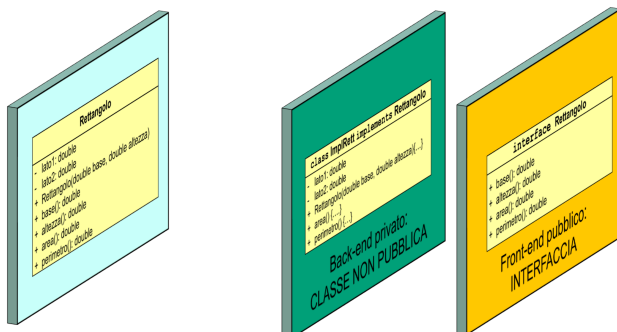
Success

Separare Interfaccia e Implementazione Utilizzare due costrutti distinti invece di uno solo.

Interfaccia (`interface`): Definisce solo il **front-end** (cosa deve fare un oggetto). Contiene solo dichiarazioni di metodi (e costanti `static final`).

Classe Concreta (`class`): Fornisce il **back-end** (implementazione reale) di una o più interfacce.

L'Interfaccia in Java



FINORA: UN SOLO COSTRUTTO

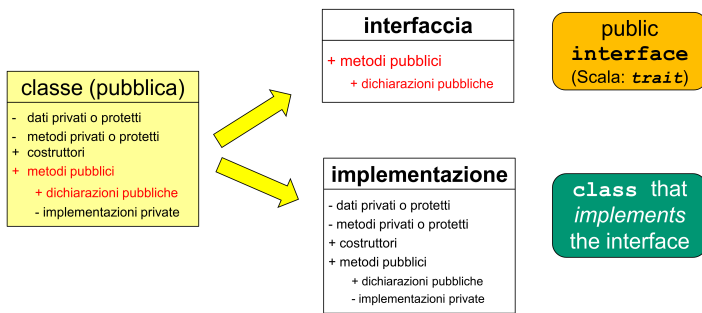
- Tutto è specificato nella *classe pubblica*
- Si specificano nello stesso costrutto e nello stesso momento sia il front-end (pubblico) sia il back-end (privato)

IDEA: USARE DUE COSTRUTTI DISTINTI

- **Front-end pubblico:** *interfaccia*
- **Back-end privato:** *classe (non più pubblica)*
- Le due parti sono specificate in costrutti diversi e non più contemporaneamente

- **Natura:** Una classe astratta portata all'estremo. **Non contiene alcuna definizione ma solo dichiarazioni**. Non contiene stato (variabili di istanza), né costruttori, né implementazioni di metodi (fino a Java 8, poi sono stati introdotti i metodi `default` e `static`, ma il concetto core rimane la pura specifica).
- **Scopo:** Introdurre un **Tipo** puro, usabile per riferimenti e argomenti di metodi.
- **Relazione di Realizzazione (`implements`):** Una classe che si impegna a fornire il codice per tutti i metodi dichiarati in un'interfaccia "realizza" (o "implementa") quell'interfaccia.compatibili
- **Il tipo classe e il tipo interfaccia sono**

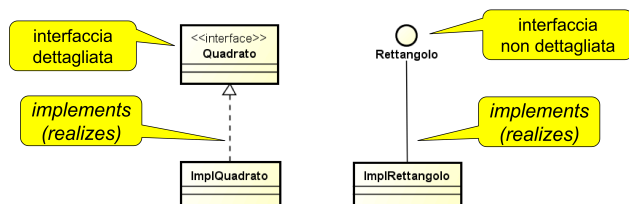
Caratteristica	Classe Astratta (<code>abstract class</code>)	Interfaccia (<code>interface</code>)
Ereditarietà	Singola (<code>extends</code>)	Multipla (<code>implements</code>)
Stato (Campi)	Può definire variabili di istanza	No (solo costanti <code>public static final</code>)
Costruttori	Sì	No
Metodi	Astratti e concreti	Astratti (e <code>default / static</code> da Java 8+)
Visibilità membri	<code>public</code> , <code>protected</code> , <code>private</code>	Solo <code>public</code> (implicito)
Relazione	Ereditarietà (sempre una sottoclasse)	Realizzazione (qualunque classe che implementa) Ereditarietà multipla



```

1  public interface Rettangolo {
2      // Metodi astratti impliciti: public abstract
3      double base();
4      double altezza();
5  }
6
7  class ImplRettangolo implements Rettangolo {
8      private double lato1, lato2;
9      public ImplRettangolo(double altezza, double base) { lato1 = base; lato2 = altezza;}
10     public double altezza() { return lato2; }
11     public double base() { return lato1; }
12     // eventuali metodi aggiuntivi
13 }

```



Ereditarietà Multipla con le Interfacce

Success

Poiché le interfacce non contengono implementazioni (nessun corpo metodo, nessun dato, 0 codice), **non c'è il rischio di collisione** tipico dell'ereditarietà multipla fra classi.

Se ci dovessero essere metodi omonimi comunque non ci sarebbero collisioni

Ereditarietà tra Interfacce: Un'interfaccia può estendere (`extends`) più altre interfacce. Questo serve a comporre tipi e stabilire relazioni concettuali tra astrazioni pure.

```

1  public interface TrapezioRettangolo extends Trapezio { ... }
2  public interface Parallelogrammo extends Quadrilatero { ... }
3
4  // Ereditarietà multipla tra interfacce
5  public interface Rettangolo extends TrapezioRettangolo, Parallelogrammo {
6      // Rettangolo eredita i metodi di entrambe le super-interfacce
7  }

```

Progettare con le Interfacce ("Design for Change")

- Prima le Interfacce:** Si modella la tassonomia dei concetti (astrazioni) usando solo interfacce. Si stabilisce **cosa** il sistema deve fare, in modo pulito e senza vincoli implementativi.
 - Poi le Classi:** Si creano le classi concrete che implementano tali interfacce, ottimizzando per efficienza, riuso del codice, ecc. Queste classi diventano "dettagli implementativi" nascosti al resto del sistema.
- Vantaggi:** Disaccoppiamento architetturale. Il codice che usa un'interfaccia non dipende da una classe specifica, ma solo dal contratto. Questo rende il sistema più facile da mantenere, estendere e testare.

Compatibilità di Tipo e Riferimenti a Interfaccia

- Un'interfaccia definisce un tipo. Non si possono creare istanze dirette (`new Rettangolo()` è illegale).
- Una classe che implementa un'interfaccia `I` è compatibile per assegnamento con il tipo `I`.
- Un riferimento di tipo interfaccia può puntare a un'istanza di **qualsiasi** classe che la implementa.

Esempio: Vista "Orizzontale" vs. "Verticale"

```

1  public interface Rettangolare {

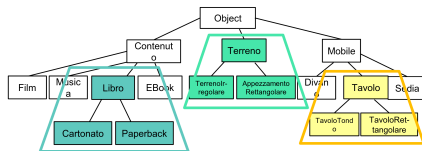
```

```

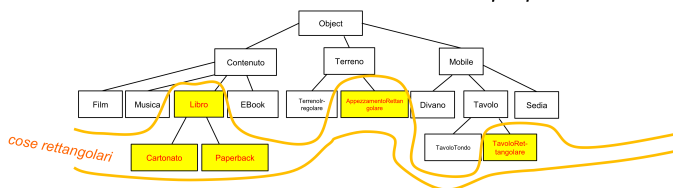
2     double larghezza();
3     double lunghezza();
4 }
5
6 public class Tavolo implements Rettangolare {
7     private double largh, lung, h;
8     public Tavolo(double largh, double lung, double h){
9         this.largh=largh; this.lung=lung; this.h=h;
10    }
11    @Override
12    public double larghezza() { return largh;}
13    @Override
14    public double lunghezza() { return lung; }
15    public double altezza() { return h; }
16
17    public double peso() {...} // in base al peso del legno... // altre cose specifiche del tavolo: colore...
18 }
19 public class Libro implements Rettangolare { ... }
20 public class Appezamento implements Rettangolare { ... }
21
22 public class MyMath {
23     // Questo metodo funziona con QUALSIASI cosa sia Rettangolare,
24     // indipendentemente dalla sua classe concreta.
25     public static double area(Rettangolare r) {
26         return r.larghezza() * r.lunghezza();
27     }
28 }
29
30 public class Main {
31     public static void main(String[] args) {
32         Rettangolare libro = new Libro(22, 16, 3);
33         Rettangolare tavolo = new Tavolo(120, 70, 74);
34
35         System.out.println(MyMath.area(libro)); // Funziona!
36         System.out.println(MyMath.area(tavolo)); // Funziona!
37     }
38 }

```

– i riferimenti a **classi** catturano viste «verticali» della tassonomia



– i riferimenti a **interfacce** catturano viste «orizzontali» della tassonomia, che intercettano entità caratterizzate da certe proprietà



Interfacce e Pattern Factory

1. Il client richiede un oggetto che rispetti una certa interfaccia (es. `Rettangolo`).
2. La **Factory**, in base a logiche interne (es. parametri passati, configurazione), decide quale classe concreta istanziare (es. `ImplRettangolo`, `Quadrato`, ecc.).
3. La Factory restituisce l'istanza come tipo **interfaccia**, nascondendo completamente al client il tipo concreto dell'oggetto.
Es. `java.time` è costruito così

la **Factory Internalizzata**: Si può inserire un metodo `static` di factory direttamente nell'interfaccia.

```

1     public interface Rettangolo {
2         double base();
3         double altezza();
4
5         // Factory internalizzata
6         public static Rettangolo of(double base, double altezza) {
7             // Qui la logica per decidere cosa creare.
8             // Se base == altezza, potremmo restituire un Quadrato
9             if (base == altezza) {
10                 return new ImplQuadrato(base);

```

```

11         } else {
12             return new ImplRettangolo(base, altezza);
13         }
14     }
15 }
16
17 // Uso lato client
18 Rettangolo r = Rettangolo.of(10, 5); // Non so se è un ImplRettangolo o un Quadrato

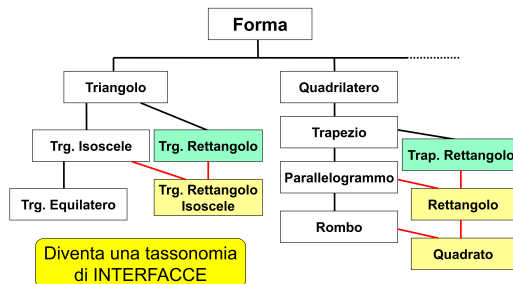
```

Forme Geometriche

Con le sole classi, l'ereditarietà singola impedisce di modellare in modo pulito criteri di classificazione ortogonali (es. "avere i lati paralleli" vs. "avere gli angoli retti"). Un Quadrato è sia un Rettangolo che un Rombo, ma non può ereditare da entrambi.

• Soluzione con Interfacce:

1. Si crea una tassonomia di **interfacce** con ereditarietà multipla.



```

1 public interface Forma { double area(); double perimetro(); }
2 public interface Quadrilatero extends Forma { ... }
3 public interface Parallelogrammo extends Quadrilatero { ... }
4 public interface Rettangolo extends Parallelogrammo, TrapezioRettangolo { ... }
5 public interface Rombo extends Parallelogrammo { ... }
6 // Ereditarietà multipla: Quadrato è sia Rettangolo che Rombo
7 public interface Quadrato extends Rettangolo, Rombo { }

```

2. Si creano **classi concrete** per l'implementazione. Poiché non c'è ereditarietà multipla fra classi, si dovrà scegliere una gerarchia di classi che massimizzi il riuso del codice. Ad esempio, `ImplQuadrato` può estendere `ImplRettangolo` per riutilizzare il codice, implementando poi l'interfaccia `Quadrato`.

```

1 class ImplRettangolo implements Rettangolo { ... }
2
3 class ImplQuadrato extends ImplRettangolo implements Quadrato {
4     public ImplQuadrato(double lato) {
5         super(lato, lato); // Chiama il costruttore di ImplRettangolo
6     }
7     // ...
8 }

```

L'utente vede solo la tassonomia pulita delle interfacce e usa le factory per ottenere le forme.

Numeri Complessi e Reali

Un numero reale è un numero complesso con parte immaginaria nulla. Modellare questa relazione con le classi è inefficiente (ogni reale deve comunque mantenere un campo `im = 0`).

1. **Interfaccia Complex**: Definisce le operazioni generali sui complessi (`getReal()`, `getIm()`, `sum(Complex)`, `mul(Complex)`, ecc.).
2. **Interfaccia Real extends Complex**: Specializza `Complex` aggiungendo metodi che restituiscono `Real` (covarianza del tipo di ritorno).
3. **Classi Separate**: Le classi `ComplexNum` e `RealNum` implementano le rispettive interfacce. `RealNum` non ha bisogno di memorizzare `im` perché il suo metodo `getIm()` restituisce sempre `0`.

```

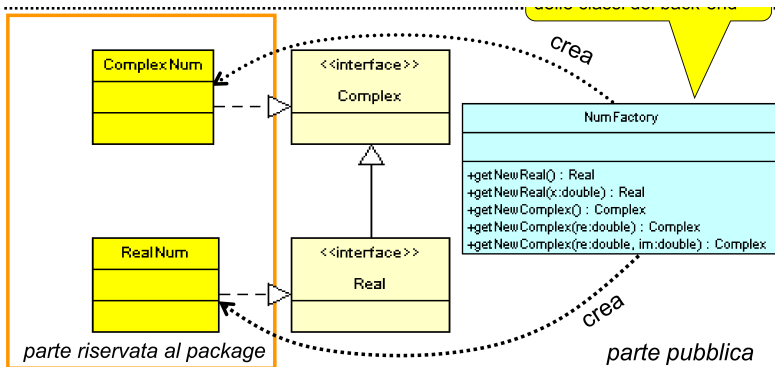
1 // Interfaccia con factory internalizzata
2 public interface Complex {
3     // ... metodi ...
4     public static Complex of(double x, double y) {
5         return new ComplexNum(x, y);
6     }
7     public static Complex of(double x) {
8         return new RealNum(x); // Restituisce un Real!
9     }
10 }
11
12 public interface Real extends Complex {
13     // ... metodi ...
14     public static Real of(double x) {

```

```

15     return new RealNum(x);
16 }
17 }
18
19 // Lato client
20 Complex c = Complex.of(3, 2); // Ottiene un ComplexNum
21 Real r = Real.of(3.14); // Ottiene un RealNum
22 Complex c2 = Complex.of(5); // Ottiene un RealNum, ma visto come Complex!
23
24 public class RealNum implements Real {...}
25 public class ComplexNum implements Complex {...}

```



questa struttura definisce una factory con più funzioni ma è possibile utilizzare una factory più intelligente.

```

1 public class NumFactory {
2     public static Real of() { return new RealNum(); }
3     public static Real of(double x){ return new RealNum(x); }
4     public static Complex of(double x, double y) {
5         return new ComplexNum(x,y);
6     }
7 }
8
9 Real r1 = NumFactory.of(18.5);

```

posso creare due factory una per i numeri complessi e una per i reali.

```

1 public interface Complex {
2     //...
3     public static Complex of(){ return new ComplexNum(); }
4 }
5
6 public interface Real {
7     //...
8     public static Real of(){ return new RealNum(); }
9 }
10
11 Real r1 = Real.of(18.5);

```

Diamond Inheritance in Java

Studente e Lavoratore estendono Persona. StudenteLavoratore è sia Studente che Lavoratore.

• Soluzione:

- Front-End (Interfacce):** Persona, Studente, Lavoratore e StudenteLavoratore sono **interfacce**. StudenteLavoratore può estendere (extends) entrambe Studente e Lavoratore senza problemi.
- Back-End (Classi):** Si creano classi concrete come LaPersona, LoStudente e IlLavoratore. La classe LoStudenteLavoratore **può estendere solo una classe** (es. LoStudente), ma **può implementare l'interfaccia** StudenteLavoratore.

```

1 interface StudenteLavoratore extends Studente, Lavoratore {
2     // interfaccia vuota! Definisce il tipo-intersezione
3 }

```

La tassonomia di **interfacce** è concettualmente perfetta e supporta l'ereditarietà multipla. La tassonomia di **classi** è un dettaglio implementativo vincolato dall'ereditarietà singola di Java, che viene gestito internamente senza influenzare l'utente dell'API.

Genericità e Polimorfismo Orizzontale

Necessità di creare componenti software (strutture dati, metodi) che **funzionino indipendentemente dal tipo specifico** di dato trattato (es. una lista di String o una lista di Frazione).

Approccio Obsoleto: Polimorfismo Verticale con `Object`

Questo metodo sfrutta la radice della gerarchia Java, `Object`, per contenere qualsiasi cosa.

```
1 package edutils;
2
3 public class List {
4     private Object elem;
5     private List next;
6
7     public List() { elem = null; next = null; }
8     public List(Object e) { elem = e; next = null; }
9     public List(Object e, List l) { elem = e; next = l; }
10
11     public Object head() { return elem; }
12     public List tail() { return next; }
13     public boolean isEmpty() { return elem == null; }
14 }
```

Metodo Generico (find) con `Object`

```
1 class Main {
2     public static boolean find(Object[] array, Object elem) {
3         for (Object obj : array)
4             if (obj.equals(elem))
5                 return true;
6         return false;
7     }
8 }
```

Danger

Perdita del Type Safety: Il compilatore non può verificare la coerenza dei tipi.

Mix Illegali: È perfettamente legale (per il compilatore) creare una lista mista, anche se non voluta.

```
1 // Codice che compila ma è logicamente scorretto
2 List l = new List(new Frazione(2,3), new List("Pippo", new List(new Counter2(18))));
```

ClassCastException a Runtime: Passare tipi sbagliati a metodi come `find` compila, ma causa errori in esecuzione.

```
1 String[] arr1 = { "Pippo", "Alberto" };
2 // Compila, ma esplode a runtime perché cerca un Counter in un array di Stringhe
3 boolean res = Main.find(arr1, new Counter(18));
```

Approccio Moderno: Polimorfismo Orizzontale con Tipi Generici (`<T>`)

La soluzione è rendere il tipo un **parametro**. Il vincolo di tipo viene spostato dalla mente del programmatore al codice stesso, permettendo al compilatore di fare controlli.

- Definizione classe: `class NomeClasse<T> { ... }`
- Definizione metodo: `public <T> TipoRitorno nomeMetodo(T param) { ... }`
L'utilizzo di del tipo generico `T` permette di non utilizzare `Object`

Struttura Dati Generica (`List<T>`)

```
1 package edutils2;
2
3 public class List<T> {
4     private T elem;
5     private List<T> next;
6
7     public List() { elem = null; next = null; }
8     public List(T e) { elem = e; next = null; }
9     public List(T e, List<T> l) { elem = e; next = l; }
10
11     public T head() { return elem; }
12     public List<T> tail() { return next; }
13     public boolean isEmpty() { return elem == null; }
14
15     public String toString() {
16         if (isEmpty()) return "";
17         String tailStr = (next == null || next.isEmpty()) ? "" : ", " + next.toString();
18         return elem.toString() + tailStr;
19     }
20 }
```

```
20 }
```

Uso Corretto:

```
1 List<String> l1 = new List<>("Pippo", new List<>("Alberto"));
2 List<Frazione> l2 = new List<>(new Frazione(2,3));
3
4 // ERRORE DI COMPILAZIONE (Mix non consentito):
5 l1 = l2; // Il compilatore blocca questa assegnazione
```

Metodo Generico (find)

La posizione del parametro `<T>` va dichiarata prima del tipo di ritorno.

```
1 public static <T> boolean idem(T[] a, T[] b) //stuttura
2
3 //esempio
4 public static <T> boolean find(T[] array, T elem) {
5     for (T obj : array)
6         if (obj.equals(elem))
7             return true;
8     return false;
9 }
```

Chiamata (Se il tipo viene omissso si usa Object di default):

```
1 String[] arr1 = { "Pippo", "Alberto", "Enrico" };
2 Frazione[] arr2 = { new Frazione(2,3), new Frazione(5,7) };
3
4 nomeclasse.<Counter>idem(array1, array2)//stuttura
5
6 // esempi
7 boolean res1 = Main.<String>find(arr1, "Giusy");
8 boolean res2 = Main.<Frazione>find(arr2, new Frazione(4,6));
9
10 // ERRORE DI COMPILAZIONE:
11 boolean err = Main.<String>find(arr1, new Frazione(2,3));
```

Composizione con Generics

È possibile comporre strutture dati generiche per crearne di più complesse.

```
1 class Stack<T> {
2     private List<T> memory;
3
4     public Stack() {
5         memory = new List<>();
6     }
7
8     // Restituisce Stack<T> per permettere la "Fluent Interface"
9     public Stack<T> push(T e) {
10         memory = new List<>(e, memory);
11         return this;
12     }
13
14     public T pop() {
15         T result = memory.head();
16         memory = memory.tail();
17         return result;
18     }
19
20     public boolean isEmpty() {
21         return memory.isEmpty();
22     }
23
24     public String toString() {
25         return memory.toString();
26     }
27 }
```

Esempio di Fluent Interface:

```
1 Stack<String> stack = new Stack<>();
2 stack.push("pippo").push("pluto").push("paperino");
```

Ereditarietà con Generics (Esempio: ReversibleList)

Le classi generiche possono essere estese.

```
1  class ReversibleList<T> extends List<T> {
2
3      // Costruttori da ridefinire esplicitamente (non ereditati)
4      public ReversibleList() { super(); }
5      public ReversibleList(T e) { super(e); }
6      public ReversibleList(T e, List<T> l) { super(e, l); }
7
8      public ReversibleList<T> reverse() {
9          // Caso base: lista vuota o con un solo elemento
10         if (this.isEmpty() || (!this.isEmpty() && this.tail().isEmpty()))
11             return this;
12         else
13             return reverse(this.tail(), new ReversibleList<>(this.head()));
14     }
15
16     // Metodo ricorsivo ausiliario
17     private ReversibleList<T> reverse(List<T> source, ReversibleList<T> accumulator) {
18         if (source.isEmpty())
19             return accumulator;
20         else
21             return reverse(source.tail(), new ReversibleList<>(source.head(), accumulator));
22     }
23 }
```

Interfacce Standard: Comparable, Comparator e Marker Interface

Le librerie Java forniscono interfacce standard per esprimere abilità o proprietà comuni degli oggetti.

- **Interfacce di abilità:** Definiscono un comportamento (es. `Comparable` per confrontare, `Serializable` per serializzare). Tipicamente contengono uno o più metodi.
- **Interfacce Marker:** Non contengono metodi. Fungono da "etichetta" (`tag`) per segnalare al compilatore o al runtime che una classe possiede una certa proprietà (es. `Serializable`, `Cloneable`).

La Confrontabilità: Ordinamento Naturale con Comparable<T>

Ordinare una lista o un array di oggetti richiede un criterio di confronto. L'approccio generico delega questa responsabilità all'oggetto stesso se esso "sa confrontarsi".

Interfaccia `Comparable<T>`

- Definisce l'**ordinamento naturale** dell'oggetto (es. alfabetico per le stringhe, numerico per i numeri).
- **Metodo:** `int compareTo(T o)`
- **Semantica del valore di ritorno:**
 - -1 (o valore negativo) se `this < o`
 - 0 se `this` equivale a `o` (deve essere coerente con `equals()`)
 - +1 (o valore positivo) se `this > o`

Esempio: Classe `Counter` che implementa `Comparable`

```
1  public class Counter implements Comparable<Counter> {
2      private int val;
3
4      // Costruttore, getter, ecc...
5
6      @Override
7      public int compareTo(Counter that) {
8          // Criterio naturale: confronto basato sul valore numerico
9          if (this.val < that.val) return -1;
10         if (this.val > that.val) return 1;
11         return 0;
12     }
13 }
```

Utilizzo con `Arrays.sort()`

Il metodo `Arrays.sort()` sfrutta internamente `compareTo()` per ordinare senza bisogno di scrivere algoritmi manuali.

```
1  public class Test {
2      public static void main(String[] args) {
3          Counter[] myCounterArray = {
4              new Counter(110),
```

```

5         new Counter(100),
6         new Counter(30),
7         new Counter(50)
8     };
9
10    System.out.println("Pre-ordinamento:");
11    for (Counter c : myCounterArray) System.out.println(c);
12
13    // Ordina usando l'ordinamento naturale definito in compareTo
14    Arrays.sort(myCounterArray);
15
16    System.out.println("\nPost-ordinamento:");
17    for (Counter c : myCounterArray) System.out.println(c);
18 }
19 }

```

Nota: Se una classe non implementa `Comparable`, chiamare `Arrays.sort()` su un suo array causerà una `ClassCastException` a runtime.

Confrontabilità Personalizzata con `Comparator<T>`

Limitazione di `Comparable`:

- Non sempre esiste un criterio "naturale" (es. ordinare una `Persona`: per nome? cognome? età?).
- Potrebbe servire un ordinamento diverso da quello predefinito (es. ordinare stringhe per lunghezza invece che alfabeticamente).

L'oggetto `Comparator`

- Un comparatore è un oggetto **esterno** alla classe da confrontare.
- Incapsula una specifica strategia di confronto.
- Interfaccia: `Comparator<T>`
- **Metodo:** `int compare(T o1, T o2)`
- **Semantica:** Analoga a `compareTo`, ma confronta due oggetti passati come argomento.

Esempio: `Comparator` per `Counter` basato sul modulo 24

Invece di modificare la classe `Counter`, creiamo un comparatore ad-hoc.

```

1  import java.util.Comparator;
2
3  public class MyComp implements Comparator<Counter> {
4      @Override
5      public int compare(Counter x, Counter y) {
6          // Ordina in base al valore % 24
7          int modX = x.getVal() % 24;
8          int modY = y.getVal() % 24;
9
10         if (modX < modY) return -1;
11         if (modX > modY) return 1;
12         return 0;
13     }
14 }

```

Utilizzo di `Comparator` con `Arrays.sort()`

Il metodo `sort` ha un overload che accetta un `Comparator` come secondo argomento.

```

1  Counter[] myCounterArray = { /* ... */ };
2
3  // Uso l'ordinamento personalizzato passando un'istanza del comparatore
4  Arrays.sort(myCounterArray, new MyComp());

```

Esempio: `Comparator` per la classe `Persona`

Se la classe `Persona` non ha un ordinamento naturale logico, creiamo tre comparatori esterni distinti.

```

1  class CognomeComparator implements Comparator<Persona> {
2      public int compare(Persona p1, Persona p2) {
3          // Sfrutta il compareTo delle Stringhe
4          return p1.getCognome().compareTo(p2.getCognome());
5      }
6  }
7
8  class NomeComparator implements Comparator<Persona> {
9      public int compare(Persona p1, Persona p2) {
10         return p1.getNome().compareTo(p2.getNome());
11     }
12 }

```

```

13
14 class EtaComparator implements Comparator<Persona> {
15     public int compare(Persona p1, Persona p2) {
16         // Sfrutta il metodo statico di confronto per interi
17         return Integer.compare(p1.getEta(), p2.getEta());
18     }
19 }
20
21 // Utilizzo
22 Arrays.sort(persone, new CognomeComparator());
23 Arrays.sort(persone, new NomeComparator());
24 Arrays.sort(persone, new EtaComparator());

```

Sintassi Compatta: Classi Anonime

Se un comparatore viene usato una volta sola, definire una classe esterna risulta verboso. Si può usare una **classe anonima** per definire e istanziare il comparatore inline.

```

1 Arrays.sort(persone, new Comparator<Persona>() {
2     @Override
3     public int compare(Persona p1, Persona p2) {
4         return p1.getCognome().compareTo(p2.getCognome());
5     }
6 });

```

- `new Comparator<Persona>() { ... }` non sta istanziando l'interfaccia (cosa impossibile).
- Il compilatore Java genera "dietro le quinte" una nuova classe anonima (es. `ClasseEsterna$1.class`) che implementa l'interfaccia e ne crea un'istanza.

Interfacce Marker (Esempio Dentable)

Un'interfaccia vuota usata per "marcare" una classe come appartenente a una categoria logica, abilitando controlli a tempo di compilazione.

Problema: Vogliamo che solo alcune classi possano essere passate a un metodo `show()`, anche se tutte le classi hanno un metodo `toString()`.

Soluzione:

1. Definire un'interfaccia marker vuota.

```

1 public interface Dentable { } // Nessun metodo

```

2. Fare implementare l'interfaccia solo alle classi "abilitate".

```

1 public class Good implements Dentable { ... }
2 public class Bad { ... } // Non implementa Dentable

```

3. Definire il metodo `show` con un vincolo di tipo.

```

1 public static void show(Dentable d) {
2     System.out.println(d.toString());
3 }

```

Risultato:

```

1 Good g = new Good();
2 show(g); // OK: Good è un sottotipo di Dentable
3
4 Bad b = new Bad();
5 show(b); // ERRORE DI COMPILAZIONE: Bad non è compatibile con Dentable

```

Null Safety e il tipo Optional

Spesso serve indicare che un oggetto o un valore **non esiste**:

- Risultato di una ricerca senza esito
 - Input non valido (es. matrice non quadrata per il determinante)
 - Argomenti opzionali di un metodo
- L'approccio classico è restituire o passare `null`, ma è **fragile**:
- Obbliga il chiamante a **controllare** sempre `if (x != null)` prima di usare l'oggetto
 - Una dimenticanza causa `NullPointerException` (NPE) in punti imprevisti, difficile debug
 - Non funziona con i tipi primitivi (`int`, `double`, ...) che non possono essere `null`

Rappresentare l'assenza nei reali: NaN

Per i tipi `float` e `double`, le classi `Float` e `Double` forniscono le costanti

- NaN (Not a Number).
- NEGATIVE_INFINITY (-infinito).
- POSITIVE_INFINITY (+infinito).
- NaN è usato per rappresentare un risultato mancante (es. forme indeterminate come $\infty - \infty$).
- Esempio: determinante di una matrice non quadrata → si restituisce `Double.NaN`.
- **Controllo: metodo statico** `Double.isNaN(valore)` o `Float.isNaN(valore)`.
- **Limite:** soluzione solo per reali, non per oggetti o altri primitivi.

L'approccio generale: il tipo Optional

Invece di passare `null`, si **incapsula** il valore in un oggetto wrapper `Optional<T>` che:

- non è mai `null`
- può contenere un valore di tipo `T` oppure essere vuoto (nessun valore)
- Si **dichiara** esplicitamente nella **firma** del metodo che un risultato potrebbe mancare, migliorando la leggibilità e la sicurezza.
- Il cliente sa che l'oggetto `Optional` esiste sempre, ma deve verificare la presenza del contenuto prima di usarlo.

Classe `java.util.Optional<T>`

Creazione:

- `Optional.of(valore)` – incapsula un valore **non nullo** (lancia `NullPointerException` se valore è `null`)
- `Optional.empty()` – crea un optional vuoto
- `Optional.ofNullable(T value)` -optional da valore potenzialmente null

Verifica presenza:

- `boolean isPresent()` – true se contiene un valore
- `boolean isEmpty()` (Java 11+) – true se vuoto

Estrazione:

- `T get()` – restituisce il valore se presente, altrimenti lancia `NoSuchElementException`
- `T orElse(T altro)` – restituisce il valore se presente, altrimenti `altro`
- `T getOrElse(Supplier<? extends T> supplier)` – restituisce il valore se presente, altrimenti invoca il supplier

Altro

`filter`, `map`, `flatMap` per trasformazioni condizionali

Optional per tipi primitivi

- `OptionalInt` → metodo `getAsInt()`
- `OptionalLong` → `getAsLong()`
- `OptionalDouble` → `getAsDouble()`
- Queste classi hanno metodi analoghi (`of`, `empty`, `isPresent`, `orElse`, ecc.) ma lavorano con il tipo primitivo.

Es. Uso di base con `OptionalDouble`

```
1 OptionalDouble aliquota = OptionalDouble.of(8.6);
2 if (aliquota.isPresent()) {
3     System.out.println(aliquota.getAsDouble());
4 } else {
5     System.out.println("aliquota indefinita");
6 }
7
8 OptionalDouble detrazione = OptionalDouble.empty();
9 if (detrazione.isEmpty()) {
10    System.out.println("nessuna detrazione");
11 } else {
12    System.out.println(detrazione.getAsDouble());
13 }
14 // Output:
15 // 8.6
16 // nessuna detrazione
```

Metodo orElse

```
1 OptionalDouble aliquota = OptionalDouble.of(8.6);
2 System.out.println(aliquota.orElse(10.6)); // 8.6
3
4 OptionalDouble aliquota2 = OptionalDouble.empty();
5 System.out.println(aliquota2.orElse(10.6)); // 10.6
```

Gestione di riferimenti potenzialmente nulli

```
1 String nome = null;
2
3 // Costruzione condizionale
```

```

4 Optional<String> opt1 = (nome == null) ? Optional.empty() : Optional.of(nome);
5
6 // Equivalente con ofNullable
7 Optional<String> opt2 = Optional.ofNullable(nome);
8
9 System.out.println(opt1); // Optional.empty
10 System.out.println(opt2); // Optional.empty

```

Combinazione con `orElse`

```

1 String s1 = null;
2 String s2 = "ciao";
3
4 Optional<String> opt1 = Optional.ofNullable(s1);
5 Optional<String> opt2 = Optional.ofNullable(s2);
6
7 String value1 = opt1.orElse("boh");
8 String value2 = opt2.orElse("buh");
9
10 System.out.println(value1); // boh
11 System.out.println(value2); // ciao

```

Esempio reale: compito BikeRent

La classe `Rate` ha attributi opzionali come durata massima e orario limite di rientro. Il costruttore e i getter usano `Optional<Duration>` e `Optional<LocalTime>`:

```

1 public class Rate {
2     private final String city;
3     private final double costFirstPeriod;
4     private final Duration firstPeriodDuration;
5     // ...
6     private final Optional<Duration> maxDuration;
7     private final Optional<LocalTime> returnTimeLimit;
8
9     public Rate(String city, double costFirstPeriod, Duration firstPeriodDuration,
10         Optional<Duration> maxDuration, Optional<LocalTime> returnTimeLimit) {
11         this.city = city;
12         // ...
13         this.maxDuration = maxDuration;
14         this.returnTimeLimit = returnTimeLimit;
15     }
16
17     public Optional<Duration> getMaxDuration() {
18         return maxDuration;
19     }
20
21     public Optional<LocalTime> getReturnTimeLimit() {
22         return returnTimeLimit;
23     }
24 }

```

Il chiamante, sapendo che i campi sono opzionali, userà `isPresent()` o `orElse()` per gestirli in modo sicuro.

Esempio reale: compito Cambia Valute

La classe `CambiaValute` ha metodi `acquisto` e `vendita` che restituiscono `OptionalDouble`: se la valuta richiesta non è trattata dall'agenzia, si restituisce un optional vuoto.

```

1  public class CambiaValute {
2      // ...
3      public OptionalDouble acquisto(double importo, String valuta) {
4          Double tasso = tassi.get(valuta);
5          if (tasso == null) {
6              return OptionalDouble.empty();
7          }
8          return OptionalDouble.of(importo / tasso);
9      }
10
11     public OptionalDouble vendita(double importo, String valuta) {
12         Double tasso = tassi.get(valuta);
13         if (tasso == null) {
14             return OptionalDouble.empty();
15         }
16         return OptionalDouble.of(importo * tasso);
17     }
18 }

```

Il cliente può usare `orElse` per fornire un valore di default o un messaggio:

```

1  CambiaValute agenzia = new CambiaValute();
2  double risultato = agenzia.acquisto(100, "USD").orElse(-1);
3  if (risultato == -1) {
4      System.out.println("Valuta non disponibile");
5  }

```

Quando usare (e NON usare) Optional

Casi d'uso appropriati:

- Incapsulare un valore di ritorno che potrebbe essere assente (es. ricerca in una collezione).
- Passare argomenti **realmente** opzionali a un metodo.

Casi da evitare:

- Non usare `Optional` per mascherare violazioni di precondizioni: se un argomento è obbligatorio e non viene fornito, è meglio lanciare un'eccezione.
- Non usarlo per evitare la validazione dell'input: un dato mancante perché l'utente ha sbagliato non è un caso "opzionale" ma un errore.
- Non usare `Optional` come campo di un oggetto serializzabile senza attenzione (la serializzazione di `Optional` era problematica in passato; meglio campi normali con getter che restituiscono `Optional`).

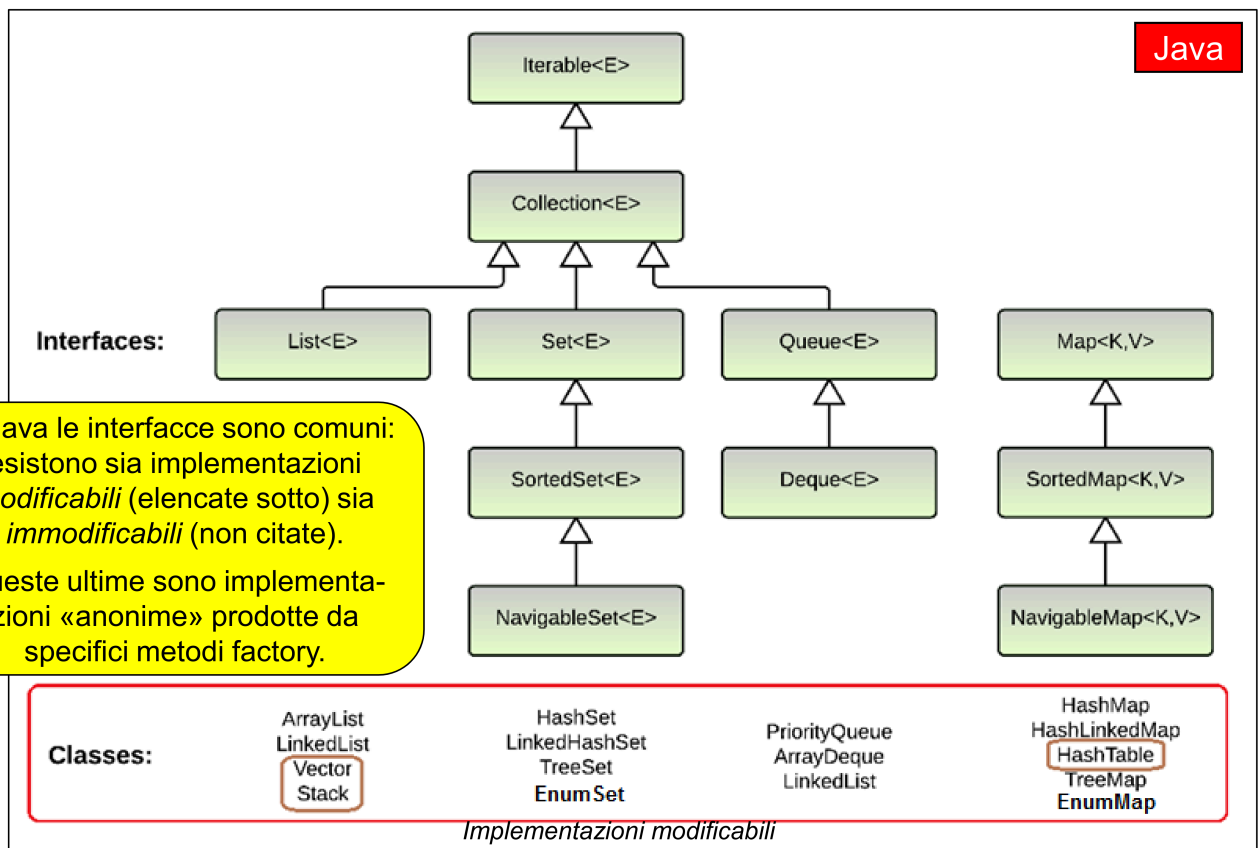
Java Collection Framework (JCF)

è un'architettura unificata per le strutture dati.

Le strutture sono **generiche** in Java: usano il parametro di tipo `<T>` (es. `List<String>`).

I tipi primitivi (`int`, `double`, ...) **non possono essere utilizzati direttamente** come tipo parametrico: si usano i wrapper (`Integer`, `Double`).

Interfacce Fondamentali



In Java le interfacce sono comuni: esistono sia implementazioni *modificabili* (elencate sotto) sia *immodificabili* (non citate).

Queste ultime sono implementazioni «anonime» prodotte da specifici metodi factory.

Iterable<T>

È il punto di partenza: se una classe implementa `Iterable`, può essere scandita con `for (T item : collection)`.

- `Collection<T>` estende `Iterable<T>`.

Collection<T>

Rappresenta un **gruppo di elementi** senza fare ipotesi su ordine, duplicati, posizione.

Operazioni base (nessun `get` !):

- `boolean add(T element)` – aggiunge un elemento (opzionale)
 - `boolean remove(Object o)` – rimuove un elemento se presente
 - `boolean contains(Object o)` – verifica presenza
 - `int size()` – numero di elementi
 - `boolean isEmpty()`
 - `Object[] toArray()` – esporta in array
 - `boolean equals(Object o)` – confronto di collezioni
- Non esiste un metodo `get` perché non esiste una nozione univoca di “posizione”.

Set<T> (estende Collection<T>)

Insieme matematico: non ammette duplicati e non esiste un ordine.

- Il metodo `add` **non inserisce se l'elemento è già presente** (restituisce `false`).
 - `equals` definisce uguaglianza insiemistica: due set sono uguali se contengono gli stessi elementi.
- Non introduce nuovi metodi rispetto a `Collection`, ma **ridefinisce il contratto** per garantire l'assenza di duplicati.

SortedSet<T> (estende Set<T>)

Aggiunge l'**ordinamento totale** sugli elementi.

Gli elementi devono implementare `Comparable<T>` o va fornito un `Comparator<T>` al momento della creazione.

Nuovi metodi:

- `T first()` – elemento minimo
- `T last()` – elemento massimo
- `SortedSet<T> headSet(T toElement)` – elementi strettamente minori di `toElement`
- `SortedSet<T> tailSet(T fromElement)` – elementi maggiori o uguali a `fromElement`
- `SortedSet<T> subSet(T from, T to)` – intervallo `f`

List<T> (estende Collection<T>)

Sequenza ordinata di elementi. Ammette duplicati.

A differenza di `SortedSet` gli elementi sono **ordinati per ordine di inserimento**.

Introduce la **nozione di posizione (indice)**. Ecco perché compare il `get`.

Metodi principali:

- `T get(int index)` – elemento alla posizione `index`
- `void add(T element)` – inserisce in coda
- `T set(int index, T element)` – sostituisce
- `T remove(int index)` – rimuove per posizione

`Queue<T>` **(estende `Collection<T>`)**

Modella una **coda** (tipicamente FIFO, ma può essere anche LIFO).

Metodi di accesso alla testa della coda (due versioni: una lancia eccezione, l'altra restituisce un valore speciale come `null` o `false`):

Metodi

- `add(e)` inserimento
 - `remove()` restituisce e rimuove
 - `element()` ispeziona
- Non c'è accesso posizionale.

`Deque<T>` **(estende `Queue<T>`)**

- **Coda doppia** (Double Ended Queue): inserimento e rimozione da entrambi gli estremi.
- Aggiunge versioni con suffisso `First / Last` (es. `addFirst`, `pollLast`).

`Map<K, V>` **(non estende `Collection`)**

Struttura dati **bidimensionale** (tabella a 2 colonne): associa una **chiave** univoca (tipo `K`) a un **valore** (tipo `V`). Non è una `Collection` perché lavora su coppie chiave-valore.

Metodi essenziali:

- `V put(K key, V value)` – inserisce/aggiorna
- `V get(Object key)` – recupera il valore associato a `key` (null se assente)
- `boolean containsKey(Object key)`
- `boolean containsValue(Object value)`
- `V remove(Object key)`

`Equals` e `hashCode` vanno ridefinite assieme.

Collection views per estrarre i dati in forma di `collection`:

- `Set<K> keySet()` – l'insieme delle chiavi (non può avere duplicati)
- `Collection<V> values()` – la collezione dei valori
- `Set<Map.Entry<K,V>> entrySet()` – l'insieme delle coppie (righe)

`SortedMap<K, V>` **(estende `Map<K, V>`)**

Mappa con **chiavi ordinate**.

Metodi aggiuntivi:

- `K firstKey()` prima chiave
- `K lastKey()` ultima chiave
- `SortedMap<K,V> headMap(K toKey)`
- `SortedMap<K,V> tailMap(K fromKey)`
- `SortedMap<K,V> subMap(K fromKey, K toKey)`

Aggiunte per `Sorted`

- L'ordinamento è basato su `Comparable` (naturale) o su `Comparator` fornito in costruzione.
- Metodi di intervallo (`headSet`, `subSet`, `tailSet`) restituiscono **viste** live, non copie.
- L'iteratore sulle `SortedSet` e sulle `SortedMap` (attraverso `keySet` / `entrySet`) segue l'ordine delle chiavi.

`java.util.Collections`

Contiene **metodi statici di utilità** per operare sulle collezioni.

- `sort(List<T> list)` – merge sort, **solo su liste** perché richiede riposizionamento.
- `binarySearch(List<T> list, T key)` – lista deve essere ordinata.
- `min(Collection<T> coll)`, `max(...)` – restituisce minimo/massimo.
- `reverse(List<T> list)`, `shuffle(List<T> list)`, `fill(List<T> list, T obj)`
- `emptyList()`, `emptySet()`, `emptyMap()` – restituiscono collezioni immutabili vuote.

Implementazioni Concrete (Classi)

Implementazioni general-purpose modificabili

Interfaccia	Hash Table	Array ridimensionabile	Albero bilanciato	Lista concatenata	Hash + lista linkata
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Queue/Deque		ArrayDeque		LinkedList	
Map	HashMap		TreeMap		LinkedHashMap

Set e Map

- Se non serve l'ordinamento `HashSet` / `HashMap` : basati su tabella hash.
- Se serve l'ordinamento `TreeSet` / `TreeMap` : implementano `SortedSet` / `SortedMap` , basati su albero bilanciato.
- `LinkedHashSet` / `LinkedHashMap` : oltre alla tabella hash mantengono una lista doppiamente linkata per tenere traccia dell'**ordine di inserimento** durante l'iterazione.
- Se gli elementi sono Enum: `EnumSet` e `EnumMap` molto efficienti.

List

- `ArrayList` : realizzata con array ridimensionabile, accesso posizionale O(1).
- `LinkedList` : doppia lista concatenata, inserimenti/rimozioni in testa e mezzo efficienti.

Interfaccia	Implementazione	Caratteristiche principali
Set	HashSet	O(1) medio, nessun ordine
	TreeSet	O(log n), ordinamento totale
List	ArrayList	accesso O(1), buona per la maggior parte dei casi
	LinkedList	O(1) per inserimenti/rimozioni in testa; implementa anche <code>Queue</code> / <code>Deque</code>
Map	HashMap	O(1) medio, nessun ordine
	TreeMap	O(log n), chiavi ordinate (interfaccia <code>SortedMap</code>)

Implementazioni immutabili

Non esistono classi pubbliche **per le versioni immutabili**.

Si ottengono tramite **factory method statici**:

- `List.of(...)`
- `Set.of(...)`
- `Map.of(...)` , `Map.ofEntries(...)`

Questi restituiscono istanze di classi interne (non visibili) che non permettono modifiche.

Costruzione delle Collezioni in Java

- **Costruttore vuoto**: `new ArrayList<T>()` , `new HashSet<T>()` , `new HashMap<T>()` ecc. → collezione modificabile inizialmente vuota.

```
1 List<String> l1 = new LinkedList<String>();
2 l1.add("Bologna"); l1.add("Modena");
3
4 List<String> l2 = new ArrayList<String>();
5 l2.add("Ferrara"); l2.add("Ravenna");
6 //costruzioni compatte
7 var l1 = new ArrayList<String>(); // type inference
8 List<String> l1 = new ArrayList<>(); // diamond operator (sottointende lo stesso tipo)
9
10 Set<String> s1 = new HashSet<String>();
11 Set<String> s2 = new TreeSet<String>();
12
13 Map<String,Integer> m = new HashMap<String,Integer>(); m.put("Bologna", 395416); m.put("Modena", 189013);
14 Map<String,Integer> m2 = new TreeMap<String,Integer>();
```

- **Costruttore di copia**: accetta un'altra `Collection` (o `Map`) per inizializzare la nuova collezione con gli stessi elementi.

```
1 List<String> list1 = new ArrayList<>(existingSet);
```

- **Factory per collezioni pre-popolate immutabili**:

Il tipo restituito è una classe ad hoc (non una semplice Map)

```
1 List<Integer> immutableList = List.of(1, 2, 3);
2 Set<String> immutableSet = Set.of("a", "b");
3 Map<String, Integer> immutableMap = Map.of("key1", 10, "key2", 20);
```

```
4 // Per mappe con più di 10 entry: Map.ofEntries(...)
```

Iteratori

`Iterable<T>` è l'interfaccia che permette a un oggetto di essere scandito con il costrutto **for-each** (`for (T x : collezione)`).

Idea base: ogni entità iterabile «sa» come navigare al proprio interno e, su richiesta, è in grado di produrre un **iteratore** adatto alla propria struttura.

Il metodo chiave è `Iterator<T> iterator()`, una sorta di **factory** che restituisce un oggetto iteratore per quella specifica collezione.

Summary

L'iteratore è un oggetto capace di navigare in una cosa «iterabile» del «suo» tipo

- Dichiarare tre metodi:
 - `T next()` – restituisce il prossimo elemento e fa avanzare l'iteratore.
 - `boolean hasNext()` – restituisce `true` se ci sono ancora elementi da visitare.
 - `void remove()` – **operazione opzionale**; rimuove l'ultimo elemento restituito da `next()`.
- Le collezioni della JCF lo implementano; le collezioni **immutabili** (es. quelle ottenute con `List.of()`) lanciano `UnsupportedOperationException` se si invoca `remove()`.

Uso dell'iteratore nel for-each

- Il costrutto `for (T x : coll)` si basa proprio sull'iteratore. Il codice:

```
1 for (T x : coll) {
2     // corpo del ciclo
3 }
```

equivale a:

```
1 for (Iterator<T> i = coll.iterator(); i.hasNext(); ) {
2     T x = i.next();
3     // corpo del ciclo
4 }
```

Grazie a ciò, è possibile iterare su qualsiasi `Collection` (e anche sugli array) con una sintassi uniforme.

Esempio: iteratore implicito ed esplicito

```
1 import java.util.*;
2
3 public class IteratorExample {
4     public static void main(String[] args) {
5         List<String> listOfStrings = List.of("Pippo", "Pluto", "Paperino", "Zio Paperone");
6         iterateOn("lista disney", listOfStrings);
7
8         Set<Number> setOfNums = Set.of(18, 22.2, 37.4F);
9         iterateOn("numeri vari", setOfNums);
10    }
11
12    public static <T> void iterateOn(String msg, Collection<T> coll) {
13        System.out.println("-----");
14        System.out.println(msg);
15
16        // Uso implicito (for-each)
17        System.out.println("uso di iteratore implicito");
18        for (T s : coll) {
19            System.out.println(s);
20        }
21
22        // Uso esplicito
23        System.out.println("uso di iteratore esplicito");
24        Iterator<T> it = coll.iterator();
25        while (it.hasNext()) {
26            System.out.println(it.next());
27        }
28    }
29 }
```

La funzione `iterateOn` è **generica** e funziona su qualunque `Collection<T>`, indipendentemente dal tipo concreto degli elementi.

Regole nell'utilizzo degli iteratori

Cosa si può fare:

- Iterare per **leggere** gli elementi.
- Su collezioni **modificabili**, modificare gli elementi stessi (es. `list.set(i, newValue)`).
Cosa NON si può fare:
- Modificare collection imm modificabili (es. `List.of`): `UnsupportedOperationException`
- Modificare la **struttura della collezione** mentre l'iteratore è attivo (**aggiungere o rimuovere elementi** direttamente sulla collezione). Farlo causa una `ConcurrentModificationException`

```

1  public static <T> void iterateAndChange(String msg, Collection<T> coll) {
2      for (T element : coll) {
3          System.out.println(element);
4          coll.add(element); // ERRORE: modifica la collezione sotto iterazione
5      }
6  }

```

Iteratori e Mappe

`Map<K,V>` non estende `Collection` , quindi **non si può iterare direttamente su una mappa**. Tuttavia, si può iterare sulle sue **viste**:

- `keySet()` → `Set<K>` (insieme delle chiavi)
- `values()` → `Collection<V>` (insieme dei valori)
- `entrySet()` → `Set<Map.Entry<K,V>>` (insieme delle righe)
- `Map.Entry<K,V>` è una **interfaccia interna** a `Map` che rappresenta una coppia chiave-valore, con metodi `getKey()` e `getValue()` .
Es

```

1  import java.util.*;
2
3  public class MapIteratorExample {
4      public static void main(String[] args) {
5          Map<String, Integer> peopleMap = Map.of(
6              "Anna", 21,
7              "Piero", 25,
8              "Silvia", 43,
9              "Guido", 56
10         );
11
12         // 1) Iterazione sulle chiavi
13         for (String name : peopleMap.keySet()) {
14             System.out.println(name + " ha " + peopleMap.get(name) + " anni");
15         }
16
17         // 2) Iterazione sui valori (es. calcolo età media)
18         float sum = 0;
19         for (int value : peopleMap.values()) {
20             sum += value;
21         }
22         System.out.println("L'età media è di " + sum / peopleMap.size() + " anni");
23
24         // 3) Iterazione sulle entry (righe)
25         for (Map.Entry<String, Integer> entry : peopleMap.entrySet()) {
26             System.out.println(entry); // stampa nella forma "chiave=valore"
27         }
28     }
29 }

```

Output:

```

1  Silvia ha 43 anni
2  Piero ha 25 anni
3  Anna ha 21 anni
4  Guido ha 56 anni
5
6  L'età media è di 36.25 anni
7
8  Silvia=43
9  Piero=25
10 Anna=21
11 Guido=56

```

Esempi

Esercizio 1 – Insieme di parole distinte (Set)

Obiettivo: analizzare un elenco di parole (es. argomenti da riga di comando) e:

- stampare le parole duplicate
- contare e stampare le parole distinte
- mostrare l'elenco delle parole senza duplicati

Struttura dati: `Set<String>` perché non ammette duplicati per definizione; il metodo `add` restituisce `false` se l'elemento è già presente.

```
1  import java.util.*;
2
3  public class FindDups {
4      public static void main(String[] args) {
5          Set<String> s = new HashSet<>(); // o TreeSet per ordinamento
6
7          for (String a : args) {
8              if (!s.add(a)) {
9                  System.out.println("Parola duplicata: " + a);
10             }
11         }
12
13         System.out.println(s.size() + " parole distinte: " + s);
14
15         // Stampa personalizzata
16         for (String st : s) {
17             System.out.print(st + " ");
18         }
19     }
20 }
```

Esecuzione:

```
1  > java FindDups Io sono Io esisto Io parlo
2  Parola duplicata: Io
3  Parola duplicata: Io
4  4 parole distinte: [Io, parlo, esisto, sono]
5  Io parlo esisto sono
```

Scelta dell'implementazione:

- `HashSet` : nessun ordine garantito, tempo medio $O(1)$ per `add` / `contains` .
- `TreeSet` : elementi ordinati (alfabeticamente), tempo $O(\log n)$. Si usa se serve un elenco ordinato.

Esercizio 2 – Scambio di elementi in una lista (`List`)

Obiettivo: scambiare due elementi in una sequenza (es. argomenti da riga di comando).

Struttura dati: `List<T>` perché fornisce accesso posizionale (indicizzato) e duplicati ammessi.

Funzione di swap generica:

```
1  static <T> void swap(List<T> list, int i, int j) {
2      T temp = list.get(i);
3      list.set(i, list.get(j));
4      list.set(j, temp);
5  }
```

Programma principale:

```
1  import java.util.*;
2
3  public class EsList {
4      public static void main(String[] args) {
5          List<String> list = new ArrayList<>(); // o LinkedList
6          for (String arg : args) {
7              list.add(arg);
8          }
9          System.out.println(list);
10         swap(list, 2, 3);
11         System.out.println(list);
12     }
13 }
```

Esecuzione:

```

1 > java EsList cane gatto pappagallo canarino cane canarino pescerosso
2 [cane, gatto, pappagallo, canarino, cane, canarino, pescerosso]
3 [cane, gatto, canarino, pappagallo, cane, canarino, pescerosso]

```

Scelta dell'implementazione:

- `ArrayList` : accesso posizionale $O(1)$, ottimo per la maggior parte degli usi.
- `LinkedList` : doppia lista concatenata, efficiente se si fanno molti inserimenti/rimozioni all'inizio o nel mezzo. Implementa anche `Queue` e `Deque`.

Esercizio 3 – Iterazione a ritroso con `ListIterator`

- `ListIterator<T>` estende `Iterator<T>` ed è specifico per le liste. Permette:
 - di muoversi all'indietro (`hasPrevious()` , `previous()`)
 - di conoscere l'indice corrente e di aggiungere/sostituire elementi durante l'iterazione.

```

1 import java.util.*;
2
3 public class EsListIt {
4     public static void main(String[] args) {
5         List<String> list = new ArrayList<>();
6         for (String arg : args) {
7             list.add(arg);
8         }
9
10        // Partiamo dalla fine
11        for (ListIterator<String> it = list.listIterator(list.size()); it.hasPrevious(); ) {
12            System.out.print(it.previous() + " ");
13        }
14    }
15 }

```

Esecuzione:

```

1 > java EsListIt cane gatto cane canarino
2 canarino cane gatto cane

```

Per iniziare dalla fine si usa `list.listIterator(list.size())` ; così il cursore è posizionato dopo l'ultimo elemento.

Esercizio 4 – Conteggio delle occorrenze (`Map`)

Obiettivo: contare quante volte ogni parola compare in un insieme di argomenti.

Struttura dati: `Map<String, Integer>` che associa a ogni parola (chiave) il numero di occorrenze (valore).

Logica:

- Se la parola non è ancora nella mappa (`get` restituisce `null`), la si inserisce con conteggio 1.
- Altrimenti, si incrementa il conteggio esistente.

```

1 import java.util.*;
2
3 public class ContaFrequenza {
4     public static void main(String[] args) {
5         Map<String, Integer> m = new HashMap<>(); // o TreeMap per ordinamento
6
7         for (String parola : args) {
8             Integer freq = m.get(parola);
9             m.put(parola, (freq == null) ? 1 : freq + 1);
10        }
11
12        System.out.println(m.size() + " parole distinte:");
13        System.out.println(m); // toString() produce {parola=occorrenze, ...}
14    }
15 }

```

Esecuzione:

```

1 > java ContaFrequenza cane gatto cane pesce gatto gatto cane
2 3 parole distinte:
3 {cane=3, pesce=1, gatto=3}

```

Iterare sulla mappa risultante:

1. Via `keySet` :

```

1 public static void myPrint(Map<String, Integer> m) {
2     for (String key : m.keySet()) {
3         System.out.println(key + "\t" + m.get(key));
4     }
5 }

```

2. Via `entrySet` (evita di chiamare `get`):

```

1 public static void myPrint2(Map<String, Integer> m) {
2     for (Map.Entry<String, Integer> entry : m.entrySet()) {
3         System.out.println(entry); // stampa "chiave=valore"
4     }
5 }

```

Esercizio 5 – Conteggio con ordinamento (SortedMap)

Obiettivo: come sopra, ma con output **ordinato per chiave**.

Struttura dati: `SortedMap<String, Integer>` con implementazione `TreeMap`.

```

1 import java.util.*;
2
3 public class ContaFrequenzaOrd {
4     public static void main(String[] args) {
5         SortedMap<String, Integer> m = new TreeMap<>();
6
7         for (String parola : args) {
8             Integer freq = m.get(parola);
9             m.put(parola, (freq == null) ? 1 : freq + 1);
10        }
11
12        System.out.println(m.size() + " parole distinte:");
13        System.out.println(m);
14    }
15 }

```

Esecuzione:

```

1 > java ContaFrequenzaOrd cane gatto cane pesce gatto gatto cane
2 3 parole distinte:
3 {cane=3, gatto=3, pesce=1}

```

L'ordine delle chiavi è garantito dal `TreeMap` (ordine naturale delle stringhe o secondo `Comparator` fornito). L'iteratore sulle viste (`keySet`, `entrySet`) seguirà lo stesso ordinamento.

Conversioni fra strutture

Costruttori per copia – tutte le classi del Collection Framework accettano una `Collection` qualsiasi come argomento e ne copiano (shallow) gli elementi.

Metodi di conversione specifici per array.

Da List a Set (e viceversa)

```

1 List<String> l1 = List.of("Pippo", "Pluto", "QuiQuoQua", "Paperino", "Zio Paperone");
2 System.out.println(l1); // [Pippo, Pluto, QuiQuoQua, Paperino, Zio Paperone]
3
4 // Costruttore per copia: da List a HashSet (non ordinato)
5 Set<String> s1 = new HashSet<>(l1);
6 System.out.println(s1); // [Paperino, QuiQuoQua, Pippo, Pluto, Zio Paperone]
7
8 // Costruttore per copia: da List a TreeSet (ordinato)
9 Set<String> s2 = new TreeSet<>(l1);
10 System.out.println(s2); // [Paperino, Pippo, Pluto, QuiQuoQua, Zio Paperone]

```

Da lista immutabile a lista modificabile

```

1 List<String> l1 = List.of("Pippo", "Pluto", "QuiQuoQua", "Paperino", "Zio Paperone");
2 List<String> l2 = new ArrayList<>(l1); // ora l2 è modificabile
3 l2.add("Archimede");
4 l2.add("Paperino");
5 System.out.println(l2); // [Pippo, Pluto, QuiQuoQua, Paperino, Zio Paperone, Archimede, Paperino]

```

Da lista ad array


```

1 List<String> l1 = List.of("Pippo", "Pluto", "QuiQuoQua", "Paperino", "Zio Paperone");
2 String[] a1 = l1.toArray(new String[0]); // l'argomento serve solo per specificare il tipo
3 // ora a1 è un array: {"Pippo", "Pluto", "QuiQuoQua", "Paperino", "Zio Paperone"}

```

Da array a lista

```

1 String[] arr = { "One", "Three", "Ten" };
2 List<String> list = Arrays.asList(arr); // metodo statico della libreria Arrays
3 System.out.println(list); // [One, Three, Ten]

```

Attenzione: la lista ottenuta con `Arrays.asList` è **modificabile** (si possono fare `set`) ma ha **dimensione fissa** (non si possono aggiungere/rimuovere elementi). Per una lista completamente modificabile si può usare:

```

1 List<String> listMod = new ArrayList<>(Arrays.asList(arr));

```

Esempi di ordinamento e ricerca

```

1 class Persona implements Comparable<Persona> {
2     private String nome, cognome;
3     //...
4     public int compareTo(Persona that) {
5         int confrontoCognomi = cognome.compareTo(that.cognome);
6         if (confrontoCognomi != 0) return confrontoCognomi;
7         return nome.compareTo(that.nome);
8     }
9 }

```

Ordinare una lista di persone

La lista deve essere **modificabile** perché `Collections.sort` modifica l'ordine degli elementi.

Opzione 1 – da array a lista tramite `Arrays.asList` :

```

1 List<Persona> l = Arrays.asList(
2     new Persona("Eugenio", "Bennato"),
3     new Persona("Roberto", "Benigni"),
4     new Persona("Edoardo", "Bennato"),
5     new Persona("Bruno", "Vespa")
6 );
7 Collections.sort(l);
8 System.out.println(l);
9 // [Roberto Benigni, Edoardo Bennato, Eugenio Bennato, Bruno Vespa]

```

Opzione 2 – da lista immutabile a lista modificabile con costruttore di copia:

```

1 List<Persona> l = new ArrayList<>(List.of(
2     new Persona("Eugenio", "Bennato"),
3     new Persona("Roberto", "Benigni"),
4     new Persona("Edoardo", "Bennato"),
5     new Persona("Bruno", "Vespa")
6 ));
7 Collections.sort(l);
8 System.out.println(l);

```

Nota: `List.of` produce una lista immutabile, per cui `Collections.sort(l)` lancierebbe `UnsupportedOperationException` se si tentasse di ordinare direttamente quella.

Ordinare un array di persone

```

1 Persona[] persone = {
2     new Persona("Eugenio", "Bennato"),
3     new Persona("Roberto", "Benigni"),
4     new Persona("Edoardo", "Bennato"),
5     new Persona("Bruno", "Vespa")
6 };
7 Arrays.sort(persone);
8 System.out.println(Arrays.toString(persone));

```

Oppure partendo da una lista e convertendo in array:

```

1 List<Persona> l = List.of( /* elementi */ );
2 Persona[] persone = l.toArray(new Persona[0]);
3 Arrays.sort(persone);

```

Ricerca binaria su lista

La lista deve essere **già ordinata** (altrimenti il risultato è imprevedibile). Può essere immutabile, purché ordinata.

```
1 List<Persona> l = List.of(
2     new Persona("Edoardo", "Bennato"),
3     new Persona("Eugenio", "Bennato"),
4     new Persona("Roberto", "Benigni"),
5     new Persona("Bruno", "Vespa")
6 ); // già in ordine naturale
7
8 System.out.println(Collections.binarySearch(l, new Persona("Bruno", "Vespa"))); // 3 (posizione)
9 System.out.println(Collections.binarySearch(l, new Persona("Bruno", "Ape"))); // -1 (non trovato)
```

Se la lista fosse inizialmente non ordinata, si può ordinare prima (richiede modificabilità):

```
1 List<Persona> l = new ArrayList<>(List.of( /* elementi */ ));
2 Collections.sort(l);
3 System.out.println(Collections.binarySearch(l, new Persona("Bruno", "Vespa")));
```

Ricerca in una mappa di persone (con chiave codice fiscale)

Si usa una `Map<String, PersonaCF>` dove la chiave è il codice fiscale.

```
1 class PersonaCF extends Persona {
2     private String cf;
3     public PersonaCF(String nome, String cognome, String cf) {
4         super(nome, cognome);
5         this.cf = cf;
6     }
7     // eventualmente equals & hashCode basati su cf
8 }

```

```
1 Map<String, PersonaCF> m = new TreeMap<>(Map.of(
2     "BNNGNEyymddxxxxu", new PersonaCF("Eugenio", "Bennato", "BNNGNEyymddxxxxu"),
3     "BNGRRTyymddxxxxu", new PersonaCF("Roberto", "Benigni", "BNGRRTyymddxxxxu"),
4     "BNNDRDyymddxxxxu", new PersonaCF("Edoardo", "Bennato", "BNNDRDyymddxxxxu"),
5     "VSPBRNyymddxxxxu", new PersonaCF("Bruno", "Vespa", "VSPBRNyymddxxxxu")
6 ));
7
8 System.out.println(m);
9 // {BNGRRTyymddxxxxu=Roberto Benigni, ...}
10 System.out.println(m.get("BNGRRTyymddxxxxu"));
11 // Roberto Benigni
```

Con `TreeMap` la mappa è ordinata per chiave (codice fiscale).

EnumSet e EnumMap (facoltativo)

EnumSet

- `EnumSet` è una **classe astratta**; non ha costruttori pubblici.
- Le istanze si ottengono tramite **metodi factory statici**:
 - `EnumSet.allOf(Class<E> elementType)` – tutte le costanti dell'enumerazione
 - `EnumSet.noneOf(Class<E> elementType)` – set vuoto
 - `EnumSet.of(E e1), of(E e1, E e2), ...,` fino a 5 argomenti
 - `EnumSet.range(E from, E to)` – intervallo di costanti (secondo l'ordine dichiarato)
 - `EnumSet.copyOf(Collection<E> c)` – copia da una collection esistente (utile anche per set ordinari)
 - `EnumSet.complementOf(EnumSet<E> s)` – complemento rispetto all'insieme totale delle costanti
- **Non accetta elementi nulli** – lancia `NullPointerException`.

Esempio d'uso:

```
1 import java.time.DayOfWeek;
2 import java.util.EnumSet;
3
4 // Set di un solo giorno
5 Set<DayOfWeek> s = EnumSet.of(DayOfWeek.MONDAY);
6 s.add(DayOfWeek.FRIDAY);
7 System.out.println(s); // [MONDAY, FRIDAY]
8
9 // Range di giorni
10 EnumSet<DayOfWeek> workingDays = EnumSet.range(DayOfWeek.MONDAY, DayOfWeek.FRIDAY);
```

```

11 System.out.println(workingDays); // [MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY]
12
13 // Set vuoto
14 EnumSet<DayOfWeek> empty = EnumSet.noneOf(DayOfWeek.class);

```

EnumMap

- EnumMap<K extends Enum<K>, V> è una mappa ottimizzata per chiavi enumerative.
- Ha costruttori pubblici:
 - EnumMap(Class<K> keyType) – mappa vuota
 - EnumMap(EnumMap<K, ? extends V> m) – copia da un'altra EnumMap
 - EnumMap(Map<K, ? extends V> m) – copia da una mappa qualsiasi (se le chiavi sono tutte dello stesso enum)
- Fornisce i classici metodi put, get, keySet, ecc.
- **Non accetta chiavi nulle** (NullPointerException).

Esempio d'uso:

```

1 import java.time.DayOfWeek;
2 import java.util.EnumMap;
3
4 EnumMap<DayOfWeek, Double> costoSosta = new EnumMap<>(DayOfWeek.class);
5 costoSosta.put(DayOfWeek.MONDAY, 1.50);
6 costoSosta.put(DayOfWeek.SATURDAY, 2.00);
7 costoSosta.put(DayOfWeek.SUNDAY, 0.0);
8
9 System.out.println(costoSosta.get(DayOfWeek.MONDAY)); // 1.5
10 System.out.println(costoSosta); // {MONDAY=1.5, SATURDAY=2.0, SUNDAY=0.0}

```

Classi generiche

La notazione <T> (tipo generico) può essere usata per definire **classi o interfacce** parametrizzate rispetto al tipo manipolato.

Caso tipico: nuovi contenitori o strutture dati (stack, code, alberi, ...).

Esempio concreto: vogliamo realizzare un nostro stack (LIFO) generico MyStack<T>, in modo da poter creare stack di interi, stringhe, ecc. senza duplicare codice.

```

1 public class MyStack<T> {
2     public void push(T elem) { /* ... */ }
3     public T pop() { /* ... */ }
4     public boolean isEmpty() { /* ... */ }
5 }

```

Implementazione basata su lista (ArrayList)

È l'approccio più semplice e pulito, perché le collection sono già generiche e non soffrono dei problemi degli array (vedi oltre).

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class MyStack<T> {
5     private List<T> storage;
6
7     public MyStack() {storage = new ArrayList<>();}
8     public void push(T elem) {storage.add(elem);}
9     public T pop() {return storage.remove(storage.size() - 1);}
10    public boolean isEmpty() {return storage.isEmpty();}
11 }

```

```

1 MyStack<Integer> stack = new MyStack<>();
2 stack.push(18);
3 stack.push(22);
4 stack.push(34);
5
6 while (!stack.isEmpty()) {
7     System.out.println(stack.pop()); // stampa 34, 22, 18
8 }

```

Miglioramento: Cascading & Fluent Interface

Per rendere l'uso più fluido (es. concatenare più push) si fa restituire a push l'oggetto stesso (this) invece di void.

```

1 public class MyStack<T> {
2     private List<T> storage;

```

```

3    // ...
4    public MyStack<T> push(T elem) {
5        storage.add(elem);
6        return this;    // permette il cascading
7    }
8    // pop() e isEmpty() invariati
9 }

```

```

1  stack.push(18).push(22).push(34);
2  while (!stack.isEmpty()) {
3      System.out.println(stack.pop());
4  }

```

Nota: in Java i tipi primitivi vengono gestiti con i wrapper (Integer , Double , ...) e l'autoboxing/unboxing automatico.

Implementazione con array e il problema del type erasure

Si potrebbe pensare di usare un array di T come “magazzino” interno:

```

1  public class MyStackBis<T> {
2      private T[] storage;
3      private int elements;
4
5      public MyStackBis(int size) {
6          storage = new T[size];    // ERRORE DI COMPILAZIONE!
7          elements = 0;
8      }
9      // ...
10 }

```

Il problema: in Java non si può istanziare un array di un tipo generico (new T[size]) a causa del *type erasure*.

Cos'è il type erasure?

- I generici in Java esistono solo a livello di compilatore; nel bytecode il tipo T viene **cancellato** e sostituito con Object (o con il tipo bound più restrittivo).
- A runtime non è nota l'effettiva identità di T , quindi non si può allocare un array del tipo esatto garantendo *type safety*.
- **Workaround “povero” (sconsigliato): cast da Object[]**
Si può forzare il cast di un array di Object a T[] :

```

1  class PoorGenericArray<T> {
2      private T[] array;
3
4      public PoorGenericArray(int size) {
5          array = (T[]) new Object[size];    // warning di tipo "unchecked cast"
6      }
7
8      public T get(int index) { return array[index]; }
9      public void set(int index, T elem) { array[index] = elem; }
10     public int size() { return array.length; }
11
12     // Metodo pericoloso: espone l'array interno
13     public T[] getArray() {
14         return array;
15     }
16 }

```

Funziona finché l'array rimane interno, perché i singoli elementi vengono letti/scritti con il tipo T

Esempio senza problemi:

```

1  PoorGenericArray<String> arr = new PoorGenericArray<>(5);
2  arr.set(0, "pluto");
3  arr.set(1, "paperino");
4  System.out.println(arr.get(0));    // pluto

```

Ma se l'array viene esposto all'esterno: getArray() restituisce un T[] che in realtà è un Object[] . Un assegnazione come

```

1  String[] strings = arr.getArray();    // ClassCastException a runtime!

```

```

1  java.lang.ClassCastException: class [Ljava.lang.Object; cannot be cast to class [Ljava.lang.String;

```

Conclusione: l'approccio è fragile e va evitato.

Workaround elegante: factory con `Arrays.copyOf`

Si sfrutta il metodo `Arrays.copyOf(array, length)` passandogli un array “farlocco” (dummy) del tipo corretto, usato solo come template.

```
1 import java.util.Arrays;
2
3 public class GenericArrayFactory {
4     @SafeVarargs
5     public static <T> T[] of(int size, T... dummyArray) {
6         // dummyArray è un varargs di tipo T, usato solo per dedurre il tipo
7         return Arrays.copyOf(dummyArray, size);
8     }
9 }
```

- `copyOf` crea un nuovo array del **vero tipo** di `dummyArray`, lungo `size`.
- L'array restituito non è più un `Object[]`, ma un `T[]` **pulito**, e può essere usato con la notazione `[]` e tutti i metodi degli array.

Esempio d'uso:

```
1 String[] strings = GenericArrayFactory.of(5, new String[0]);
2 Integer[] integers = GenericArrayFactory.of(9, new Integer[0]);
3
4 strings[0] = "pluto";
5 strings[1] = "paperino";
6 strings[3] = "paperone";
7 System.out.println(String.join(",", strings));
8 // stampa: pluto,paperino,null,paperone,null
9
10 integers[0] = 22;
11 integers[1] = 33;
12 integers[2] = 44;
13 System.out.println(Arrays.toString(integers));
14 // [22, 33, 44, null, null, null, null, null, null]
```

Gestione IO - Generale

Java gestisce l'I/O in modo **agnostico rispetto al dispositivo** (file, console, rete, array di byte...), usando il concetto di **stream** (canale di comunicazione unidirezionale).

L'architettura è a “cipolla” (adapter + chain of responsibility): si parte da uno stream di base e lo si “avvolge” con altri stream per aggiungere funzionalità.

La vecchia classe `File` (`java.io`)

- Modella un **file o una directory** (può anche non esistere).
- Costruttore: `new File("percorso")`
- Metodi utili: `exists()`, `isDirectory()`, `list()`, `renameTo()`, `delete()`, `mkdir()`, `isFile()`, ecc.

Problemi di `File`:

- Molti metodi non lanciano eccezioni (solo `boolean` di fallimento, senza motivo).
- `rename` si comporta diversamente tra piattaforme.
- Supporto limitato a link simbolici e metadati.
- Problemi di scalabilità con directory grandi.

`Path` (`java.nio.file`)

È un'interfaccia → nessun costruttore. Si ottiene tramite la factory `Paths.get()`.

Supporta operazioni su percorsi (normalizzazione, risoluzione, relativizzazione, confronti...).

```
1 Path p1 = Paths.get("/tmp/foo");
2 Path p2 = Paths.get(System.getProperty("user.home"), "logs", "foo.log");
3
4 System.out.println(p1.getFileName()); // foo
5 System.out.println(p1.getParent()); // /tmp
6 System.out.println(p1.getNameCount()); // 2
```

Metodi utili di `Path`:

- `normalize()` – rimuove `.` e `..` superflui
- `resolve(Path other)` – concatena percorsi
- `relativize(Path other)` – restituisce il percorso relativo tra due percorsi
- `equals()`, `compareTo()`, `startsWith()`, `endsWith()`

```
1 Path base = Paths.get("/home");
2 Path assoluto = Paths.get("/home/sally/bar");
```

```
3 System.out.println(base.relativize(assoluto)); // sally\bar
```

La classe `Files` (`java.nio.file`)

Contiene solo **metodi statici** per operare su file/directory:

- **Operazioni base:** `exists()`, `isDirectory()`, `move()`, `copy()`, `delete()`, `createDirectory()`, `list()`, `walk()` ...
- **Lettura/scrittura in blocco** (solo per file **piccoli**):
 - `readAllBytes(Path)` → `byte[]`
 - `readAllLines(Path, Charset)` → `List<String>`
 - `write(Path, byte[])` / `write(Path, Iterable<? extends CharSequence>)`

```
1 // Lettura di un piccolo file di testo
2 Path percorso = Paths.get("dati.txt");
3 try {
4     List<String> righe = Files.readAllLines(percorso, StandardCharsets.UTF_8);
5     for (String riga : righe) {
6         System.out.println(riga);
7     }
8 } catch (IOException e) {
9     System.err.println("Errore di lettura: " + e.getMessage());
10 }
```

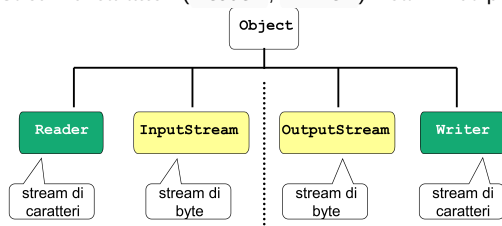
```
1 // Scrittura di un piccolo file binario
2 byte[] dati = {10, 20, 30, 40};
3 Files.write(Paths.get("output.bin"), dati, StandardOpenOption.CREATE);
```

⚠ **Non usare** `readAllBytes` / `readAllLines` **per file grandi** (rischio di memoria e prestazioni).

Il package `java.io` – le basi

Concetto di stream

- Canale **unidirezionale** (input o output).
 - Stream di **byte** (`InputStream`, `OutputStream`) – per dati binari.
 - Stream di **caratteri** (`Reader`, `Writer`) – ottimizzati per testo e Unicode.



Operazione	Byte stream	Character stream
Input	<code>java.io.InputStream</code>	<code>java.io.Reader</code>
Output	<code>java.io.OutputStream</code>	<code>java.io.Writer</code>

Architettura a cipolla (Adapter + Chain of responsibility)

- **Stream di nodo** (incapsulano la sorgente/destinazione fisica):
`FileInputStream`, `FileOutputStream`, `FileReader`, `FileWriter`, `ByteArrayInputStream`, `SocketInputStream` ...
- **Stream di filtro/adattamento** (avvolgono uno stream esistente per aggiungere funzionalità):
`BufferedInputStream`, `DataInputStream`, `ObjectInputStream`, `BufferedReader`, `PrintWriter` ...

Esempio (lettura efficiente di testo da file):

```
1 // Stream di nodo
2 FileReader fr = new FileReader("input.txt");
3 // Stream di adattamento (buffer + readLine)
4 BufferedReader br = new BufferedReader(fr);
5
6 String linea;
7 while ((linea = br.readLine()) != null) {
8     System.out.println(linea);
9 }
10 br.close();
```

Vantaggio: si può comporre a piacere la “cipolla” per ottenere esattamente il comportamento desiderato (buffer, parsing di tipi primitivi, serializzazione oggetti...).

Separatori dipendenti dalla piattaforma

`File` (e anche `Path`) espone costanti per scrivere percorsi portabili:

- `File.separator` → `/` su Unix/Mac, `\` su Windows
- `File.pathSeparator` → `:` su Unix/Mac, `;` su Windows

Meglio usare `Paths.get()` con stringhe “normali” (che accettano entrambi i separatori) o costruire il percorso a pezzi.

Nota: in presenza di eccezioni di I/O (es. `IOException`), Java richiede **controllo obbligatorio** (checked exception) – quindi vanno gestite con `try/catch` o dichiarate con `throws`.

I/O Binario

Stream di byte: classi base astratte

`InputStream` – canale di input a byte:

- `int read()` : legge un byte (0-255) o restituisce -1 se lo stream è terminato; può bloccarsi in attesa di dati.
- `int read(byte[] b)`, `read(byte[] b, int off, int len)` : legge più byte.
- `void close()` : chiude lo stream.

`OutputStream` – canale di output a byte:

- `void write(int b)` : scrive un byte (0-255).
- `void write(byte[] b)`, `write(byte[] b, int off, int len)` : scrive più byte.
- `void flush()` : forza la scrittura dei buffer.
- `void close()` : chiude lo stream.

Entrambe sono astratte: le sottoclassi concrete implementano `read` / `write` adeguate alla specifica sorgente/destinazione.

Stream di byte per file: `FileInputStream` e `FileOutputStream`

`FileInputStream` – input da file:

- Costruttori: `FileInputStream(String nome)` o `FileInputStream(File file)`, lancia `FileNotFoundException` (obbligatoria).
- `int read()` : legge un byte dal file, restituisce -1 a fine file.

`FileOutputStream` – output su file:

- Costruttori: `FileOutputStream(String nome)`, `FileOutputStream(String nome, boolean append)`, analoghi con `File`. Lancia `FileNotFoundException`.
- `void write(int b)` : scrive il byte sul file.

Esempio di lettura di un file binario byte per byte:

```
1  import java.io.*;
2
3  public class LetturaDaFileBinario {
4      public static void main(String args[]) {
5          FileInputStream is = null;
6          try {
7              is = new FileInputStream(args[0]);
8          } catch (FileNotFoundException e) {
9              System.out.println("File non trovato");
10             System.exit(1);
11         }
12         try {
13             int x = is.read();
14             int n = 0;
15             while (x >= 0) {
16                 System.out.print(" " + x);
17                 n++;
18                 x = is.read();
19             }
20             System.out.println("\nTotale byte: " + n);
21         } catch (IOException ex) {
22             System.out.println("Errore di input");
23             System.exit(2);
24         }
25     }
26 }
```

Esempio di scrittura di alcuni byte su file:

```
1  import java.io.*;
```

```

2
3 public class ScritturaSuFileBinario {
4     public static void main(String args[]) {
5         FileOutputStream os = null;
6         try {
7             os = new FileOutputStream(args[0]);
8         } catch (FileNotFoundException e) {
9             System.out.println("Imposs. aprire file");
10            System.exit(1);
11        }
12        try {
13            for (int x = 0; x < 10; x += 3) {
14                System.out.println("Scrittura di " + x);
15                os.write(x);
16            }
17        } catch (IOException ex) {
18            System.out.println("Errore di output");
19            System.exit(2);
20        }
21    }
22 }

```

Stream di adattamento per dati primitivi

`DataInputStream` (implementa `DataInput`) e `DataOutputStream` (implementa `DataOutput`)

- Incapsulano rispettivamente un `InputStream` o un `OutputStream` e forniscono metodi per leggere/scrivere tipi primitivi Java (`readInt` , `writeInt` , `readFloat` , `writeFloat` , `readBoolean` , `writeBoolean` , `readChar` , `writeChar` , `readDouble` , `writeDouble` , ...) e stringhe in formato UTF.
- I metodi di lettura lanciano `EOFException` se lo stream termina inaspettatamente.

Esempio: scrittura di valori primitivi su file usando `DataOutputStream` sopra un `FileOutputStream` :

```

1 import java.io.*;
2
3 public class Esempio1 {
4     public static void main(String args[]) {
5         FileOutputStream fs = null;
6         try {
7             fs = new FileOutputStream("Prova.dat");
8         } catch (IOException e) {
9             System.out.println("Apertura fallita");
10            System.exit(1);
11        }
12        DataOutputStream os = new DataOutputStream(fs);
13        float f1 = 3.1415F;
14        char c1 = 'X';
15        boolean b1 = true;
16        double d1 = 1.4142;
17        try {
18            os.writeFloat(f1);
19            os.writeBoolean(b1);
20            os.writeDouble(d1);
21            os.writeChar(c1);
22            os.writeInt(12);
23            os.close();
24        } catch (IOException e) {
25            System.out.println("Scrittura fallita");
26            System.exit(2);
27        }
28    }
29 }

```

Esempio di rilettura corrispondente con `DataInputStream` :

```

1 import java.io.*;
2
3 public class Esempio2 {
4     public static void main(String args[]) {
5         FileInputStream fin = null;
6         try {
7             fin = new FileInputStream("Prova.dat");
8         } catch (FileNotFoundException e) {
9             System.out.println("File non trovato");
10            System.exit(3);

```



```

11     }
12     DataInputStream is = new DataInputStream(fin);
13     float f2; char c2; boolean b2; double d2; int i2;
14     try {
15         f2 = is.readFloat();
16         b2 = is.readBoolean();
17         d2 = is.readDouble();
18         c2 = is.readChar();
19         i2 = is.readInt();
20         is.close();
21         System.out.println(f2 + ", " + b2 + ", " + d2 + ", " + c2 + ", " + i2);
22     } catch (IOException e) {
23         System.out.println("Errore di input");
24         System.exit(4);
25     }
26 }
27 }

```

Nota: Solo chi scrive conosce l'esatto ordine e tipo dei dati – occorre leggere nello stesso ordine.

Serializzazione di oggetti: `ObjectInputStream` e `ObjectOutputStream`

- **Serializzare** = salvare un oggetto in una rappresentazione binaria (con `ObjectOutputStream`).
- **Deserializzare** = ricostruire l'oggetto dalla sua forma binaria (con `ObjectInputStream`).
- Non tutte le classi sono automaticamente serializzabili: devono implementare l'interfaccia `java.io.Serializable` (marcatore vuoto, nessun metodo). Questo obbliga il progettista a un'esplicita scelta di design.

Interfaccia `Serializable` e numero di versione `serialVersionUID`

- Ogni classe serializzabile dovrebbe dichiarare un campo:

```
1 private static final long serialVersionUID = 1L;
```

- **private** (non accessibile dall'esterno), **static** (vale per l'intera classe), **final** (costante).
- Serve per distinguere versioni della classe: se al momento della deserializzazione il `serialVersionUID` salvato non corrisponde a quello corrente, viene lanciata `InvalidClassException`.
- Se non dichiarato, ne viene calcolato uno di default basato su dettagli del compilatore → rischio di eccezioni inattese. Pertanto è **fortemente consigliato** definirlo esplicitamente.

Esempio: classe `Punto2D` serializzabile

```

1 public class Punto2D implements java.io.Serializable {
2     float x, y;
3     private static final long serialVersionUID = 1;
4
5     public Punto2D(float xx, float yy) { x = xx; y = yy; }
6     public float getX() { return x; }
7     public float getY() { return y; }
8 }

```

Metodi principali

- `ObjectOutputStream.writeObject(Object obj)` – serializza un oggetto (deve essere `Serializable`).
- `ObjectInputStream.readObject()` – restituisce un `Object`; necessario un **downcast** al tipo concreto noto solo a chi ha progettato il protocollo.

Esempio: salvataggio di un `Punto2D`

```

1 Punto2D p = new Punto2D(3.2F, 1.5F);
2 try {
3     FileOutputStream f = new FileOutputStream("data.bin");
4     ObjectOutputStream os = new ObjectOutputStream(f);
5     os.writeObject(p);
6     os.flush();
7     os.close();
8 } catch (IOException e) { ... }

```

Esempio: rilettura

```

1 Punto2D p = null;
2 try {
3     FileInputStream f = new FileInputStream("data.bin");
4     ObjectInputStream is = new ObjectInputStream(f);
5     p = (Punto2D) is.readObject();

```

```

6      is.close();
7      System.out.println("x,y = " + p.getX() + ", " + p.getY());
8  } catch (IOException | ClassNotFoundException e) { ... }

```

Serializzare collezioni di oggetti

Invece di serializzare un oggetto alla volta, si può serializzare l'intera collezione (array, `List`, ecc.) come un unico oggetto, perché le collezioni sono a loro volta oggetti serializzabili. La rilettura restituisce direttamente la collezione.

Esempio con array di `Punto2D` :

```

1  public static void salvaPunti(String filename, Punto2D[] punti) {
2      try {
3          FileOutputStream f = new FileOutputStream(filename);
4          ObjectOutputStream os = new ObjectOutputStream(f);
5          os.writeObject(punti);
6          os.close();
7      } catch (IOException e) {
8          System.err.println("Errore in fase di salvataggio dati");
9      }
10 }
11
12 public static Punto2D[] leggiPunti(String filename) {
13     Punto2D[] punti = null;
14     try {
15         FileInputStream f = new FileInputStream(filename);
16         ObjectInputStream is = new ObjectInputStream(f);
17         punti = (Punto2D[]) is.readObject();
18         is.close();
19     } catch (IOException | ClassNotFoundException e) {
20         System.err.println("Errore in fase di rilettura dati");
21     }
22     return punti;
23 }

```

Utilizzo in un `main` di prova:

```

1  public static void main(String[] args) {
2      final String filename = "punti.bin";
3      Punto2D[] punti = { new Punto2D(3.14F, 1.57F), new Punto2D(8, -8.88F) };
4      System.out.println("Writing " + Arrays.toString(punti));
5      salvaPunti(filename, punti);
6      System.out.println("Now reading...");
7      var puntiRiletti = leggiPunti(filename);
8      System.out.println("Riletti " + Arrays.toString(puntiRiletti));
9  }

```

Nota: Il `serialVersionUID` è obbligatorio per le classi normali. Gli array sono esenti dal controllo di versione (struttura fissa a livello bytecode), ma altre collezioni (es. `ArrayList`) necessitano del numero di versione come ogni altra classe.

Java 7 – Factory methods con `Files`

A partire da Java 7, oltre alla creazione diretta con `new`, si possono ottenere stream binari tramite i metodi statici della classe `Files` (pacchetto `java.nio.file`):

- `InputStream Files.newInputStream(Path p, OpenOption... options)`
- `OutputStream Files.newOutputStream(Path p, OpenOption... options)`

Questi restituiscono stream “pronti all'uso” (tipicamente `FileInputStream` / `FileOutputStream` ma restituiti come tipo astratto), e supportano opzioni di apertura (es. `StandardOpenOption.APPEND`). Si usano insieme a `Paths.get()`.

Esempio: riscrittura del primo esempio di scrittura usando `Files` e incapsulando in `DataOutputStream` :

```

1  import java.io.*;
2  import java.nio.file.*;
3
4  public class NioEsempio1 {
5      public static void main(String[] args) {
6          try (OutputStream fs = Files.newOutputStream(Paths.get("Prova.dat"))) {
7              DataOutputStream os = new DataOutputStream(fs);
8              // ... operazioni di scrittura
9          } catch (IOException e) {
10             System.out.println("Apertura fallita");

```

```

11         System.exit(1);
12     }
13 }
14 }

```

Allo stesso modo per la lettura:

```

1 try (InputStream fin = Files.newInputStream(Paths.get("Prova.dat"))) {
2     DataInputStream is = new DataInputStream(fin);
3     // ... operazioni di lettura
4 }

```

Try-with-resources (Java 7)

Per semplificare la gestione delle risorse (stream, file, socket, ...) è stato introdotto il costrutto **try-with-resources**:

- Le risorse sono aperte all'interno della dichiarazione del `try` :

```

1 try (InputStream is = Files.newInputStream(Paths.get(file))) {
2     // uso lo stream
3 } catch (IOException e) { ... }

```

- Al termine del blocco (sia normalmente che per eccezione) le risorse vengono chiuse automaticamente (devono implementare `AutoCloseable`, condizione soddisfatta da tutti gli stream).
- Elimina il blocco `finally` e il boilerplate di chiusura, migliorando leggibilità e robustezza.
- Attenzione:** le risorse aperte con try-with-resources sono utilizzabili **solo dentro il blocco**; dopo l'uscita sono già chiuse.

Confronto: versione classica vs. try-with-resources per lettura file binario.

Classica:

```

1 FileInputStream is = null;
2 try {
3     is = new FileInputStream(args[0]);
4 } catch (FileNotFoundException e) {
5     System.out.println("File non trovato");
6     System.exit(1);
7 }
8 try {
9     int x = is.read(), n = 0;
10    while (x >= 0) {
11        System.out.print(" " + x); n++; x = is.read();
12    }
13    System.out.println("\nTotale byte: " + n);
14 } catch (IOException ex) {
15     System.out.println("Errore di input");
16     System.exit(2);
17 }

```

Con try-with-resources e factory Files :

```

1 try (InputStream is = Files.newInputStream(Paths.get(args[0]))) {
2     int x = is.read(), n = 0;
3     while (x >= 0) {
4         System.out.print(" " + x); n++; x = is.read();
5     }
6     System.out.println("\nTotale byte: " + n);
7 } catch (IOException ex) {
8     System.out.println("Errore di input");
9     System.exit(2);
10 }

```

PrintStream – menzione

`PrintStream` è uno stream di adattamento per l'output di testo (`print` , `println`), ma è **obsoleto** (deprecato) e non dovrebbe essere usato in nuovo codice. Va ricordato solo perché `System.out` e `System.err` sono di tipo `PrintStream` per ragioni di retrocompatibilità (introdotti in Java 1.0). L'alternativa moderna è `PrintWriter`.

I/O di Testo

L'I/O di testo si basa su **stream di caratteri** (Reader/Writer), più adatti dei byte stream perché gestiscono automaticamente la codifica UNICODE e le convenzioni locali.

- un nucleo (es. `FileReader` , `FileWriter`) che incapsula la sorgente/destinazione fisica
- uno o più adapter (es. `BufferedReader` , `PrintWriter`) che aggiungono funzionalità (bufferizzazione, lettura per righe, stampa formattata).

FileReader e **FileWriter** permettono di leggere/scrivere un file un carattere alla volta.

```
1  import java.io.*;
2
3  public class LetturaDaFileDiTesto {
4      public static void main(String[] args) {
5          FileReader r = null;
6          try {
7              r = new FileReader(args[0]);
8              int x, n = 0;
9              while ((x = r.read()) >= 0) {
10                 System.out.print(" " + (char) x);
11                 n++;
12             }
13             System.out.println("\nTotale caratteri: " + n);
14         } catch (FileNotFoundException e) {
15             System.out.println("File non trovato");
16             System.exit(1);
17         } catch (IOException e) {
18             System.out.println("Errore di input");
19             System.exit(2);
20         } finally {
21             if (r != null) try { r.close(); } catch (IOException e) { }
22         }
23     }
24 }
```

Output esempio:

```
1  N e l   m e z z o   d e l   c a m m i n   d i   n o s t r a   v i t a
2  Totale caratteri: 35
```

BufferedReader e PrintWriter

- `BufferedReader` incapsula un `Reader` e introduce la **lettura per righe** tramite `readLine()` , che restituisce `null` a fine file.
- `PrintWriter` incapsula un `Writer` e offre i metodi `print()` , `println()` e `printf()` per una scrittura semplice e senza eccezioni controllate.

Esempio: lettura e scrittura bufferizzata <-

```
1  // Lettura per righe
2  BufferedReader br = new BufferedReader(new FileReader("input.txt"));
3  String riga;
4  while ((riga = br.readLine()) != null) {
5      // elabora la riga
6  }
7
8  // Scrittura per righe
9  PrintWriter pw = new PrintWriter(new FileWriter("output.txt"));
10 pw.println("Una riga di testo");
11 pw.print("Altra riga senza a capo");
12 pw.close();
13
14 // Costruttore diretto dal nome del file
15 PrintWriter pw2 = new PrintWriter("output.txt");
16 pw2.println("Hello");
```

Input da console

Java 25 introduce la classe `java.lang.IO` (importata automaticamente) con metodi statici `println()` e `readln()` per un accesso alla console più semplice e immediato, adatto anche ai principianti.

```
1  import static java.lang.IO.*;
2
3  void main() {
4      println("Hello world");
5      String nome = readln("Immettere il nome: ");
6      String cognome = readln("Immettere il cognome: ");
7      String eta = readln("Immettere l'età: ");
```

```

8     println(nome + " " + cognome + " ha " + eta + " anni.");
9 }

```

Tokenizzazione delle righe di testo

Dopo aver letto una riga, occorre estrarre i singoli **token** (parole, numeri, date, ecc.) in base a dei **separatori**. In Java gli strumenti principali sono `Scanner` e `String.split`. `StringTokenizer` è considerata obsoleta.

Scanner

Supera i limiti di `StringTokenizer`: può essere costruito su `String`, `File`, `InputStream`, `Readable`.

Non incapsula staticamente la stringa, quindi è efficiente per elaborare più righe.

I delimitatori e i pattern sono basati su **espressioni regolari** (regex).

Metodi principali: `hasNext()`, `hasNextInt()`, `hasNextLine()`, `next()`, `nextInt()`, `useDelimiter(pattern)`, `skip(pattern)`, `findInLine(pattern)`.

Costruzione di uno Scanner

```

1 Scanner sc1 = new Scanner(unaStringa);
2 Scanner sc2 = new Scanner(new File("dati.txt"));
3 Scanner sc3 = new Scanner(System.in); // da tastiera

```

Letture di base (spazi/tab come delimitatore di default)

```

1 Scanner sc = new Scanner(System.in);
2 int i = sc.nextInt();
3 String word = sc.next();

```

Cambio delimitatore

```

1 sc.useDelimiter(":"); // delimitatore personalizzato
2 sc.reset();           // torna al default (spazi/tab)

```

Esempio: separatore unico (virgola)

```

1 Scanner sc = new Scanner(reader);
2 sc.useDelimiter("\\s*,\\s*"); // virgola con eventuali spazi
3 while (sc.hasNext()) {
4     System.out.println(sc.next()); // token già puliti
5 }

```

Esempio 3 (separatori impliciti) con Scanner

Formato: Madre Arianna3471234567 – token: ruolo, nome, telefono.

```

1 Scanner sc = new Scanner(reader);
2 while (sc.hasNext()) {
3     String ruolo = sc.next(); // token fino allo spazio
4     sc.useDelimiter("\\d+"); // ora delimiter = cifre
5     String nome = sc.next(); // token fino alle cifre
6     sc.reset();
7     String tel = sc.next(); // token fino a spazio/EOF
8     System.out.println(ruolo + "/" + nome + "/" + tel);
9 }

```

`findInLine()` (approccio avanzato, sconsigliato se non necessario)

Permette di descrivere l'intera riga con un pattern, estraendo i gruppi:

```

1 String pattern = "(\\w+)(?:\\s+)(\\d+)(\\d+)";
2 if (sc.findInLine(pattern) != null) {
3     MatchResult res = sc.match();
4     String first = res.group(1);
5     String second = res.group(2);
6     String third = res.group(3);
7     sc.nextLine(); // consuma il newline rimasto
8 }

```

`String.split()`

- Metodo della classe `String`, internamente usa `Scanner`.
- Divide la stringa in base a un'espressione regolare e restituisce un array di token.
- **Vantaggi**: semplice quando il delimitatore è costante per tutta la riga.
- **Svantaggi**: non può cambiare delimitatore a metà riga; per casi complessi servono più passi di split annidati.

Esempio base

```
1 String[] parti = "nome, cognome, città".split(",");
2 // token: "nome", " cognome", " città" – serve trim()!
```

Miglioramento con regex che include spazi

```
1 String[] parti = riga.split("\\s*,\\s*");
2 // token: "nome", "cognome", "città" – già puliti
```

Esempio: tabulazioni (una o più)

```
1 String[] parti = riga.split("\\s*\\t+\\s*");
2 // robusto contro più tab consecutivi e spazi attorno
```

Esempio con separatori multipli assimilabili (virgola o trattino)

```
1 String[] items = riga.split("\\s*(,|-)+\\s*");
```

Limite: l'Esempio 3 (separatore implicito tra secondo e terzo token) non è gestibile con una sola `split`. Occorrono più passi:

```
1 String[] parts1 = line.split("\\s+"); // ruolo e resto
2 String ruolo = parts1[0];
3 String other = parts1[1];
4 String[] parts2 = other.split("\\d+"); // nome
5 String nome = parts2[0];
6 String[] parts3 = other.split("\\D+"); // telefono (part3[1] perché part3[0] vuoto)
7 String tel = parts3[1];
8 // Soluzione fragile: fallisce se il nome contiene spazi.
```

Java 21: `splitWithDelimiters()`

Nuovo metodo che include i delimitatori nell'array risultante, utile per separatori impliciti.

```
1 String[] items = line.splitWithDelimiters("\\s+|\\d+", 0);
2 // items: [ruolo, " ", nome, "3471234567"]
3 // Il delimitatore " " può essere scartato.
4 String ruolo = items[0];
5 String nome = items[2];
6 String tel = items[3];
```

Attenzione: funziona bene solo se non ci sono spazi all'interno dei token (altrimenti meglio usare le tabulazioni).

Caso	Caratteristica	Strumento consigliato
1 – Separatore fisso (es. \$)	Semplice, unico delimitatore	<code>split</code> con <code>\\\$</code>
2 – Separatori diversi ma assimilabili (, e -)	Possono essere trattati come uno solo	<code>split</code> con <code>`</code>
3 – Separatori diversi, non assimilabili	Virgola anche all'interno di un token, trattino no	<code>Scanner</code> con cambio delimitatore e <code>skip</code>
4 – Separatori impliciti (cambio tra lettere e numeri)	Es: Arianna3471234567	<code>Scanner</code> con <code>useDelimiter</code> dinamico
5 – Larghezza fissa	Non servono separatori	<code>String.substring()</code>

Parsing di valori numerici e date

Dopo l'estrazione dei token, quasi sempre occorre convertirli in tipi appropriati (es. `int`, `double`, `LocalDate`, `LocalTime`) per costruire il modello dati.

Parsing di stringhe numeriche

Si usano i formattatori di `java.text.NumberFormat`. **Attenzione a:**

- Simbolo delle migliaia e separatore decimale dipendono dalla `Locale`.
- Il simbolo delle migliaia viene ignorato dal parser, causando potenziali errori silenziosi.
- Percentuali e valute hanno formattatori dedicati.

```
1 import java.text.NumberFormat;
2 import java.util.Locale;
3
4 NumberFormat nf = NumberFormat.getNumberInstance(Locale.ITALY);
5 Number num = nf.parse("1.234,56"); // 1234.56
```

```

6  double value = num.doubleValue(); // 1234.56
7
8  // Valuta
9  NumberFormat cf = NumberFormat.getCurrencyInstance(Locale.ITALY);
10 Number prezzo = cf.parse("€ 1.234,56");
11 // Percentuale
12 NumberFormat pf = NumberFormat.getPercentInstance(Locale.ITALY);
13 Number perc = pf.parse("36,75%"); // 0.3675

```

Problema delle migliaia: parse accetta simboli di raggruppamento ovunque, senza segnalare errori. Esempio: "367.5" viene letto come 3675 (il punto è inteso come separatore delle migliaia, non decimale). L'unica soluzione è un controllo manuale: recuperare il `groupingSeparator` e rimuoverlo con `replace` prima del parsing, o validare la posizione con `DecimalFormatSymbols`.

Parsing di date e orari

Si utilizza `java.time.format.DateTimeFormatter`. Due approcci:

- **Dal formattatore:** `formatter.parse(str)` restituisce un `TemporalAccessor`, poi convertito con `LocalDate.from()`.
- **Dalla classe data/ora:** `LocalDate.parse(str, formatter)` direttamente.

```

1  import java.time.*;
2  import java.time.format.*;
3  import java.util.Locale;
4
5  // Stile localizzato
6  DateTimeFormatter fmt = DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT)
7                                     .withLocale(Locale.ITALY);
8  // Approccio 1
9  TemporalAccessor ta = fmt.parse("12/02/23");
10 LocalDate data = LocalDate.from(ta); // 2023-02-12
11
12 // Approccio 2
13 LocalDate data2 = LocalDate.parse("12/02/23", fmt);
14 LocalTime ora = LocalTime.parse("12:30", DateTimeFormatter.ofPattern("HH:mm"));

```

Formati personalizzati: ad esempio "dd/MM/yyyy".

```

1  DateTimeFormatter customFmt = DateTimeFormatter.ofPattern("dd/MM/yyyy");
2  LocalDate data = LocalDate.parse("25/01/2022", customFmt);

```

Nota: il parsing fallisce con `DateTimeParseException` se il formato non corrisponde.

Struttura tipica di un lettore da file in un compito

1. Aprire il file con `BufferedReader(new FileReader(...))`.
2. Leggere riga per riga con `readLine()`.
3. Tokenizzare la riga (con `split` o `Scanner`).
4. Convertire i token in tipi concreti (es. `Double.parseDouble`, `Integer.parseInt`, `NumberFormat.parse`, `LocalDate.parse`).
5. Costruire gli oggetti del modello e inserirli in una collezione.

Esempio da un ipotetico file di spese sanitarie (separatore `;`):

```

1  List<Spesa> spese = new ArrayList<>();
2  try (BufferedReader br = new BufferedReader(new FileReader("spesesanitarie.txt"))) {
3      String line;
4      while ((line = br.readLine()) != null) {
5          String[] parts = line.split("\\s*;\\s*");
6          LocalDate data = LocalDate.parse(parts[0], DateTimeFormatter.ofPattern("dd/MM/yyyy"));
7          String emittente = parts[1];
8          int numVoci = Integer.parseInt(parts[2]);
9          NumberFormat cf = NumberFormat.getCurrencyInstance(Locale.ITALY);
10         double totale = cf.parse(parts[3]).doubleValue();
11         // ... leggere le voci successive con un ciclo for
12     }
13 }

```

Lambda Expression

La visione tradizionale

- **Separazione netta:** Il software è suddiviso rigidamente tra **Dati** (entità passive, semplici o strutturate, come variabili, array o istanze di oggetti) e **Codice** (entità attive e immutabili che rappresentano il comportamento).

- **Ispirazione hardware:** Questa impostazione deriva dall'architettura dei processori (es. architettura di von Neumann/Assembly), dove dati e codice risiedono in segmenti di memoria ben separati.
- **Limitazioni nei linguaggi tradizionali:** In questa ottica, una funzione è un mero costrutto sintattico dotato di nome, firma (signature) e corpo, invocabile solo tramite l'operatore `()`. Di conseguenza, una funzione: non può essere assegnata a variabili; non può essere passata come argomento ad altre funzioni (i puntatori a funzione del C contengono solo un indirizzo di memoria, non sono vere entità funzionali strutturate); non può essere restituita da un'altra funzione (non si può "sintetizzare comportamento" a runtime); non può essere definita "al volo" o dinamicamente (es. tramite un operatore simile a `new`).

La visione funzionale

- **Funzioni come dati:** Le funzioni vengono interpretate come un particolare tipo di dato strutturato. Possiedono proprietà intrinseche (nome, argomenti, tipo di ritorno, codice associato) accessibili tramite metodi accessori, ma mantengono la caratteristica unica di essere **eseguibili**.
- **First-class entities:** Nei linguaggi puramente funzionali, le funzioni godono degli stessi diritti di qualsiasi altro tipo di dato (assegnazione, passaggio come parametro, generazione dinamica).
- **Utilità del passaggio di "comportamento":** Permette di iniettare logica flessibile dentro algoritmi generali (es. comparatori personalizzati per algoritmi di ordinamento) o definire funzioni di callback per la gestione asincrona degli eventi nelle interfacce grafiche.

La sintassi delle lambda expression in Java

Una lambda expression è una **funzione anonima**, espressa con una notazione leggera analoga a quella matematica (es. $x \rightarrow 2x + 1$).

Struttura sintattica generale

```
1 (lista_argomenti) -> { corpo_della_funzione }
```

Regole di omissione e semplificazione

1. **Omissione dei tipi (type inference):** Di norma, i tipi dei parametri in ingresso possono essere omessi se il compilatore è in grado di dedurli in modo univoco dal contesto d'uso (*target typing*).
2. **Omissione delle parentesi tonde:** Se la funzione accetta **un solo argomento**, le parentesi tonde della lista argomenti possono essere omesse.
3. **Omissione delle parentesi graffe e di `return`:** Se il corpo della funzione è composto da **una sola espressione (statement)**, si possono omettere le parentesi graffe e la parola chiave `return`. Il valore dell'espressione viene restituito automaticamente.

Esempi di scrittura sintattica in Java:

```
1 // Forma estesa, con tipi espliciti e blocco di codice
2 (int x, int y) -> { return 2 * x - y; }
3
4 // Forma abbreviata, con inferenza dei tipi e statement singolo
5 (x, y) -> 2 * x - y
6
7 // Forma con un solo argomento (niente tonde, niente graffe)
8 x -> x * x
```

Le interfacce funzionali (interfacce SAM)

In Java, il tipo di una lambda expression è rappresentato da una **Interfaccia Funzionale**.

- **Definizione:** Una normale interfaccia che dichiara **un solo metodo astratto** (*Single Abstract Method* o SAM).
- **Annotazione `@FunctionalInterface`:** Non è obbligatoria per la compilazione, ma funge da vincolo per lo sviluppatore; se l'interfaccia contiene per errore più di un metodo astratto (o nessuno), il compilatore solleva un errore di compilazione.
- Il metodo astratto presente nell'interfaccia è quello che incapsulerà il codice della lambda.

Le 4 macro-categorie principali (generiche):

1. `Function<T, R>`: Rappresenta una trasformazione da un tipo `T` (dominio) a un tipo `R` (codominio). Metodo astratto: `R apply(T t)`.
2. `Consumer<T>`: Accetta un argomento di tipo `T` ed esegue un'operazione senza restituire alcun risultato (codominio vuoto/void). Metodo astratto: `void accept(T t)`.
3. `Supplier<T>`: Non accetta argomenti (dominio vuoto) e produce/fornisce un valore di tipo `T`. Metodo astratto: `T get()`.
4. `Predicate<T>`: Accetta un argomento di tipo `T` e restituisce un valore booleano (`boolean`). Metodo astratto: `boolean test(T t)`.

Estensioni a due argomenti:

- Abbinate al prefisso `Bi-` per gestire domini composti da due variabili generiche: `BiFunction<T, U, R>`, `BiConsumer<T, U>`, `BiPredicate<T, U>`.
Operatori specializzati (dominio == codominio):
- Quando i tipi di ingresso e uscita coincidono, si usano gli operatori per evitare ridondanze: `UnaryOperator<T>` (estende `Function<T, T>`) e `BinaryOperator<T>` (estende `BiFunction<T, T, T>`).
- Per evitare boxing e unboxing java introduce corrispettive interfacce per i tre tipi primitivi
- **Esempi:** `IntUnaryOperator`, `DoubleBinaryOperator`, `LongSupplier`, `ToIntFunction<T>`.

Assegnazione e chiamata (invocazione) delle lambda

Dal punto di vista del compilatore, scrivere un literal lambda equivale a istanziare una classe anonima preposta a implementare l'interfaccia funzionale di riferimento.

```
1 // Scrittura sviluppatore:
2 IntUnaryOperator f = x -> x + 1;
3
4 // Traduzione concettuale del compilatore:
5 IntUnaryOperator f = new IntUnaryOperator() {
6     @Override
7     public int applyAsInt(int x) {
8         return x + 1;
9     }
10 };
```

Poiché la lambda viene convertita in un oggetto classico, per eseguirla è **obbligatorio chiamare esplicitamente il nome del suo unico metodo astratto**. Non è possibile usare la sintassi diretta `f(argument)`.

- **Inestetismo del naming variabile:** Il nome del metodo interno da invocare non è fisso o universale, ma cambia a seconda dell'interfaccia funzionale adottata.

```
1 // Caso 1: Operatore unario su interi (metodo applyAsInt)
2 IntUnaryOperator f = x -> x + 3;
3 int res1 = f.applyAsInt(4);
4
5 // Caso 2: Funzione generica (metodo apply)
6 Function<String, Integer> convertitore = s -> s.length();
7 int res2 = convertitore.apply("Java");
8
9 // Caso 3: Fornitore primitivo (metodo getAsDouble)
10 DoubleSupplier casuale = () -> Math.random();
11 double res3 = casuale.getAsDouble();
```

method reference

Quando il corpo di una lambda si limita a invocare un metodo già esistente senza effettuare elaborazioni aggiuntive o alterare gli argomenti, è possibile utilizzare una scorciatoia sintattica compatta basata sull'operatore doppio due punti `::`.

L'uso dell'operatore `::` non esegue il metodo, ma genera un riferimento ad esso mappabile sulla firma dell'interfaccia funzionale attesa.

1. Metodi statici di una classe:

Sintassi: `NomeClasse::nomeMetodoStatico`

Equivalenza: `x -> Math.sin(x)` diventa `Math::sin`

2. Metodi di istanza di un oggetto specifico:

Sintassi: `NomeIstanza::nomeMetodoIstanza`

Equivalenza: `x -> oggettoContenitore.elabora(x)`

Caso this: All'interno di una classe, `this::myHandle` fa riferimento a un metodo d'istanza della classe corrente.

3. Metodi di istanza di un oggetto arbitrario (fornito come primo parametro):

Sintassi: `NomeClasse::nomeMetodoIstanza`

Equivalenza: `p -> p.getCognome()` diventa `Persona::getCognome` (il compilatore capisce che il metodo `getCognome` va invocato sull'oggetto di tipo `Persona` passato come parametro della lambda).

Comparator e la fabbrica dei comparatori

L'interfaccia `Comparator<T>` ha l'unico scopo di esporre il metodo `int compare(T o1, T o2)`.

```
1 // Ordinamento per stringhe
2 Arrays.sort(persone, (p1, p2) -> p1.getCognome().compareTo(p2.getCognome()));
3
4 // Ordinamento per tipi primitivi (usando i metodi statici di confronto delle classi wrapper)
5 Arrays.sort(persone, (p1, p2) -> Integer.compare(p1.getEtà(), p2.getEtà()));
```

La fabbrica dei comparatori (`Comparator.comparing`)

Java quindi dei metodi factory statici all'interno dell'interfaccia `Comparator` per automatizzare la sintesi dei comparatori a partire da una funzione estrattrice (*extractor*):

```
1 // Uso della Factory con Lambda Extractor
2 Arrays.sort(persone, Comparator.comparing(p -> p.getCognome()));
3
4 // Massima espressione: Factory combinata con Method Reference
5 Arrays.sort(persone, Comparator.comparing(Persona::getCognome));
```

Varianti per tipi primitivi

Al fine di evitare la creazione di oggetti wrapper durante i confronti massivi, si devono usare le factory dedicate per primitivi:

- `Comparator.comparingInt(Persona::getEtà)`
- `Comparator.comparingDouble(...)`
- `Comparator.comparingLong(...)`

Ordinamenti multipli in cascata (`thenComparing`)

è possibile concatenare molteplici criteri di ordinamento (principale, secondario, terziario, ecc.) in modo dichiarativo ed estremamente leggibile:

```
1  Arrays.sort(persone,
2      Comparator.comparing(Persona::getCognome)    // Criterio 1: Cognome
3      .thenComparing(Persona::getNome)              // Criterio 2: Nome
4      .thenComparingInt(Persona::getEtà)            // Criterio 3: Et  (primitivo)
5  );
```

Limiti della type inference

Il compilatore esegue l'inferenza del tipo partendo dal contesto della firma del metodo ospitante (es. se `playlist`   una `List<Song>` , `Arrays.sort` si aspetta un `Comparator<Song>`). Tuttavia, nelle catene di metodi lunghe, il compilatore potrebbe non riuscire a determinare i tipi dei singoli passaggi intermedi, causando un fallimento dell'inferenza.

```
1  // ERRORE DI COMPILAZIONE: La Type Inference fallisce sui passaggi successivi al primo
2  Collections.sort(playlist,
3      comparing(p1 -> p1.getTitle())                // Qui il tipo Song viene dedotto da playlist
4      .thenComparing(p1 -> p1.getDuration())         // ERRORE: Contesto intermedio non chiaro
5  );
```

Soluzioni

1. **Usare riferimenti a metodo espliciti (consigliato):** Forniscono direttamente la classe di partenza (`Song::getTitle`), eliminando l'ambiguit .
2. **Tipizzare esplicitamente l'argomento della lambda:** Forzare il tipo all'interno dei parametri in ingresso della prima lambda: `(Song p1) -> p1.getTitle()` .
3. **Specificare esplicitamente i generics nella chiamata del metodo:** Scrivere la chiamata antepo­endo i tipi alla factory: `Comparator.<Song, String>.comparing(...)` .

Riferimenti ai costruttori (constructor reference)

I riferimenti ai costruttori permettono di memorizzare o passare come parametro la capacit  di generare una nuova istanza di una classe.

```
1  NomeClasse::new
```

Il costruttore referenziato viene mappato automaticamente su un'interfaccia funzionale compatibile con la sua lista argomenti:

- **Costruttore senza argomenti (default):** Si mappa su un `Supplier<T>` .
- **Costruttore con 1 argomento (U):** Si mappa su una `Function<U, T>` .
- **Costruttore con 2 argomenti (U , V):** Si mappa su una `BiFunction<U, V, T>` .

```
1  // Associazione a un costruttore vuoto (Senza argomenti)
2  Supplier<Persona> ctorVuoto = Persona::new; // Mappa public Persona()
3  Persona p1 = ctorVuoto.get();
4
5  // Associazione a un costruttore a due argomenti
6  BiFunction<String, String, Persona> ctorDati = Persona::new; // Mappa public Persona(String c, String n)
7  Persona p2 = ctorDati.apply("Rossi", "Mario");
```

In presenza di costruttori complessi (es. a 3 o 4 argomenti) lo sviluppatore **deve obbligatoriamente definire un'interfaccia personalizzata (custom SAM)**:

```
1  // Definizione delle interfacce custom
2  interface TriFunction<T, U, V, R> { R apply(T t, U u, V v); }
3  interface QuadriFunction<T, U, V, W, R> { R apply(T t, U u, V v, W w); }
4
5  // Cattura dei costruttori complessi tramite Method Reference
6  TriFunction<String, String, Integer, Persona> ctor3Args = Persona::new;
7  QuadriFunction<String, String, Integer, Boolean, Persona> ctor4Args = Persona::new;
```

Caso d'uso

I riferimenti ai costruttori risultano fondamentali quando si deve comunicare a un algoritmo o a una struttura dati centralizzata *come* istanziare gli elementi interni.

Un caso classico   il metodo degli Stream `Stream.toArray()` , che necessita del riferimento al costruttore dell'array specifico per allocare correttamente la memoria a runtime: `Persona[]::new`

Iterazione interna (forEach, andThen e compose)

Si delega il controllo del ciclo direttamente alla collezione stessa invocando il metodo `forEach()`. Lo sviluppatore si limita a passare l'azione (sotto forma di `Consumer`) da applicare a ciascun elemento.

```
1 // Iterazione Esterna Tradizionale
2 for (Persona p : persone) {
3     System.out.println(p);
4 }
5
6 // Iterazione Interna con Lambda Expression
7 persone.forEach(p -> System.out.println(p));
8
9 // Iterazione Interna ottimizzata con Method Reference
10 persone.forEach(System.out::println);
```

Vincolo architetturale sugli array: il metodo `forEach` è definito sulle classi che implementano l'interfaccia `Collection`, non sugli array nativi. Per applicare l'iterazione interna su un array, è necessario prima effettuare la conversione in lista tramite `Arrays.asList(array)`.

Il vincolo delle variabili di chiusura (*effectively final*)

Quando una lambda expression fa riferimento a una variabile locale dichiarata all'esterno del suo corpo, si genera una **chiusura (closure)**.

Non è consentito riassegnare o incrementare variabili locali dentro la lambda.

```
1 // QUESTO CODICE NON COMPILA IN JAVA
2 int sommaEta = 0;
3 Arrays.asList(persone).forEach(p -> sommaEta += p.getEta()); // ERRORE: la variabile esterna deve essere final!
```

Per aggirare questo limite e accumulare dati mantenendo l'iterazione interna, è necessario incapsulare il valore primitivo all'interno di un oggetto wrapper modificabile.

```
1 // Soluzione tramite la definizione di una classe anonima Object dotata di proprietà mutabile
2 var wrapper = new Object() { int ageSum = 0; };
3
4 Arrays.asList(persone).forEach(p -> wrapper.ageSum += p.getEta()); // Compila! Il riferimento a 'wrapper' non cambia
5
6 System.out.println("Età media = " + (double) wrapper.ageSum / persone.length);
```

Composizione di funzioni e iterazioni composte

Java consente di unire due oggetti-funzione per sintetizzare una terza funzione complessa senza scriverne la logica da zero.

- `f.andThen(g)` : Esegue prima la funzione `f` e sul risultato applica la funzione `g` (sequenza logica: $g(f(x))$).
- `f.compose(g)` : Esegue prima la funzione `g` e sul risultato applica la funzione `f` (composizione logica: $f(g(x))$).

```
1 IntUnaryOperator mulBy3 = x -> x * 3;
2 IntUnaryOperator incBy1 = x -> x + 1;
3
4 // Sequenza logica: (4 * 3) + 1 = 13
5 int y = mulBy3.andThen(incBy1).applyAsInt(4);
6
7 // Composizione logica: 3 * (4 + 1) = 15
8 int z = mulBy3.compose(incBy1).applyAsInt(4);
```

Vincolo omogeneo di Java: In Java, `andThen` e `compose` possono essere usate per comporre solo interfacce funzionali del medesimo tipo omogeneo (es. due `IntUnaryOperator` o due `IntConsumer`). Non è ammesso miscelare interfacce diverse nativamente. Nel caso dei `Consumer` la composizione `andThen` significa che entrambi i consumatori eseguiranno in sequenza la loro operazione sul medesimo valore in ingresso.

Iterazioni composte e fallimento del target type

L'abbinamento di funzioni composte all'interno di un'iterazione interna (es. combinare due filtri o due azioni in un `forEach`) rappresenta uno scenario avanzato che spinge l'inferenza del compilatore al punto di rottura, specialmente se si usano i *method reference*.

```
1 // SCENARIO: Si vogliono stampare le persone usando due metodi d'istanza alternativi della classe Persona
2 // ERRORE DI COMPILAZIONE: Il compilatore non riesce a dedurre il tipo intermedio del Method Reference
3 elencoPersone.forEach(Persona::printIfMature.andThen(Persona::printIfYoung)); // NON COMPILA!
```

Motivo del fallimento: Un *method reference* preso singolarmente non fornisce al compilatore informazioni strutturate sufficienti sui parametri e sul contesto d'uso immediato se agganciato a un metodo in cascata come `.andThen()`.

Soluzioni

1. Riformulazione tramite cast esplicito:

Si inserisce un cast esplicito sul primo elemento della catena per comunicare al compilatore l'interfaccia funzionale di riferimento, sbloccando

l'inferenza per i passi successivi:

```
1  elencoPersone.forEach(  
2      ((Consumer<Persona>) Persona::printIfMature).andThen(Persona::printIfYoung)  
3  );
```

2. Spostamento in una funzione ausiliaria tipizzata:

Si incapsula la `andThen` all'interno di un metodo generico statico (`combina`) i cui parametri di ingresso forzano la tipizzazione corretta dei parametri:

```
1  // Metodo ausiliario definito nella classe  
2  private static <T> Consumer<T> combina(Consumer<T> f1, Consumer<T> f2) {  
3      return f1.andThen(f2);  
4  }  
5  
6  // Chiamata pulita nel flusso principale  
7  elencoPersone.forEach(combina(Persona::printIfMature, Persona::printIfYoung));
```

3. Uso di una funzione identità (metodo `itself`):

Si crea una funzione di passaggio che restituisce l'argomento ricevuto, ma che ha il compito strutturale di esplicitare il tipo formale dell'interfaccia al compilatore:

```
1  // Metodo identità ausiliario  
2  private static <T> Consumer<T> itself(Consumer<T> f) {  
3      return f;  
4  }  
5  
6  // Chiamata nel flusso  
7  elencoPersone.forEach(itself(Persona::printIfMature).andThen(Persona::printIfYoung));
```

Varianza e Wildcard

Gli array in Java soffrono di un problema più subdolo legato all'ereditarietà: la **covarianza**. Poiché `Integer` deriva da `Object`, Java permette di fare questo:

```
1  Integer[] arrayOfInt = new Integer[4];  
2  Object[] arrayOfObjects = arrayOfInt; // Compila correttamente  
3  arrayOfObjects[1] = "ciao"; // ATTENZIONE!
```

Sebbene la compilazione abbia successo, a run-time l'inserimento di una stringa in un array che in memoria è effettivamente di interi causerà una `java.lang.ArrayStoreException`. Gli array in Java non sono "type safe" in scrittura.

Le collezioni sono volutamente "blindate" per evitare disastri: non puoi usare una lista di Gatti come se fosse una lista generica di Animali, altrimenti rischiaresti di infilarti dentro un Cane per sbaglio. Questa rigidità garantisce la sicurezza del programma, ma rende difficilissimo scrivere codice flessibile e riutilizzabile. La soluzione sono le **Wildcard** (`?`), che fungono da "permessi speciali": ti ridanno libertà di mescolare i tipi "parenti", a patto di promettere al compilatore se userai la collezione **solo per leggere** i dati (usando `? extends`) o **solo per scriverli** (usando `? super`). Così facendo, ottieni un codice flessibile senza mai sacrificare la sicurezza.

Covarianza, Controvarianza e Invarianza

- **Covarianza (Array):** La compatibilità di tipo dell'array varia nello stesso senso di quella degli elementi. È sicura solo in lettura, ma causa disastri in scrittura.
- **Controvarianza:** Sarebbe la compatibilità in senso opposto. Una funzione che scrive interi in un array dovrebbe poter accettare un `Number[]` (sicuro semanticamente), ma gli array Java non lo permettono.
- **Invarianza (JCF):** Per risolvere i problemi degli array, la JCF è stata progettata con strutture dati **invarianti**. Una `List<Integer>` NON è compatibile con una `List<Object>`, bloccando l'errore a tempo di compilazione.

<T>

tipo *invariante*

<? super T>

tipo *controvariante* rispetto a T (*lower bound*)

- si usa per i tipi-collezione in cui si devono *inserire*, *aggiungere*, "*scrivere*" elementi di tipo (al più) T

<? extends T>

tipo *covariante* rispetto a T (*upper bound*)

- si usa per i tipi-collezione da cui si devono *estrarre*, *togliere*, "*leggere*" elementi di tipo (almeno) T

<?>

tipo *unbounded*

- si usa per i tipi-collezione sui cui elementi non si devono fare ipotesi (tipo sconosciuto): in tali collezioni non si possono né scrivere, né leggere elementi di tipo T
- è uno shortcut per **<? extends Object>**

Risolvere la rigidità dell'invarianza: Il principio PECS

L'invarianza garantisce la massima sicurezza, ma porta a un'eccessiva rigidità. Se vogliamo copiare elementi da una `Collection<T>` sorgente a una destinazione, costringere entrambe ad essere esattamente dello stesso tipo `T` è limitativo. Immaginando la copia come un "tubo":

- La **sorgente** (che produce elementi) può tranquillamente essere più *specific*a (covariante).
- La **destinazione** (che consuma elementi) può tranquillamente essere più *generale* (controvariante). Questo porta alla regola aurea **PECS (Producer Extends, Consumer Super)**.

I tipi parametrici varianti in Java: Le Wildcard

Java offre la *use-site variance* (si specifica la varianza nel singolo metodo) tramite le **wildcard** (tipi parametrici varianti).

- `<? extends T>` (**Upper Bound / Covarianza**): Usato per i *Producers*. Accetta `T` o i suoi sottotipi. È sicuro estrarre (leggere) elementi, ma NON si può scrivere (inserire).
- `<? super T>` (**Lower Bound / Controvarianza**): Usato per i *Consumers*. Accetta `T` o i suoi supertipi. È sicuro inserire (scrivere) elementi, ma NON si può leggere assumendo un tipo specifico.
- `<?>` (**Unbounded**): Usato quando il tipo è sconosciuto. Equivale a `<? extends Object>`. Accetta qualsiasi collezione, ma non permette modifiche al contenuto. Non va confuso con il *diamond operator* `<>`.

Esempio del refactoring di `copy`

Utilizzando le wildcard, il metodo di copia diventa flessibile e "type-safe":

```
1 public static <T> void copy( Collection<? extends T> src, Collection<? super T> dest) {
2     for (T elem : src) dest.add(elem);
3 }
```

La sorgente fornisce tipi "almeno pari a T", la destinazione accetta tipi "più grandi di T".

Compatibilità fra tipi varianti

I tipi contenenti wildcard hanno regole di compatibilità insiemistiche basate sulla tassonomia. Ad esempio:

- `List<? extends Double>` è un vincolo più restrittivo di `List<? extends Number>`. Quindi, il primo può essere assegnato al secondo, ma non viceversa.
- `List<? super Number>` accetta solo collezioni di `Number` e `Object`.

Sviluppare Classi Generiche

Quando si crea una nuova struttura dati, bisogna decidere la varianza in base al suo scopo:

- `MyContainer<T>` (**Sia Producer che Consumer**): Poiché ha metodi `get()` (legge) e `put()` (scrive), **deve** essere invariante.
- `MyProducer<T>` (**Puro Producer**): Fornisce solo dati tramite `get()`. Sarebbe concettualmente covariante.
- `MyWriter<T>` (**Puro Consumer**): Riceve solo dati tramite `put()`. Sarebbe concettualmente controvariante.

Limiti di Java (Use-site Variance)

Java **non** permette di dichiarare un'intera classe come covariante o controvariante (la cosiddetta *declaration-site variance* non è supportata). Le classi personalizzate come `MyProducer<T>` nascono invarianti.

Per sfruttare la varianza in Java, bisogna applicare le wildcard al momento dell'utilizzo (use-site variance):

```
1 // Non si può dichiarare la classe covariante, ma la si usa in modo covariante:
2 static void testProducer(MyProducer<? extends Number> producer) {
3     System.out.println(producer.get());
4 }
5
6 static void testWriter(MyWriter<? super Integer> writer) {
7     writer.put(18);
8 }
```

MyStack

Se creiamo una classe `MyStack<T>` con metodi `push(T)` e `pop()`, questa è invariante. Due stack di tipi diversi sono totalmente incompatibili (`stack1 = stack2` non compila).

Tuttavia, se aggiungiamo un metodo per travasare elementi da un altro stack, possiamo (e dovremmo) usare le wildcard per non limitarlo inutilmente:

```

1 // Senza wildcard: Accetta solo uno stack esattamente dello stesso tipo
2 // public void moveFrom(MyStack<T> source)
3
4 // Con wildcard (Applicazione della covarianza in lettura):
5 public void moveFrom(MyStack<? extends T> source) {
6     while(!source.isEmpty()) push(source.pop());
7 }

```

Questo permette a uno Stack di Number di prelevare elementi, ad esempio, da uno Stack di Integer.

Numeri Reali

La sensazione che i calcoli siano perfetti è dovuta alle funzioni di Input/Output di Java (come `System.out.println()`), che arrotondano i valori per "non spaventare" l'utente.

*Esperimento pratico: **Se dichiari** `double val1 = 0.1;` **e** `double val2 = 0.3;`, **Java stamperà esattamente 0.1 e 0.3.**

Tuttavia, se esegui `double d = val1 + val1 + val1;`, **il risultato** `d` non sarà uguale** a `val2` (`d != val2`).

Stampando la differenza (`d - val2`), Java rivela un errore residuo pari a `5.551115123125783E-17`.

Precisione: `float` **vs** `double`

In Java, la scelta del tipo di dato determina le cifre significative disponibili a causa dei bit dedicati alla mantissa.

- `float` (**Precisione Singola**): La mantissa ha 24 bit, garantendo un errore relativo che permette di avere circa **7-8 cifre decimali significative**.
- `double` (**Precisione Doppia**): La mantissa ha 53 bit, garantendo circa **16-17 cifre decimali significative**.

Questo è il motivo per cui `System.out.println(1/3.0F)` (nota il suffisso `F` per i `float`) stampa 8 cifre (`0.33333334`), mentre `System.out.println(1/3.0)` ne stampa 16 (`0.3333333333333333`).

Spiare la Memoria della Virtual Machine

Java mette a disposizione le classi wrapper (`Float` e `Double`) per estrarre i bit esatti.

Tipo di dato	Metodo per estrarre i bit	Tipo restituito
<code>float</code>	<code>Float.floatToIntBits(f)</code>	<code>int</code> (32 bit)
<code>double</code>	<code>Double.doubleToLongBits(d)</code>	<code>long</code> (64 bit)

Come visualizzarli: Una volta convertito il valore, puoi usare `Integer.toBinaryString(internalRep)` per ottenere una stringa e visualizzare esattamente il bit di segno, l'esponente e la mantissa.

Tipi di Errori

Errore di Incolonnamento (Alignment Error)

Si verifica quando si sommano numeri con ordini di grandezza molto distanti: il numero più piccolo viene "de-normalizzato" perdendo cifre.

- Esperimento:** Se dichiari `double val1 = 12345.0;` (5 cifre) e `double val2 = 1e-13;`, l'operazione `val1 + val2 == val1` risulterà **vera**.
- Motivo:** Il numero richiede 18 cifre totali per essere rappresentato, ma il `double` arriva al massimo a 16-17 cifre significative. Il valore di `1e-13` viene perso durante l'incolonnamento. Con `val2 = 1e-12`, invece, si è al limite e il calcolo va a buon fine.

Errore di Cancellazione

Avviene quando si sottraggono due numeri vicini fra loro che in precedenza erano già stati affetti da errore di troncamento.

- Esperimento:** Matematicamente, $1/3 - 1/4$ è uguale a $1/12$.
 - Su JShell, `System.out.println(1/3.0 - 1/4.0)` restituisce `0.08333333333333331`.
 - Invece, `System.out.println(1/12.0)` restituisce `0.08333333333333333`.
- Controesempio:** Se nessuno dei due operandi iniziali è stato troncato (es. potenze di 2 al denominatore), il problema non si manifesta. `System.out.println(1/4.0 - 1/64.0)` dà un risultato esatto senza errori di cancellazione.

Accumulazione degli Errori in Loop

Gli errori si accumulano portando a risultati assurdi, specie se usi precisioni non adatte in cicli iterativi lunghi.

- Esperimento di Euclide per il Pi Greco:** Utilizzando i `float` per calcolare il Pi Greco, con una precisione richiesta di `float eps = 1E-4F`, il ciclo termina correttamente fornendo un'approssimazione accettabile.
- Se si forza la precisione a `eps = 1E-8F` (il limite della precisione singola), gli errori si accumulano in modo catastrofico.
- Al crescere dei lati del poligono (`nlati`), i valori sballano fino ad arrivare a `n1 = 32768.0`, dove la lunghezza del lato calcolata diventa `0.0` e di conseguenza **il Pi Greco calcolato da Java crolla a 0.0**.